

班 级 2016021
学 号 20009100883

西安电子科技大学

本科毕业设计论文



题 目 开源鸿蒙操作系统的高效任务调度策略

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 魏子捷

导 师 姓 名 韩高

西安电子科技大学

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《开源鸿蒙操作系统的高效任务调度策略》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者： 魏子捷 （签字） 时间：2024 年 5 月 31 日
指导教师已阅： 韩磊 （签字） 时间：2024 年 5 月 31 日

西安电子科技大学

毕业设计（论文）任务书

学生姓名 魏子捷 学号 20009100883 指导教师 韩高 职称 讲师

学院 空间科学与技术学院 专业 空间科学与技术

题目名称 开源鸿蒙操作系统的高效任务调度策略

任务与要求

在无人系统集群协同执行复杂任务时，调度系统的挑战在于实时协调众多无人机的通信、目标识别和数据交互，同时高效管理电池、传感器和计算资源。虽然鸿蒙的轻量级内核 LiteOS 支持实时任务调度，但在面对高并发和复杂任务的实时响应时，调度器可能由于庞大的任务量，导致处理器在高负载的并发任务下进行任务调度以及资源分配的问题上的实时性能大大降低。为此，研究将围绕如何利用鸿蒙内核的轻量化优势，实现任务的快速响应和优先级管理，同时处理优先级反转问题。此外，调度策略将包括确定性分析，确保系统的行为可预测。模拟和实地测试的实施，将针对鸿蒙内核特性，评估调度算法性能。具体指标要求如下：

(1) 平均任务切换时间： $< 500 \text{ us}$;

开始日期 2023. 11. 27 完成日期 2024. 5. 31

院长（签字）金科 2023 年 11 月 3 日

注：本任务书一式两份，一份交学院，一份学生自己保存。

西安电子科技大学

毕业设计（论文）工作计划

学生姓名 魏子捷 学 号 20009100883

指导教师 韩高 职 称 讲师

学 院 空间科学与技术学院 专 业 空间科学与技术

题目名称 开源鸿蒙操作系统的高效任务调度策略

一、毕业设计（论文）进度

起止时间	工作内容
2023.12.1-2024.1.30	理解课题的具体含义以及要求，安装 openharmony 系统安装环境，并学习如何利用该环境进行开发；完成英文文献的翻译；查阅相关资料，并进行前期的知识储备。
2024.2.1-2024.3.15	完成基本代码的编写，并测试其在具体工作场景下的系统实时性。
2024.3.16-2024.4.15	对基本代码进行更进一步的完善，进一步优化系统的实时性，并测试优化后系统的实时性。
2024.4.16-2024.5.30	完成论文的撰写并修改论文

二、主要参考书目（资料）

1. 鸿蒙操作系统官方文档：网址

<https://developer.harmonyos.com/cn/develop/>

2. 鸿蒙内核源代码：通过访问鸿蒙开源代码仓库（<https://gitee.com/openharmony>）获取相关代码

3. 技术论坛和社区的相关博客：Github,Stackoverflow 等

4.HarmonyOS: A Lightweight and Distributed Operating System for IoT" -发表于 2020 年的 ACM Transactions on Embedded Computing Systems。

5.Distributed Task Scheduling Mechanism for Resource-Constrained IoT Devices in HarmonyOS" -发表于 2021 年的 International Conference on Innovative Trends in Computer Engineering。

6.余兵,黎忠文.Linux 操作系统实时性分析[J].计算机技术与发展,2007,17(9):41-44.

7.陈万桥,杜承烈,江南.操作系统实时性指标研究与测试结果分析[J].航空计算技术,2014,44(01):109-112.

8.郝英华,王瑞,李晓娟.Nuttx 操作系统进程切换调度实时性的形式化验证[J].计算机应用与软件,2018,35(07):15-20+51.

9.李孝成.SylixOS 大型实时操作系统实时性分析[J].单片机与嵌入式系统应用,2021,21(04):8-11.

10.王远鹏.提高实时操作系统的实时性能和可靠性策略[J].计算机光盘软件与应用,2012,15(16):134+136.

- 11.任慰.以实时操作系统为中心的嵌入式系统平台化设计研究[D].华中科技大学,2013.
- 12.周洁,李正凡.嵌入式实时操作系统的调度策略[J].华东交通大学学报,2005,(04):110-112+133.
- 13.丁仕林, 吴强.嵌入式实时系统 $\mu\text{C}/\text{OS-II}$ 任务调度算法分析及改进[J].电子技术应用,2009,35(01):20-23+27.2009-4-29
- 14.薄紫彤,乔颖,冷昶等. 基于深度强化学习的实时调度策略[C].中国科学院软件研究所学术年会, 2021.
- 15.张文越.基于 Min-Min 和 Max-Min 算法改进的网格调度算法的研究[D].北京工商大学,2010.
- 16.Augonnet C, Thibault S, Namyst R. StarPU: a runtime system for scheduling tasks over accelerator-based multicore machines[D]. INRIA, 2010.
- 17.Meng Q, Humphrey A, Schmidt J, et al. Investigating applications portability with the uintah dag-based runtime system on petascale supercomputers[C]//Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. 2013: 1-12.
- 18.Etminani K, Naghibzadeh M. A min-min max-min selective algorithm for grid task scheduling[C]//2007 3rd IEEE/IFIP International Conference in Central Asia on Internet. IEEE, 2007: 1-7.
- 19.Sharma N, Tyagi S, Atri S. A Comparative Analysis of Min-Min and Max-Min Algorithms based on the Makespan Parameter[J]. International Journal of Advanced Research in Computer Science, 2017, 8(3).
- 20.Parsa S, RASA R E M. A New Grid Task Scheduling Algorithm[J].

International Journal of Digital Content Technology and its Applications, 2009, 3(4).

三、主要仪器设备及材料

安装有 openharmony 开发环境的电脑，后期实时性测试场景的模拟等。

四、教师的指导安排情况（场地安排、指导方式等）

答疑时间：线上答疑，线下每周固定时间答疑等

答疑场地：西安电子科技大学 G216 办公室

实验场地：自己通过电脑进行代码编写

指导方式：每周一次组会布置当周任务并跟进进度；并根据实际情况进行引导，解答代码编写中遇到的困难

五、对计划的说明

注：1.本计划一式两份，一份交学院，一份学生自己保存（计划双面打印）。

2.各学院可根据本学院专业认证要求及细则对表格进行修改，修改后的表格需向本科生院备案。

西安电子科技大学 空间科学与技术 学院

本科生毕业论文（设计）开题报告

（ 2024 届）

学生姓名 魏子捷

专 业 空间科学与技术

学 号 20009100883

指导教师 韩高

2023 年 12 月 28 日

（本表一式三份，学生、指导教师、学院各一份）

一、论文名称及项目来源

开源鸿蒙操作系统的高效任务调度策略

项目来源：科研项目

二、研究目的和意义

智能无人系统在诸多领域如自动驾驶、工业自动化、无人航空器等取得了显著的应用进展，这些系统对实时性能的要求极为严格。高强度实时性的嵌入式系统要求在毫秒的时间做出决策，从而减少了潜在的事故风险，并提高系统的整体效率。特别是，随着系统的智能化程度不断提高，无人飞行器和自动驾驶车辆需要更强大的计算能力和决策制定能力。这就需要深入研究和开发高性能的嵌入式智能系统，以处理大规模数据、实时感知和复杂的决策任务。

开源鸿蒙（OpenHarmony）凭借其开源性、模块化设计、以及对跨平台兼容性，提供了一个在多种设备和平台上运行的强大基础。这一优势使其成为理想的智能无人系统平台，尤其在需要高度可靠性和实时响应的场景中。通过将鸿蒙操作系统的内核优化为一个高度可靠和响应的实时系统，将极大地推动无人技术的革新，并为相关产业带来变革。这不仅能够加强现有设备的性能，还能够为未来的技术发展铺平道路，如支持更加复杂的无人系统和机器人应用，促进智能城市 and 智能交通的实现。此外，还将加速开源生态系统的成长，吸引全球开发者和企业共同参与到系统的开发和完善中来。这样的合作将推动创新，激发新的商业模式和服务，最终促进整个社会的数字化转型。

但是，在无人系统集群协同执行复杂任务时，调度系统的挑战在于实时协调众多无人机的通信、目标识别和数据交互，同时高效管理电池、传感器和计算资源。虽然鸿蒙的轻量级内核 LiteOS 支持实时任务调度，但在面对高并发和复杂任务的实时响应时，庞大的任务量，导致调度处理器在高负载的并发任务下进行任务调度以及资源分配的问题上的实时性能大大降低。在此过程中，如何平衡进程调度器的性能和系统开销，确保任务能够在严格的时间限制下可靠执行，是一个亟待克服的难题。

三、国内外研究现状和发展趋势

国内研究现状：

1. 2007 年, 厦门大学信息科学与技术学院的余兵等教授对 Linux 内核进程调度器的工作原理进行了深入分析, 并阐述了调度延迟是造成 Linux 内核实时性不强的原因, 然后介绍通过可抢占机制对 Linux 内核进行改造, 测试了改进后的内核的实时性.

2. 2014 年, 西北工业大学计算机学院的陈万桥, 杜承烈, 江南三位教授研究了操作系统的实时性能指标及其影响因素, 以常用的实时操作系统与通用操作系统作比较, 通过编写测试用例和测试程序, 使用基于硬件的 us 级计时器获得时间参考, 测试影响系统实时性相关的参数, 并通过对比 Windows 和 RTX+ Windows 的实时指标, 评估系统的实时性能, 结果为数值分析和研究报告. 该实时性测试方法可以使用户在特定的领域选择适用的实时操作系统.

3. 2017 年, 电子科技大学樊珍的硕士毕业论文研究了携带不同资源类型的多类型无人机实时任务分配技术, 其目的是提供一种安全系数高、突防概率大的任务分配方案, 使无人机系统的整体作战效能最佳.

该文将任务分配分为两个阶段进行研究, 第一阶段是执行任务前根据全局信息进行的任务预分配, 第二阶段为任务执行过程中根据实时环境的变化对分配方案的动态调整, 并对可能引起的冲突问题进行协调.

4. 2018 年, 首都师范大学的郝英华, 王瑞, 李晓娟三位教授, 提出了对 Nutt X 实时操作系统的三种任务调度方式: 先来先服务, 时间片轮转法和零星调度进行切换, 使得当任务到来时, 能够根据自己的特点选择不同的调度策略, 并在模型检测工具自动机中进行验证, 并将研究成果发表在《计算机应用与软件》杂志中 (2018 年 7 期)。

5. 2019 年, 北京航天长征飞行器研究所的李苗伟, 冯东对航天星载计算机实时性需求进行了分析, 提出了一个对实时性进行评测的体系, 并根据其中的各个指标, 以 SPARC 架构为基础的 S689-MIL 嵌入式平台评测了航天领域中较为典型的嵌入式操作系统. 结果表明, 该评价体系能全面评价和分析嵌入式操作系统的实时性, 并能为选择合适的嵌入式操作系统提供更加真实、可靠的决策. 研究成果发布于 2019 年第 23 期《科技创新导报》中.

6. 2020 年至 2021 年, 南京某信息技术有限公司的李孝成研究员对影响 Syilx OS 实时性的因素进行了分析, 并给出优化方法, 最后给出了 Syilx OS 和 VxWorks

两个 OS 的测试对比。本研究成果发布于 2021 年第 4 期《单片机与嵌入式系统应用》杂志中。

国外研究现状：

1. 2010 年 5 月, Samuel Thibault, Raymond Nays 两位教授在《哈尔科学》杂志上发表文章, 提出了用于在基于加速器的多核机器上面调度任务的运行时系统 Star PU。

2. 2012 年, Qing Yu Meng, Alan Humphrey, Martin Berzins 三位教授发表论文描述了一种新的统一、多线程运行时系统的开发, 用于在异构系统上执行异步任务。其源于 Uintah 框架, 该框架旨在为解决结构化自适应网格上的一类耦合问题提供环境。论文被 IEEE Xplore 收录。

3. 2016 年, Marc Sergent, David Goldin, Samuel Thibault, Oliver Aumage 四位教授研究了基于任务的分布式应用程序代码与运行时系统引擎之间的合作, 以在整个执行过程中控制内存订阅级别。该论文被 IEEE Xplore 收录。该论文研究成果表明, 任务范式允许通过限制任务提交流速来控制应用程序的内存占用, 从而在预期任务提交的性能优势和由此产生的内存消耗之间达成折衷。该论文阐述了该论文成果应用于压缩密集线性代数分布式应用程序上的贡献的好处。

4. 2018 年, Alexey Ilyushkin, Dick Edema 两位教授在 IEEE Xplore 上发表文章, 研究了七种调度策略: 其中四种是文献中流行的 (批量) 工作负载的现有策略, 包括一种不知道任务运行时估计的简单回填策略, 两种是新的面向工作负载的策略, 包括一种针对公平性的策略, 一种是针对适应在线工作负载场景的单个工作流的众所周知的 HEFT 策略。他们模拟了同构和异构分布式系统, 以评估这些策略在任务运行时估计的不同准确性下的性能。结果表明, 对于高利用率, 工作流的处理顺序比了解正确的任务运行时估计更重要。在低利用率下, 所有考虑的策略都显示出良好的结果, 即使是不使用任务运行时估计的策略也是如此。他们还表明, 他们的公平工作流优先级 (FWP) 策略有效地减少了工作流速度减慢的方差, 从而实现了公平性, 并且从 HEFT 派生的基于计划的调度策略并没有显示出太大的性能改进, 同时给调度过程带来了额外的复杂性。

4. 2021 年, David Alvarez, Kevin Sara, Marcos Madronas, Vincent Beltran, Aleix Roca 五位教授分析了基于任务的运行时系统可扩展性的限制因素, 并针对

每一个挑战提出了单独的解决方案，包括无等待依赖系统和基于委托的新型可扩展调度处理器设计。他们还评估了单独还是组合两种优化如何影响运行时的整体性能。他们还将生成的运行时与最先进的 OpenMP 实现进行了比较，显示出同等或更好的性能，尤其是对于细粒度任务。

四、主要研究内容、要解决的问题及本文的初步方案

1. 主要研究内容：对于鸿蒙操作系统的任务调度算法进行优化，从而提高实时性，并将优化好的鸿蒙操作系统应用于无人机控制中。

（1）用基于市场机制的合同网的协议方法，通过模拟市场活动中的招标、投标、中标、签约过程，并选择合适的策略对招标范围、投标数目、招标者职责进行完善，实现突发状况中无人机任务调度算法和优先级的动态调整，并对可能引起的冲突问题进行协调

（2）应用机器学习相关算法，进行任务进程的预分配，并通过神经网络模型根据已知条件选择最合适的优先级顺序和调度算法的选择

2. 要解决的问题：

根据任务的优先级，任务的规模，以及任务的执行具体环境等全局信息，进行算法的设计，使其具有对不同任务进程的调度算法，以及任务进程的优先级进行动态切换和调整的能力，并可以实现最大调度延迟时间的缩短以及任务进程优先级反转。

3. 本文的初步方案：

（1）进行对开源鸿蒙操作系统任务调度算法代码的研究；

（2）针对无人机场景下任务调度算法进行优化，并编写完善代码；

（3）最后将优化好的 OS 平台应用于无人机控制平台进行测试。

五、工作的主要阶段、进度和完成时间

本毕业设计项目分为三个阶段（形成逻辑闭环）

1. 调研阶段，阅读文献及参考资料，确定对无人机控制算法进行优化的方案

完成时间 2024. 1. 31

2. 针对无人机场景任务，进行算法优化代码的编写

完成时间 2024. 3. 31

3. 进行测试，测试算法的实时性能

完成时间 2024. 4. 15

4. 进行论文的撰写及修改工作

完成时间 2024. 5. 15

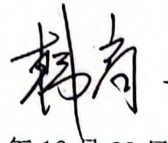
六、已进行的前期准备工作

前期已经进行了文献的查找和部分文献的阅读，国内外研究现状的调研，工作大方向的确定，进行基础知识的学习，环境配置等工作。

七、指导教师意见

开题报告合格，可开始进行正式毕业设计

签名



2023 年 12 月 28 日

八、学院审核意见

通过

签名



2023 年 12 月 28 日

学号 20009100883

西安电子科技大学
本科生毕业设计（论文）中期报告

题 目	开源鸿蒙操作系统的高效任务调度策略
学 生 姓 名	魏子捷
专 业	空间科学与技术
学 号	20009100883
指 导 教 师	韩高
报 告 日 期	2024.3.19

西安电子科技大学

本科生毕业设计（论文）中期报告要求

一、本科生在完成学位论文开题之后三个月内，必须进行学位论文中期考核，考核由各学院自行组织，具体要求参照毕业设计文件执行。

二、中期考核结论分为两种：1. 通过，按专家意见修改后继续学位论文撰写工作；2. 不通过，重新考核，正式答辩前达不到通过标准的，答辩延期进行。

三、中期报告由学生填写，填写完成后，需在限定时间内，在教务系统中上传最终版（如有更新，可重新上传覆盖）。

四、中期考核时需携带此表，本表一式三份，本人、指导教师、学院各一份，用 A3 纸张正反套印；承担毕业设计单位审核盖章后的表格最终胶装入存档论文中。

五、表格填写要求：正文字体宋体，字号小四，行间距固定值 20 磅，可续页，请勿更改表格样式。

1、毕业设计工作是否更换题目及是否按开题报告预定的内容及进度安排进行
未更换题目

基本按照开题报告进度安排进行

2. 目前已完成的研究工作及结果（内容要详实充分）

（一）理论基础学习

（1）宏内核与微内核

宏内核（如 Linux）存在容错性差，可靠性和安全性较低的问题，故如 Linux 这样的操作系统内核无法胜任如无人机飞行控制这样对安全性要求较高的任务。

微内核（例如本课题应用到的开源鸿蒙操作系统 LiteOS）具有较高的可靠性和安全性，它虽然也具有一定的实时性，但是面对高并发和复杂任务的实时响应时，庞大的任务量，导致调度处理器进行任务调度以及资源分配的问题上的实时性能大大降低。

（2）开源鸿蒙操作系统的任务调度算法简述

1. 优先级调度算法：

优先级调度算法的核心思想是根据任务的优先级来确定进程上 CPU 运行的顺序。在开源鸿蒙操作系统中，我们为每一个进程赋予一个数字，用这个数字来代表进程的优先级，通常数字的值越大，进程的优先级越高。进程的优先级可能是静态设定的，也可以动态调整，比如根据进程类型，用户请求，系统资源需求，进程等待时间，进程执行的紧急程度等因素确定。

开源鸿蒙操作系统的优先级调度算法采用抢占式调度策略。当一个较高优先级

进程由阻塞态或初始态转换为就绪态时，正在执行的低优先级进程会立即下 CPU 运行，较高优先级进程获得 CPU 资源。

开源鸿蒙操作系统会维护多个优先级队列，相同优先级的进程会储存于同一队列中。每次调度从最高优先级队列中调度一个进程上 CPU 执行。

2. 先来先服务算法：

先来先服务算法的原理是，当一个进程变为就绪态时，其同时也会进入队列，每次调度队列中最先进入等待的线程执行。先来先服务算法为非抢占式调度算法。

3. 时间片轮转算法：

时间片轮转法的原理是，在同一队列进程优先级相同的情况下，定义一个时间段，我们称之为时间片，每个进程每次只能上 CPU 执行一个时间片的时间，队列中每个进程轮流上 CPU 执行。时间片轮转算法是一种抢占算法。

4. 短作业优先算法：(SJF)

SJF 算法，也称为最短作业优先调度算法，是一种在操作系统中用于进程调度的策略。该算法基于以下原则：当 CPU 从一个进程切换到另一个进程时，系统会选择当前就绪队列中预计执行时间最短（或剩余执行时间最短）的进程来分配 CPU。如果有两个或多个进程具有相同的最短 CPU 时间，则可能采用先来先服务原则，即按照它们进入就绪队列的顺序进行调度。

（二）开发环境的配置：

本毕业设计研究所使用的代码为开源鸿蒙操作系统源码，源码下载于 VirtualBox 虚拟机上，虚拟机操作系统为 Ubuntu 20.04。我们使用主机上的

VS Code 编写代码，故我们需要建立主机与虚拟机间的通信，使用 ssh-key 进行主机与虚拟机的连接。

（三） 代码的编写：

（1） 内核增加 SJF 调度算法

Step1:

kernel\liteos_a\kernel\base\include\los_task_pri.h 中,TCB 结构体字段添加 predict_time 字段，记录执行时间

Step2:

kernel/liteos_a/kernel/base/core/los_process.c 中，定义 SJF 调度算法的执行时机

Step3:

kernel\liteos_a\kernel\base\include\los_process_pri.h 中，添加该调度策略的宏

Step4:

kernel\liteos_a\kernel\base\include\los_task_pri.h 中，添加 SJF 调度算法实现对应的宏

Step5:

kernel/liteos_a/kernel/base/include/los_priqueue_pri.h 中，添加 SJF 实现函数的声明

Step6:

kernel/liteos_a/kernel/base/sched/sched_sq/los_priqueue.c 中，编写

SJF 算法实现代码

(2) TCB 入就绪队列

1. 放在对应优先级队列的头部:

- (1) 时间片轮转策略并且时间片没用完
- (2) FIFO 策略并且是运行态，即刚从 CPU 上下来

2. SJF 代码:

在创建的时候根据它的预测时间将其放到合适的位置;
如果是运行完的 TCB 也会再次根据它的预测时间来重新插入就绪队列。
如果不被发生抢占的情况，当前预测时间最短的会回到头部，因此会重新又一次上 CPU，直到其被运行完。

(3) SJF 算法的具体实现

如果该优先级下这个循环双向链表是空的说明这个优先级下暂时还没有 TCB，直接插入即可
如果找到一个 TCB 的预测时间大于当前待插入 TCB 的预测时间，就需要将待插入 TCB 放在这个 TCB 的前面，使用内核中的插入 TCB 函数即可实现
如果当前循环链表中的预测时间都小于待插入的 TCB，则将这个 TCB 放在最后即可

(4) 编写测试线程

编写三个测试线程，并烧录至 STM32 开发板（目前暂未完成）。

3. 后期拟完成的研究工作及进度安排（要有可行性）

任务	完成时间
将编写好的 SJF 算法代码及测试程序 烧写至 STM32 开发板观察效果	2024.4.1 前

对 SJF 算法(静态优先级算法)改进, 实现动态优先级调度	2024. 4. 20 前
完成论文的撰写	2024. 5. 1 前

4. 存在的困难与问题

目前存在的问题主要为两点:

1. 使用 STM32 单片机进行硬件烧录时出现问题。
2. 对代码进行动态优先级改进存在问题。

以上问题现阶段正在努力解决。

5. 如期完成全部论文工作的可能性

可以完成全部任务

6、指导教师意见

加强对于开源鸿蒙任务调度算法代码原理的理解,探究动态优先级调度算法的实现,并实现算法的硬件移植

导师签名:



2024 年 3 月 21 日

7、中期报告检查组意见

(中期考核结论分为两种: 1. 通过, 按专家意见修改后继续学位论文撰写工作; 2. 不通过, 重新考核, 正式答辩前达不到通过标准的, 答辩延期进行。评语重点指出中期报告存在的问题并提出具体修改意见和建议。)

通过 建议后面加快进度, 有不懂的问题和困难随时与老师沟通

组长签名:

曹明华

成员签名:

李洋 韩海

2024 年 3 月 21 日

8、承担毕业设计单位审核(盖章)

(校内毕设学生由学院审核, 校外毕设学生由承担毕设企业或单位审核)

审核意见:

通过

盖章:

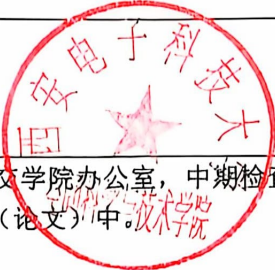


2024 年 3 月 21 日

西安电子科技大学
毕业设计（论文）中期检查表

学生姓名	魏子捷	学 号	20009100883	班 级	2016021
学 院	空间科学与技术学院		专 业	空间科学与技术	
导师姓名	韩高	职 称	讲师	学 院	空间科学与 技术学院
题目名称	开源鸿蒙操作系统的高效任务调度策略				
检 查 内 容		检 查 结 果			
题目是否更换及更换原因		否			
学生出勤情况		良好			
进度评价 (完成总工作量的百分比)		70%，进度良好			
质量评价		优			
总体评价 (按优、良、中、及格、不及格 五档评价)		优			
存在的问题与建议		建议加快进度			
学院审核（盖章）		通过			

注：此表由指导教师填写，与学生填写的中期报告配套，填写完成交学院办公室，中期检查成绩将作为毕业设计（论文）总成绩的一部分；此表装订入毕业设计（论文）中。



西安电子科技大学
毕业设计（论文）成绩登记表

学 院	空间科学与技术学院			专 业	空间科学与技术
姓 名	魏子捷	学 号	20009100883	成 绩	优
题目名称	开源鸿蒙操作系统的高效任务调度策略				
指导教师	韩高		职 称	讲师	
指导教师评语及对成绩的评定意见	（指导教师可从以下几方面对毕业设计（论文）进行评审：1.独立查阅文献及调查论证的能力；2.方案设计与实验技能；3.分析与解决问题的能力；4.工作量、工作态度；5 论文质量；6.创新能力。） 论文主要研究了开源鸿蒙的任务调度策略，作者独立完成查阅和调查资料，设计的任务调度策略具有可行性，编写的代码和仿真体现了作者具有良好的分析和解决问题的能力，工作量符合，论文质量良好，具有创新性。 签名 <u>韩高</u> 2024年5月29日				

评阅人评语及成绩评定意见	(评阅人主要从以下几方面对毕业设计(论文)进行评价:(1)综合运用理论知识和基本技能的能力;(2)分析和解决实际问题的能力;(3)论文写作能力;(4)所解决的问题与论文质量评价。) 论文运用Min-Max, Max, Min理论优化了一种新的调度算法,并进行了验证,论文书写能力较好,能解答论文中提出的问题,论文质量较好。 签名 <u>李博</u> 2024年5月30日
答辩小组意见	答辩通过。 签名 <u>李博</u> 2024年5月30日
学院答辩委员会意见	通过 答辩委员会主任签名 <u>李博</u> (学院盖章) 2024年5月31日

注: 1.学院、专业名均写全称;成绩登记表双面打印。

2.各学院可根据本学院专业认证要求及细则对表格进行修改,修改后的表格需向本科生院备案。

班 级 2016021
学 号 20009100883

西安电子科技大学

本科毕业设计论文



题 目 开源鸿蒙操作系统的高效任务调度策略

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 魏子捷

导 师 姓 名 韩高

西安电子科技大学

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《开源鸿蒙操作系统的高效任务调度策略》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者： 魏子捷 （签字） 时间：2024 年 5 月 31 日
指导教师已阅： 韩磊 （签字） 时间：2024 年 5 月 31 日

摘要

实时操作系统指对于任务响应和切换时间有着严格要求限制的操作系统，因为实时操作系统的这一限制，它被广泛应用于需要快速且准确对大量任务进程进行相应的场景中，例如工业自动化，无人机飞控任务，航天器的导航系统，汽车的自动驾驶等，以减小因为反应不及时导致故障或事故的可能性。在目前的现有的操作系统中，以 Linux 为代表的宏内核的实时性能比较好，同时这类宏内核操作系统也可支持大批量进程的任务调度。但是这样的操作系统存在很多问题。Linux 等宏内核将许多功能模块集中于其内核中，这些任务模块紧密关联；其内核的臃肿也无法使其广泛应用于一些要求使用轻量内核的物联网应用场景中；它的安全性和保密性也比较差，无法应用于对安全性能要求较高的场景中。LiteOS 操作系统作为轻量级内核，具有扩展性强，功能简单，轻量高效，安全性高等特点，凭借这些特点，其被广泛应用于各大物联网应用场景中；同时其代码开源，本文可以对其功能进行改进。但是，LiteOS 操作系统缺乏同 Linux 等宏内核一样的实时性能。

因此本文致力于研究一种高效的任务调度算法来解决这一问题，提高其内核的实时性能。传统的时间片轮转算法可以实现各个进程的公平响应，但是其不支持高优先级的进程优先获得响应，因此本文在目前 LiteOS 现有的时间片轮转算法的基础上，为 LiteOS 添加高优先级抢占任务调度算法，将两个算法结合起来，结合二者的长处。在该算法下，针对相同优先级进程，系统执行时间片轮转调度算法，针对不同优先级，系统执行高优先级抢占式调度算法。最后，本文将改进好的算法代码，移植入 GD32F303 开发板中，并编写代码创建进程，使其在开发板 CPU 上，通过本调度算法进行调度。最后本文通过串口助手，观察本算法的调度效果。实验结果表明，本算法较好的改进了 LiteOS 操作系统内核的实时性能。

关键词：LiteOS 实时性 任务调度 时间片轮转算法 抢占式调度算法

Abstract

A real-time operating system refers to an operating system that has strict requirements on task response and switching times. Because of this limitation of the real-time operating system, it is widely used in scenarios that require fast and accurate response to a large number of task processes, such as industrial automation, UAV flight control tasks, spacecraft navigation systems, automatic driving of cars, etc. , to reduce the possibility of failure or accident caused by untimely response. Among the current operating systems, macro kernels represented by Linux have better real-time performance. At the same time, this type of macro kernel operating system can also support task scheduling of large batches of processes. But there are many problems with such an operating system. Macro kernels such as Linux concentrate many functional modules into their kernels, and these task modules are closely related, and all will be lost if one is lost; the bloatedness of its kernel also prevents it from being widely used in some Internet of Things application scenarios that require the use of lightweight kernels; its security and confidentiality are also relatively poor, and it cannot be used in scenarios that require high security performance. As a lightweight kernel, the LiteOS operating system has the characteristics of strong scalability, simple functions, lightweight and efficient, and high security. With these characteristics, it is widely used in major Internet of Things application scenarios; at the same time, its code is open source, and we can improve its functionality. However, the LiteOS operating system lacks the same real-time performance as macro kernels such as Linux.

Therefore, this paper is committed to researching an efficient task scheduling algorithm to solve this problem and improve the real-time performance of its kernel. The traditional time slice rotation algorithm can achieve fair response for each process, but it does not support high-priority processes to get responses first. Therefore, we add high-priority preemption to LiteOS based on the existing time slice rotation algorithm of LiteOS. The task scheduling algorithm combines the two algorithms and combines the strengths of both. Under this algorithm, for processes with the same priority, the system executes the time slice round-robin scheduling algorithm, and for processes with different priorities, the system executes the high-priority preemptive scheduling algorithm. Finally, we transplant the improved algorithm code into the GD32F303 development board, and write the code to create a process so that it can be scheduled on

the development board CPU through this scheduling algorithm. Finally, we use the serial port assistant to observe the scheduling effect of this algorithm. Experimental results show that this algorithm can better improve the real-time performance of the LiteOS operating system kernel.

Keywords: LiteOS real-time task scheduling time slice rotation algorithm
preemptive scheduling algorithm

目录

摘要	I
Abstract	II
第一章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	1
1.2.1 国内研究现状	1
1.2.2 国外研究现状	2
1.3 研究内容及主要工作	3
1.4 本文的结构安排	3
第二章 开源鸿蒙操作系统 Lite OS 任务调度原理简述	5
2.1 Lite OS 操作系统简介	5
2.2 LiteOS 进程任务相关基本操作	6
2.2.1 任务的结构体定义及初始化	6
2.2.2 任务创建	7
2.2.3 任务删除	8
2.2.4 任务延时	9
2.2.5 任务挂起	10
2.2.6 任务恢复	11
2.2.7 任务各状态的关系与状态转化	11
2.3 任务切换的流程	12
2.4 操作系统基本任务调度方法	14
2.4.1 抢占式调度与非抢占式调度	14
2.4.2 优先级调度算法	14
2.4.3 时间片轮转算法	15
2.4.4 先来先服务算法（FIFO）	16
2.4.5 短进程优先算法	16
2.5 操作系统的信号量机制	17
第三章 开源鸿蒙操作系统高效调度算法的设计	19

3.1 算法设计思路	19
3.2 算法评价指标	20
3.3 算法代码实现	22
第四章 算法的硬件移植实验	25
4.1 GD32F303 开发板简述	25
4.2 开发环境的搭建及代码的烧录	26
4.3 实验方案设计	30
4.3.1 无调度算法参与进行进程任务调度情形	30
4.3.2 基于本文提出的调度算法进行进程任务调度情形	31
4.4 实验结果	33
4.4.1 无调度算法参与进行进程任务调度情形实验结果	33
4.4.2 基于本文提出的调度算法进行进程任务调度情形实验结果	34
第五章 未来研究展望	37
5.1 对测试实验的优化设想	37
5.2 对算法设计的优化设想	37
致谢	39
参考文献	41
附录 A	43

第一章 绪论

1.1 研究背景及意义

在自动驾驶、工业自动化、无人航空器等领域中，系统对于操作系统实时性能有着极高的要求，因为在这些领域中，对于一些较为紧急的情况，系统需要在极短的时间内做出反应和决策，以减少事故风险并提高效率。例如，当一架无人机面临一些情况即将坠毁的情况下，为了避免损失，需要紧急切换至迫降进程；自动驾驶应用领域中，当道路出现状况时，也需要紧急启动减速避障功能来保证乘客的安全。

随着智能化程度的提高，无人飞行器和自动驾驶车辆等应用场景对计算能力和决策制定能力的需求也在不断增加。因此，开发高性能的嵌入式智能系统，以处理大规模数据、实时感知和复杂决策任务，变得尤为重要。而实现一个能够实时处理大量数据，并根据现实的情况实时做出决策判断的智能系统，离不开一个高实时性操作系统的支持。

而 Linux 等操作系统作为一个宏内核操作系统，其功能较为完善强大，实时性能较好，可应用于任务量较大的情况。但是作为实时性较好的代价，其安全性、可靠性等较低。诸如无人机飞控，自动驾驶等领域对于系统的安全性和可靠性等性能要求较高，但是其对于实时性的要求同样也比较高。因此，如何使得一个操作系统内核既具有较高的安全性能和可靠性能，又具有较高的实时性，是本篇论文需要研究的问题。

1.2 国内外研究现状

1.2.1 国内研究现状

国内近些年对于操作系统实时性和任务调度算法的研究，主要有以下几篇论文。厦门大学信息科学与技术学院余兵等教授对 Linux 内核进程调度器的工作原理进行了深入分析，认为非可抢占机制下调度延迟是造成 Linux 内核实时性不强的原因^[1]。这篇论文引入了可抢占机制对 Linux 内核进行改造，经测试内核的实时性得到改进。西北工业大学计算机学院的陈万桥，杜承烈，江南三位教授研究

了操作系统的实时性能指标及其影响因素，并提出了一种评估操作系统实时性能的方法^[2]。他们的工作以常用的实时操作系统与通用操作系统作比较，通过编写测试用例和测试程序，以基于硬件的微秒级计时器为时间参考，测试影响系统实时性相关的参数，并通过对比 Windows 和 RTX+ Windows 的实时指标，评估系统的实时性能。首都师范大学的郝英华，王瑞，李晓娟三位教授，基于 NuttX 实时操作系统，提出了针对其三种任务调度方式：先来先服务，时间片轮转法和零星调度进行切换的方式^[3]。该方式使得当任务到来时，能够根据自己的特点选择不同的调度策略，并在模型检测工具自动机中进行验证。南京翼辉信息技术有限公司李孝成研究员对影响国产 SylixOS 操作系统实时性的因素进行了分析，并给出了优化方法^[4]。南平市消防支队司令部王远鹏根据自己的实践经验，提出了几种实现操作系统实时性能的方式，并重点围绕提升操作系统可靠性提出了几种方案^[5]。华中科技大学任慰的博士毕业论文中提出了一种以实时操作系统为中心的嵌入式系统平台化构想^[6]。周洁，李正凡^[7]和丁仕林，吴强^[8]等人均针对嵌入式实时内核 uc/os-II 的调度算法优化做了研究，并实现了针对算法的优化，使得操作系统内核可以应用于更加复杂的系统。薄紫彤，乔颖等人^[9]的研究成果使用了深度强化学习方法进行了多处理器系统的实时任务调度研究。张文越的硕士学位论文基于 Max-Min 和 Min-Min 算法，改进了操作系统任务调度算法^[10]。

1.2.2 国外研究现状

国外关于实时操作系统的论文主要有这两篇。Samuel Thibault, Raymond Nays 两位教授^[11]在《哈尔科学》杂志上发表文章，提出了一种用于在基于加速器的多核机器上面调度任务的运行时系统 StarPU。^[12]Qing Yu Meng, Alan Humphrey, Martin Berzins 三位教授^[12]发表论文描述了一种新的统一、多线程运行时系统的开发，该系统用于在异构系统上执行异步任务。该系统源于 Uintah 框架，该框架旨在为解决结构化自适应网格上的一类耦合问题提供环境。

国外也有一些论文专门讨论了操作系统的高效任务调度算法。以下几篇论文是有关 Min-Min 和 Max-Min 算法的。Kobra Etminani 和 M.Naghizadeh 两人^[13]在两种传统调度算法 Min-Min 和 Max-Min 的基础上引入了一种新的调度算法，以利用它们的优点，同时弥补它们的缺点。该算法根据资源上任务的预期完成时间的标准差在两种算法之间进行选择。两人在名为 GridSim 的网格模拟器中评估一种

调度启发式算法，即选择性算法。两人还将他们的方法与其两种基本启发法进行了比较。实验结果表明，这种新的启发式方法可以为各种场景带来显著的性能提升。文中描述了两种基于启发式的算法：Min-Min 和 Max-Min，并在 CloudSim 上进行了模拟，然后考虑了生产时间参数进行了基于比较的分析^[14]。文中，作者提出一种新的任务调度算法 RASA^[15]。该算法是通过对两种著名的任务调度算法 Min-min 和 Max-min 的综合研究和分析而构建的。RASA 充分利用了这两种算法的优点并避免了它们的缺点。为了实现这一点，RASA 首先估计每个可用网络资源上任务的完成时间，然后交替应用 Max-min 和 Min-min 算法。在这方面，RASA 采用 Min-min 策略先执行小任务再执行大任务，并采用 Max-min 策略避免大任务执行的延迟并支持大任务和小任务执行的并发。论文作者在网格环境中应用 RASA 来调度独立任务的实验结果证明了 RASA 在以相对较低的完工时间实现调度方面的适用性。

1.3 研究内容及主要工作

开源鸿蒙操作系统（LiteOS）具有工作可靠，安全性高的特点。与此同时，其内核的轻量级和高效性也使得其非常适合应用于无人机的飞行控制平台等物联网设备中，但是 LiteOS 操作系统的实时性还是没有达到一些紧急情况应用的实时性要求。

本文的工作致力于研究一种任务调度算法，可以使得 LiteOS 系统具有较高的实时性能。首先本文对 LiteOS 操作系统相关内容进行研究，其次，本文对于操作系统的几种基本调度算法，例如先来先服务，时间片轮转法，优先级抢占式调度算法等进行讨论。在这几种基本调度算法的基础上，本文将提出一种新的操作系统任务调度算法，该任务调度算法是针对操作系统基础调度算法的优化。

最后，本文尝试进行该调度算法的移植。这里本文使用支持 LiteOS 操作系统的 GD32F303 开发板进行该调度算法的移植与烧录，并使用该开发板进行进程调度实验，通过进程调度实验进行算法性能的测试，并对实时性做出评价。

1.4 本文的结构安排

本毕业论文后续的内容主要分为 4 章。在第二章中，本文对于 LiteOS 操作系

统进行简单介绍，之后本文讨论其进程基本操作（任务结构体定义，创建，删除，延时，挂起，恢复）的实现方式。之后本文对于操作系统进程任务进行切换的方法，几种操作系统基本进程调度算法（FIFO，抢占式与优先级调度，时间片轮转），操作系统的信号量机制等内容进行讨论和分析。在第三章中，本文基于基本任务进程调度算法，提出一种新的进程任务调度算法，这里将讨论算法思路和具体代码实现细节。在第四章中，本文使用 GD32F303 开发板进行算法代码的移植，并设计测试实验，编写测试进程，并观察算法的运行效果。在第五章中，本文对本研究的优化方向和未来的研究方向提出设想。

第二章 开源鸿蒙操作系统 Lite OS 任务调度原理简述

2.1 Lite OS 操作系统简介

开源鸿蒙操作系统是华为研发的一款国产操作系统，该操作系统一经推出，即被广泛的应用于物联网设备中。LiteOS 是开源鸿蒙操作系统的轻量级内核。LiteOS 相较于其他操作系统内核，主要有安全性高，扩展性强，支持多核心架构，应用范围广，开放性和自由度高等特点。

工作可靠，安全性高体现在 LiteOS 作为微内核仅提供基础功能，系统较为简单，系统中各个模块没有复杂的相互依赖的关系，因此其运行出错的概率较低。同时，LiteOS 内核内置了安全保护功能，可有效保护系统数据和设备的安全性。

轻量级及高效性体现在 LiteOS 作为轻量级内核，其内存占用非常小，运行速度可以达到非常快的速度，从而提高该内核的工作效率。这一特点使得其应用于非常多的物联网设备当中。

扩展性强的表现是，因为 LiteOS 提供的功能较 Linux 等宏内核少，若想其实现较为复杂的功能，需要对 LiteOS 内核进行功能的扩展。扩展的功能若出现错误，则仅影响该功能而非 LiteOS 内核本身，这也体现出 LiteOS 的可靠性。

支持多核心架构指 LiteOS 支持在多核处理器上运行，从而提升其工作效率。同时，LiteOS 可兼容多种不同的处理器架构，如 x86，RISC-V，ARM 架构等，适配性更强。

应用范围广表现在 LiteOS 操作系统目前广泛应用于各个领域，特别是对安全性要求较高的应用场景之中，例如无人机飞行控制平台。因为该内核轻量级和高效的特点，它也被广泛应用于智能穿戴，智能家居，智慧城市等物联网应用场景之中。

开放性和自由度指 LiteOS 源代码现已开源，用户可根据自身需求或项目要求对内核源码进行优化。

2.2 LiteOS 进程任务相关基本操作

2.2.1 任务的结构体定义及初始化

任务控制块由一个名为 `LosTaskCB` 的结构体定义，它代表了一个操作系统任务进程。这个结构体包含了这个任务进程所需的各种信息和控制数据，用于操作系统内核管理任务的调度和状态。

该结构体内含 22 个变量，而这 22 个变量即为该任务进程的基本信息。下列为 22 个基本信息变量的详细介绍：

表 2.1 任务控制块结构体各个参数

<code>*stackPointer</code>	任务栈指针，指向任务栈的栈顶
<code>taskStatus</code>	任务状态，用于表示任务的当前状态，如就绪、运行、阻塞等
<code>priority</code>	任务优先级，数值越小，优先级越高
<code>timeSlice</code>	时间片，表示任务每次可以在 CPU 上运行的时间
<code>waitTimes</code>	等待次数，用于统计任务在阻塞状态下的等待次数
<code>SortLinkedList sortList</code>	任务超时排序链表节点，用于将任务按照超时时间排序
<code>startTime</code>	任务开始时间，记录了任务启动的时刻
<code>stackSize</code>	任务栈大小，表示为任务分配的栈空间大小
<code>topOfStack</code>	栈顶指针，它指向当前栈顶的地址
<code>taskID</code>	给进程赋予了一个任务编号 <code>Id</code> ，它唯一标识一个任务
<code>TSK_ENTRY_FUNC taskEntry</code>	任务入口函数，定义了任务执行的起始函数
<code>*taskSem</code>	任务持有的信号量，用于同步任务
<code>*taskMux</code>	导致任务阻塞的互斥锁，用于保护共享资源
<code>arg</code>	任务入口函数的参数，传递给任务入口函数的参数
<code>*taskName</code>	任务名称，用于标识任务
<code>LOS_DL_LIST pendList</code>	就绪队列等链表节点，用于将任务链接到就绪队列
<code>LOS_DL_LIST timerList</code>	任务超时排序链表节点，用于管理任务的超时
<code>EVENT_CB_S event</code>	事件控制块，用于处理事件
<code>eventMask</code>	事件掩码，用于屏蔽不需要的事件
<code>eventMode</code>	事件模式，定义了事件的触发方式
<code>*msg</code>	分给给队列的内存，用于消息队列
<code>errorNo</code>	错误号，表示任务执行过程中的错误代码

2.2.2 任务创建

LOS_TaskCreate()为任务创建函数，LiteOS 中的进程任务创建均会调用该函数接口。以下对其实现过程进行剖析：

在最开始，创建一个新的任务控制块，为其分配内存，并定义一个任务控制块结构体指针，用以保存其任务信息。创建好任务控制块后，系统为其分配其所需其他资源。任务此时处于创建态。

在这之后，将任务阻塞并开始创建进程，这里使用 LOS_TaskCreateOnly()函数进行任务创建，但是该函数没有配置任务状态信息的功能。在任务创建好后，本文定义一个参数 puwTaskID，其作为一个指针，指向该定义好的任务。后面的操作均将通过该指针获取任务控制块的信息。

当任务创建完成时，将任务状态变量（pstTaskCB->usTaskCB）值由阻塞态（OS_TASK_STATUS_SUSPEND）变为就绪态（OS_TASK_STATUS_READY），并进入就绪队列，此时即表明任务可参与调度。

在这之后，系统将根据具体的调度算法原理，从 pstTaskCB 中获取对应的变量（例如抢占式调度需要获取该新创建任务的优先级 priority，时间片轮转法需要获取 timeSlice 变量），若需要立即进行调度，则首先进行任务的调度，然后再返回创建成功（return LOS_OK），若无需立即调度，则直接返回创建成功。

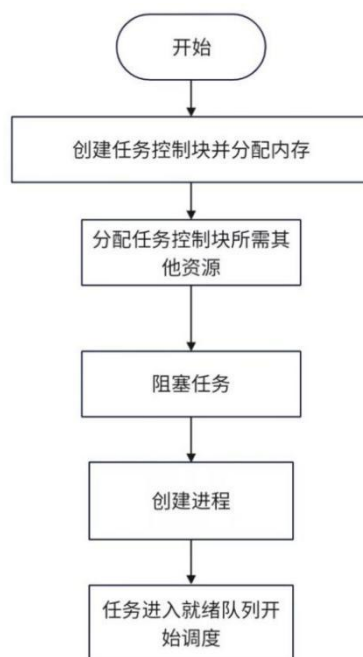


图 2.1 任务创建工作流程

2.2.3 任务删除

LiteOS 操作系统支持显示删除任务，当任务不需要的时候，会对任务进行删除。任务删除所使用的函数接口为 `LOS_TaskDelete()`，以下为对该函数代码进行具体分析：

首先先进行边界条件的排除。当待删除任务的 task 状态为 `OS_TASK_STATUS_UNUSED` 时，此时表明该任务根本就没有被创建，此时任务将无法被删除，因此会返回 `LOS_ERRNO_TSK_NOT_CREATED`，代表任务未被创建。

对于这几种处于特殊状态下的任务删除，本文要对任务进行特殊处理后才可进行删除操作：任务正在 CPU 上运行且调度器已经上锁时，本文需要将调度器解锁，具体操作为将调度器锁变量 `g_usLosTaskLock` 的值设为 0（1 为调度器已经上锁），之后进行删除操作；任务处于就绪态时，将任务从就绪队列中移除后删除；任务处于阻塞态时，将任务从阻塞列表中移除后删除；任务处于延时态时，将任务从延时列表中移除后删除。

因调度过程需当前进程参与，为了保证进程删除后不影响系统的正常运行，对于待删除任务为当前正在 CPU 上运行的任务，本文需要维护一个任务回收列表（`g_stTskRecycleList`），将待删除任务添加至其中，待调度结束后再删除该进程。

最后，将任务控制块插回系统可用任务列表中，清空其内存，并将任务的栈顶指针指向 `NULL`。如果删除成功，返回 `LOS_OK`，否则返回错误代码。

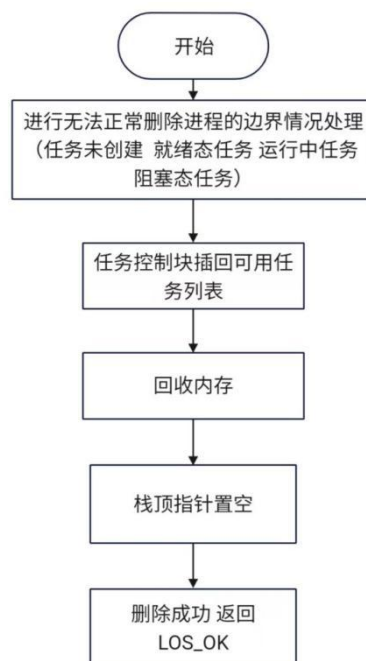


图 2.2 任务删除工作流程

2.2.4 任务延时

当一个高优先级进程长时间占用 CPU 时，一些低优先级进程会产生“饥饿”现象。解决这个问题的方法，除了动态优先级调整，LiteOS 还提供了一种方法，就是使用任务延时函数 `LOS_TaskDelay()`。该函数的作用是，将调用该函数的任务阻塞，同时调度下 CPU，这样使得其他低优先级进程能够得到 CPU 资源。下面为对该函数的具体分析：

首先进行边界条件的判定。若进程处于中断状态，此时将该进程延时非法，系统返回错误代码（`LOS_ERRNO_TSK_DELAY_IN_INT`）。若系统调度器被锁定，此时延时该进程也为非法状态，因为延时操作需要调度器的调度。此时系统也会返回错误代码（`LOS_ERRNO_TSK_DELAY_IN_INT`）。

延时的时间大小通过配置 `LOSCFG_BASE_CORE_TICK_PER_SECOND` 定义，其基本时间单位为 Tick。`LOSCFG_BASE_CORE_TICK_PER_SECOND` 配置为 1000；一个 Tick 为 1ms。如果系统需进行 0 个 Tick 的延时，当前任务将主动调度下 CPU，并进行任务的强制切换。

当进行延时操作时，LiteOS 将调用该延时函数的进程调度下 CPU，系统状态调整为阻塞态（延时状态），并将其挂在延时链表中。当延时时间到达时，系统将该进程恢复为就绪态。当该进程重新回到 CPU 上运行时，此时返回 `LOS_OK`

表示延时成功。

2.2.5 任务挂起

计算机同时支持运行的进程个数是有限的，当操作系统无法支持同时管理诸多进程时，此时操作系统会将多余的进程挂起，挂起的进程除非恢复，否则在任何情况下都无法被调度上 CPU 运行。

在 LiteOS 操作系统中，任务挂起函数通过调用 `LOS_TaskSuspend()` 实现。下面对该函数进行分析：

首先通过作为参数传入该函数的定义好的指定任务 ID (`uwTaskID`) 获取待挂起的任务的任务控制块。

接下来判断不能执行任务挂起操作的情形：若该任务为未使用状态，或者该任务已经被挂起，又或者任务运行中且调度器被锁住，则返回错误代码。

接下来正式进入挂起操作。其代码实现流程为，首先将任务从就绪列表中删除，同时将任务状态变量 `pstTaskCB->usTaskStatus` 的值由就绪态 (`OS_TASK_STATUS_READY`) 改设为挂起态 (`OS_TASK_STATUS_SUSPEND`)。之后进行一次任务调度。

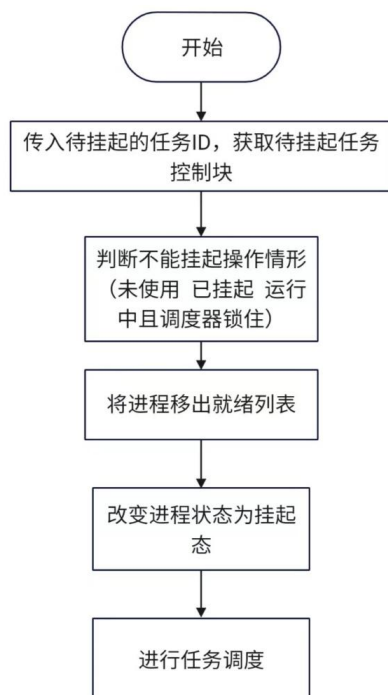


图 2.3 任务挂起工作流程

2.2.6 任务恢复

任务恢复操作和任务挂起操作相对应，作用为将被挂起的函数恢复为就绪态。该操作调用 `LOS_TaskResume()` 实现。下面为对该函数源码及实现过程进行分析：

首先通过作为参数传入该函数的定义好的指定任务 ID（`uwTaskID`）获取待恢复的任务的任务控制块。与挂起函数不同的是，这里需首先判断任务 ID 是否有效，若无效则返回错误代码。

接下来判断不能执行任务恢复操作的情形：若该任务为未使用状态，或者该任务未被挂起，则返回错误代码。

接下来正式进入恢复操作。其代码实现流程为，将任务状态变量 `pstTaskCB->usTaskStatus` 的值由挂起态（`OS_TASK_STATUS_SUSPEND`）改设为就绪态（`OS_TASK_STATUS_READY`）并将其加入就绪队列中，之后该进程参与系统的任务调度。

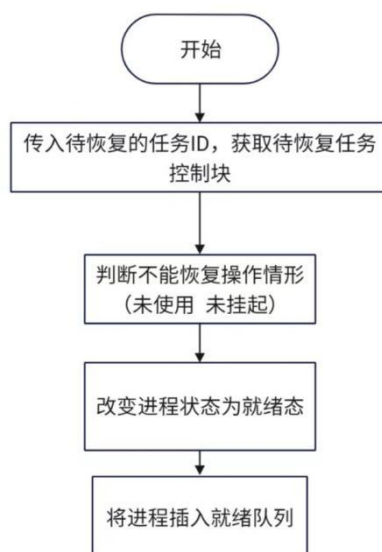


图 2.4 进程恢复工作流程

2.2.7 任务各状态的关系与状态转化

下面这张图展示了系统的 7 种状态，即创建态，退出态，就绪态，运行态，阻塞态，就绪挂起态，阻塞挂起态之间的转换关系。

在创建过程中的进程处于创建态。当创建进程成功后，进程进入就绪态，当需要进行任务的调度时，操作系统根据任务调度算法从就绪队列中选择一个进程上 CPU 进行运行，在 CPU 上运行的进程处于运行态。处于运行态的进程可以转

化为就绪态，例如在时间片轮转法中，时间片用完后运行态进程变为就绪态。当进程任务运行结束后，系统进入退出态，回收进程控制块和堆栈等资源。

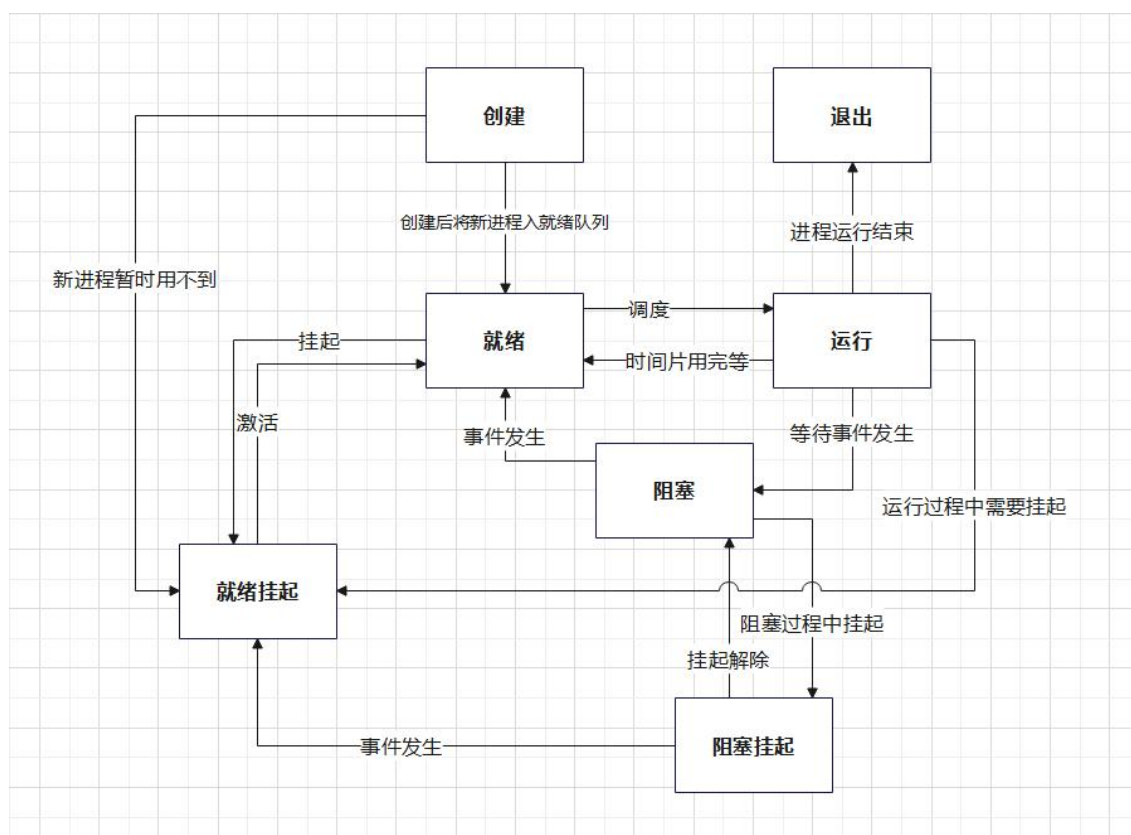


图 2.5 任务各状态的关系与转化

运行态的进程在运行过程中，若需要等待某一事件的发生进程才能继续运行，则进程调度下 CPU，变为阻塞态，插入阻塞队列。当时间发生后，该进程由阻塞态进入就绪态，进程由阻塞队列中被取出，并被插入至就绪队列中。

当某一进程短时间内无需运行时，可将进程挂起。挂起的进程无法参与进程状态的转换，更没有机会调度上 CPU 运行。创建态的进程可在创建完成后，直接进入挂起态；同时处于就绪态和阻塞态的进程也可以通过挂起操作，进入挂起态，其中，处于就绪态的进程挂起后为就绪挂起态，被挂在就绪挂起队列中，就绪挂起态的进程恢复为就绪态；处于阻塞态的进程挂起后为阻塞挂起态，被挂在阻塞挂起队列中，阻塞挂起态的进程恢复为阻塞态。阻塞挂起态中的进程待等待事件发生后，进入就绪挂起状态。

2.3 任务切换的流程

当进行任务调度时，操作系统必然会进行任务的切换。目前本文假设拥有两

个进程，分别设为 P0 和 P1。以下以 P0 切换为 P1 进程为例，简述任务切换的全流程：

在最开始的时候，任务 P0 处于用户态下。当需要进行进程切换时，会触发中断。触发中断后，系统会保存 P0 用户态下的断点和上下文到 P0 的内核堆栈中，保存好以后，P0 进入内核态。进入内核态后，P0 进程在中断向量表中查找自己对应的中断服务程序，并跳转至对应的中断服务程序运行。当进程调度的时机成熟时（例如有高优先级进程需要使用 CPU，P0 时间片刚好用完），操作系统执行调度程序（调度算法是调度程序的核心），并选中就绪进程 P1。

当进行进程切换时，P0 进程的上下文将被保存在 PCB0 中（PCB 为进程的进程控制块），与此同时操作系统会从 P1 的 PCB 中（PCB1）取出 P1 的上下文，包括地址空间和堆栈切换等。最后系统从 P1 的内核堆栈中恢复 P1 用户态下的断点和上下文，之后 P1 进程开始正常运行。

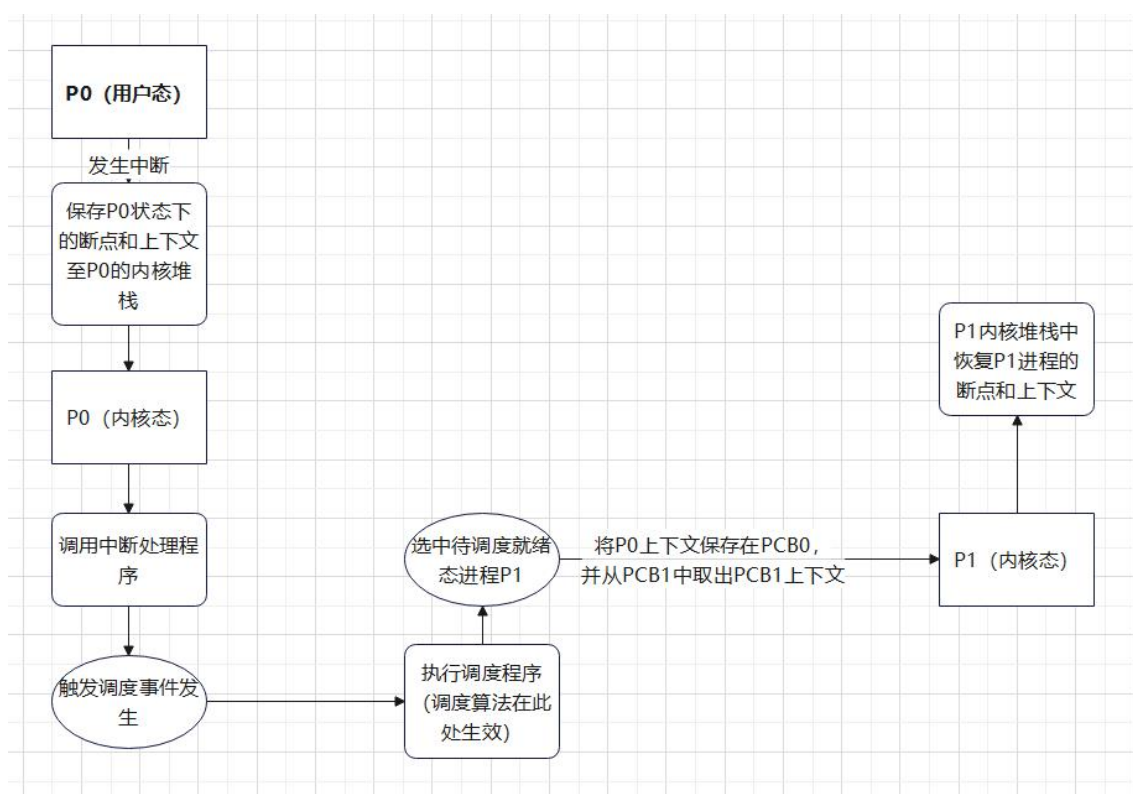


图 2.6 进程切换流程图

2.4 操作系统基本任务调度方法

2.4.1 抢占式调度与非抢占式调度

一个任务进程此时正在 CPU 上运行，这时当一个更高优先级的任务变为就绪态时，即使当前正在运行的任务尚未完成并主动放弃 CPU，调度器也会立即中断当前任务，并将 CPU 控制权转交给优先级更高的任务，而当前任务进程会立即下 CPU，这种调度方式称为抢占式调度。它保证了高优先级任务可以及时得到响应，被广泛应用于实时性要求较高的系统。系统会维护一个任务列表，包括所有任务的 TCB，状态优先级等参数，内核根据该任务列表进行任务的切换。若任务的优先级可以实时调整，则可以动态的提高一个较为紧急的任务的优先级，并将其设置为就绪态，并进行抢占式调度，这样使得较紧急的任务进程可以优先获得 CPU。这样的调度方式有效提高了系统的实时性，使得高优先级任务和紧急任务可以优先被响应，同时也增加了系统的开销和复杂性。

非抢占式调度是指，一个任务开始执行后会持续运行直至其主动让出 CPU 或者进入阻塞状态，不会因为有其他同优先级或较低优先级的任务变成就绪而被中断。

在某些情形下，任务可通过调用 `sleep` 函数等 API 来让出 CPU 资源，之后操作系统会根据任务的状态和优先级调度新的进程上 CPU 运行。对于实时性要求不是很高的场景，这样的非抢占式调度方式可以减小系统的开销和复杂性。

具体实现上，系统通常基于优先级调度算法，通过任务控制块（TCB）来管理各个任务的状态和优先级。调度器维护一个就绪队列，根据优先级进行排序，对于支持抢占的场合，只要检查到有优先级更高的任务进入就绪状态，便会触发上下文切换；而对于非抢占行为，则需要任务自身调用相关 API 显式释放 CPU。

2.4.2 优先级调度算法

当有很多进程正在等待上 CPU 运行，但是只有一个进程可以被调度上 CPU，在这种情况下，进程的优先级决定了进程被调度上 CPU 的先后顺序。每一个进程会被赋予一个优先级变量，优先级较高的进程执行的紧急性较高，因此会被优先调度上 CPU 执行，而优先级较低的进程

优先级的定义一般由一个变量 `priority` 来定义，变量的值代表优先级的大小。

0-31 中的一个数值代表了优先级，数值越小，该进程的优先级越大。

根据分配好的优先级是否会根据外界情况的改变而改变，本文将优先级调度算法分为静态优先级调度算法和动态优先级调度算法。在静态优先级调度算法中，优先级变量值一经赋予无法改变，这样减小了系统的开销，但是其会使得一些优先级较低的进程始终无法上 CPU 运行从而造成“饥饿”现象。在动态优先级算法中，优先级变量值赋予后其值也可随时进行更改，这样提升了系统的灵活性，但是同时也会增加系统的开销。

2.4.3 时间片轮转算法

当操作系统同时需要处理多个进程任务时，需要使用到时间片轮转算法。在时间片轮转算法中，所有进入就绪态的进程会按照达到就绪状态时间的顺序排列在一个队列中，操作系统按照该队列顺序依次调度各进程上 CPU 运行。系统会设置一个固定大小的时间长度，每一个进程每次调度上 CPU 运行，只能够运行这个时间长度大小的时间。时间到了以后，即使该进程任务未完成，也会被调度下 CPU 运行，操作系统会将刚刚被调度下的 CPU 插在队列的队尾。与此同时，队头的任务进程出队并被调度上 CPU 运行，该进程同理也只能运行一个时间片长度大小的时间，当时间到了以后，也会被调度下 CPU 并被插回至队尾。所有进程均同理，如此循环往复，直到每一个任务进程均执行完毕为止。若一个进程时间片未用完但是任务进程已经执行完毕，则该进程也会被调度下 CPU，并不会被插入回任务队列中。

时间片轮转调度算法最大的优点，在于它对于每一个进程都是公平的。每个进程轮流上 CPU 运行相同大小的时间，不存在因优先级低长时间得不到 CPU 资源导致的“饥饿”现象。

在操作系统中，时间片变量和任务的优先级一样，也是由一个变量 `timeslice` 控制。`timeslice` 变量由系统实现确定好，调度过程中不发生改变。`timeslice` 的值不宜设的过大或过小，若时间片大小的值设的过大，则失去时间片轮转调度算法本身的存在意义，退化为先来先服务（FIFO）调度算法；而时间片值若设的过小，频繁的任务切换会使得系统的开销过大。因此，时间片大小设置要比较合适，既不太大，也不太小。

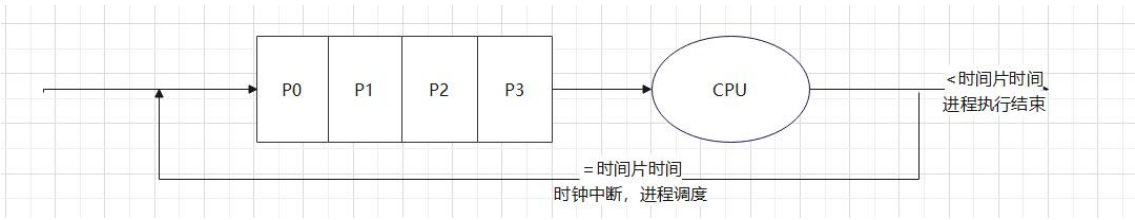


图 2.7 时间片轮转法示意图

2.4.4 先来先服务算法（FIFO）

先来先服务算法（FIFO）是调度算法中最简单的一种。进程变为就绪态后，进入就绪队列，操作系统调度先进入到就绪队列中的进程上 CPU 运行。由于仅需维护一个就绪队列，而无需考虑优先级，时间片等参数，其实现的难度也是最低的。作为时间片轮转调度算法的简化版本，其也具有时间片轮转法各个任务进程公平执行的特点。

但是这样简单的设计也会带来一些问题。若某一进程所需时间过长，其后面进程将迟迟得不到 CPU 资源。由于调度进程的顺序固定，其也无法灵活响应一些优先级较高的进程和紧急进程。

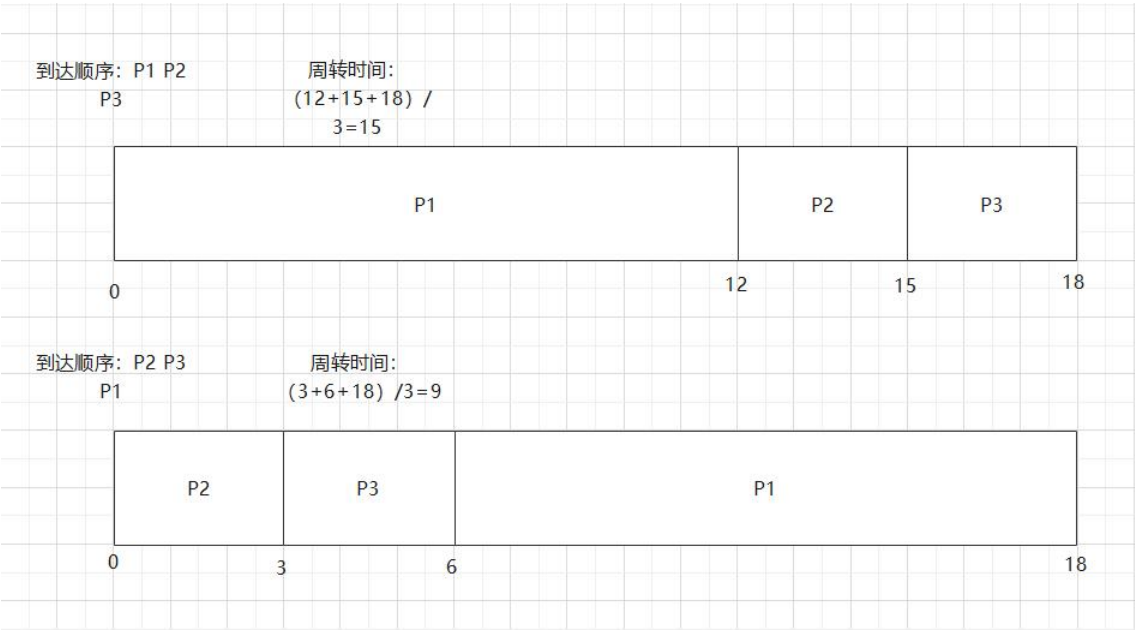


图 2.8 先来先服务算法示意图

2.4.5 短进程优先算法

短进程优先算法是一种以进程所需的执行时间为调度进程上 CPU 运行的指标的任务调度算法。进程按照所需执行时间，在就绪队列中进行排队。之后系统按

照就绪队列中进程的顺序，从队头至队尾依次调度进程上 CPU 运行的进程任务调度算法。

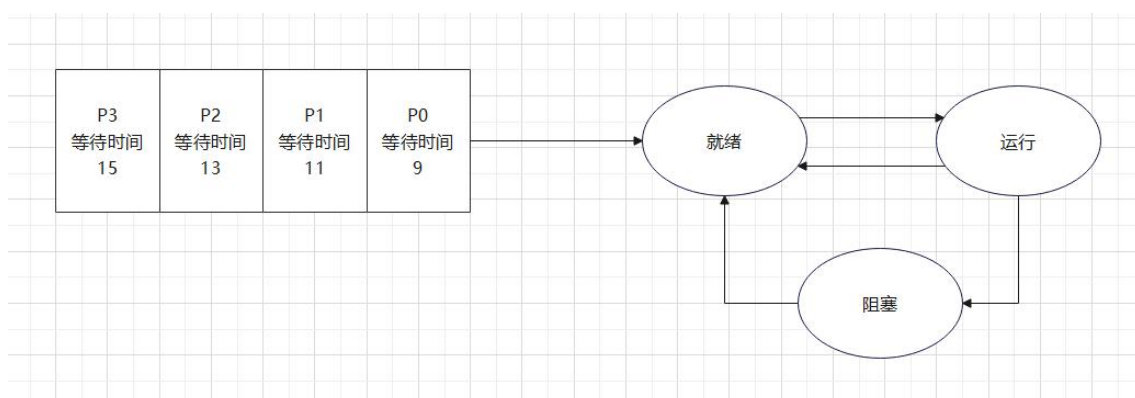


图 2.9 短进程优先调度算法示意图

短进程优先调度算法有以下几个特点。首先，短进程优先算法为非抢占式调度算法，一旦一个进程开始执行，进程将不会被调度下 CPU 直至完成。此外，短进程优先算法具有最优的平均周转时间。但是短进程优先算法也有着几个明显缺陷。首先由于它的非抢占性，预计执行时间长的进程或许永远得不到 CPU 资源，因为新的短进程也会源源不断的进入就绪队列；与此同时，由于需要根据执行时间的预计值调度进程上 CPU 运行，不乏会出现实际需要很长时间运行的进程伪装成短进程的情况，为该调度算法的实现带来了一些难度。

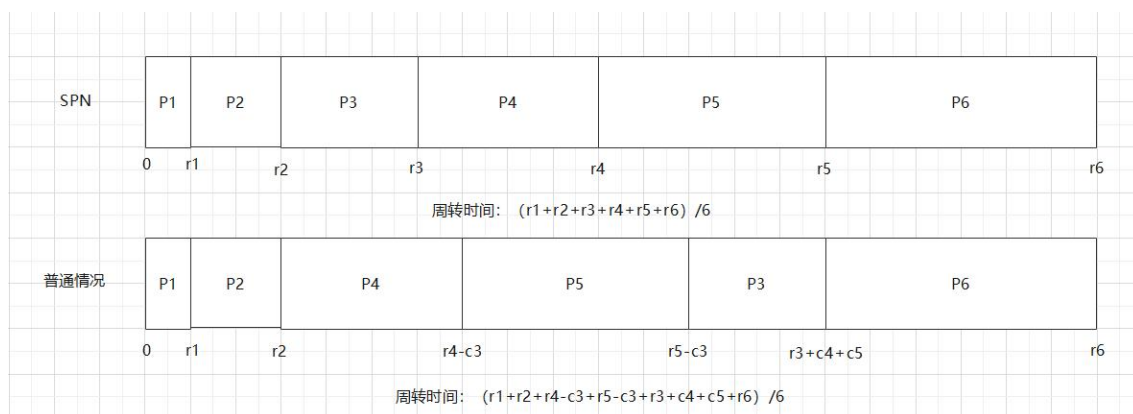


图 2.10 短进程优先算法具有最优的平均周转时间推导过程

2.5 操作系统的信号量机制

信号量机制是操作系统用于控制进程的运行，以及进程对资源的访问的一种机制。

该机制的工作原理可以这样理解。本文将信号量视作进程进行运行需要获得

的一种资源。本文会事先根据实际场景，确定信号量作为一种资源的个数。当一个进程需要获得某种系统资源（例如进程调度需要上 CPU 运行）从而开始运行时，其会首先获取信号量资源，凭借信号量资源获取其运行所需要的系统资源；而目前占有该资源的进程（例如目前正在 CPU 上运行的进程）需要释放信号量资源后，待获取系统资源进程才会获取释放的信号量资源。本文在这里称获取信号量资源操作为 P 操作，释放信号量资源操作为 V 操作。P 操作和 V 操作均为原子操作，其无法被其他外部事件中断和干扰。

由于信号量机制的特性，对于任务调度这一需要使用 CPU 系统资源的应用场景，本文可在这里引入信号量机制。

第三章 开源鸿蒙操作系统高效调度算法的设计

3.1 算法设计思路

在诸如无人机集群通信之类的场景中，操作系统需要同时处理多个复杂进程，在这种情况下，若没有进程任务调度算法参与调度，则会大大降低系统的实时性能，无疑是“低效”的。

单一的时间片轮转算法可以实现各个进程的公平响应，但是其不支持高优先级的进程优先获得响应；而单一的抢占式调度算法可以实现让优先级高的进程优先上 CPU 运行，但是它明显的缺点是会造成“饥饿”现象，进程执行的公平性会大大降低。因此本文在现有的单一的时间片轮转算法的基础上，提出了一种基于时钟中断的优先级抢占式轮转调度算法，这种算法在现有时间片轮转调度算法基础上，增加了抢占式优先级调度算法，这样其将两者的优点结合起来。算法的执行原理为，对于相同优先级的任务进程，系统执行时间片轮转算法；当有更高优先级的进程需要上 CPU 运行时，系统会调度现在正在进行的进程下 CPU，调度该更高优先级进程上 CPU 运行。使用该算法的操作系统不仅有着较高的进程运行公平性，还考虑了可以使高优先级进程优先上 CPU 运行的情况，有效提高了操作系统的实时性能。

系统调度的触发器使用时钟中断。时间片轮转法基于任务调度实现，且当时钟中断发生时，操作系统会检查是否需要进程调度。

下图为该算法的运行原理图。Task1，Task2，Task3 为三组不同优先级进程，其中每组内部进程的优先级均相同。下图中，中优先级进程组第一部分运行完成后空闲进程（空闲任务）。低优先级进程组在时间片轮转法运行时分别受到来自高进程优先级进程组和中优先级进程组第二部分任务的干扰，且高进程优先级进程组和中优先级进程组第二部分任务所有进程任务均完成时，低优先级进程组进程才能恢复运行。此时任务调度方法则为本文提出的基于时钟中断的优先级抢占式轮转调度算法。

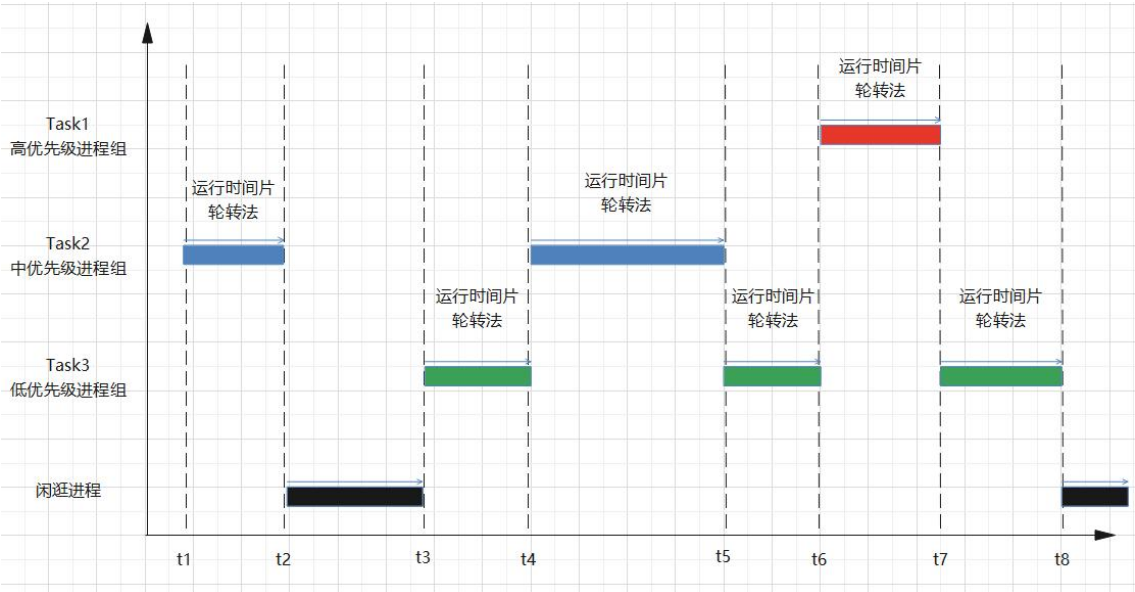


图 3.1 算法运行原理图

3.2 算法评价指标

进程切换所需的时间是操作系统实时性评价的重要指标。本文将从一个进程被调度下 CPU 开始算起，到另一个进程被调度上 CPU 开始运行的时间称作抢占时间，抢占时间越小，操作系统的实时性越好。因此本文选用抢占时间作为评价操作系统实时性和操作系统调度算法的指标。

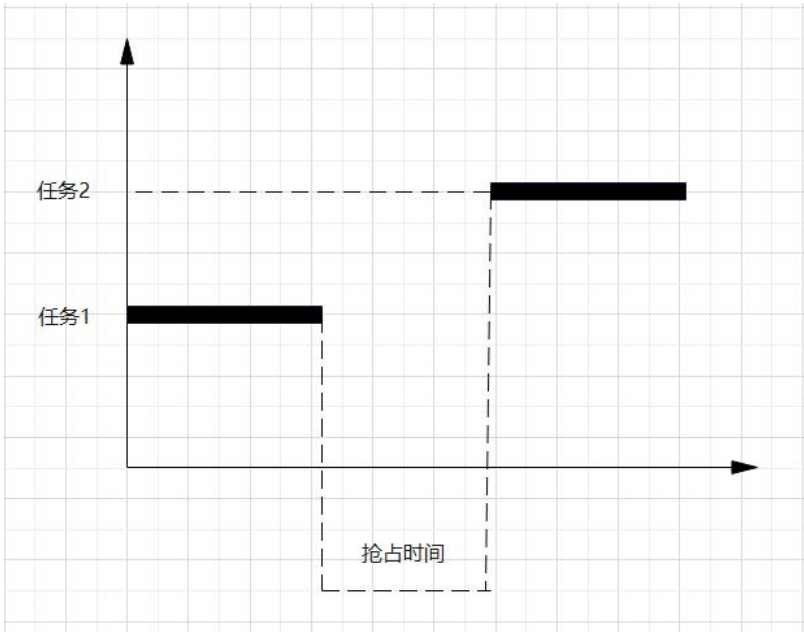


图 3.2 抢占时间示意图

下面为编写程序来进行平均抢占时间的求算方法的流程和代码实现细节。

首先进行全局变量的设置。`g_StartTime` 和 `g_EndTime` 分别用于记录测试开始和测试结束的时间；`g_SwitchCount` 用于累计总共执行的上下文切换次数；而 `g_TestCount` 定义了每个任务进程需要进行的上下文切换次数，也为任务需要被调度的轮次数。

任务的索引通过参数 `param` 传入至函数中。当当前任务为第一个任务（即索引为 0）时，操作系统通过 `LOS_SysCycleGet()` 函数记录任务调度的开始时间。

之后系统开始进行调度。在这里任务的调度不涉及任何调度算法，任务上 CPU 运行是随机的，但是每个进程调度的总次数是有限的，到达总次数规定次数后，该进程不再参与调度。`LOS_TaskYield()` 函数为系统进行任务切换的函数。该函数的工作原理为让当前进程放弃 CPU，并进行任务的切换。使用该函数以后，进程自身的运行时间即为 0，便于后续的计算。

循环结束后，本文将每个当前任务的上下文切换次数加到累计执行上下文切换次数 `g_SwitchCount()` 上。当执行至最后一个任务进程时，即任务索引变量的值为 `TASK_COUNT-1` 时，测试结束，这里需要调用 `LOS_SysCycleGet()` 函数获取并记录调度测试的结束时间 `g_EndTime`。之后即可计算出系统总的上下文切换时间，即为 `gEndTime-gStartTime`。

系统总的上下文切换次数为 `g_SwitchCount`。系统的平均抢占时间即为以上两个数字相除，公式为 $(gEndTime - gStartTime) / g_SwitchCount$ 。

这个值表征了在没有调度算法的作用下，任务的平均抢占时间。后面本文将设计实验，将上述平均抢占时间与执行本文所提调度算法条件下进行任务调度的任务平均抢占时间进行对比，从而显示该调度算法的高效性。

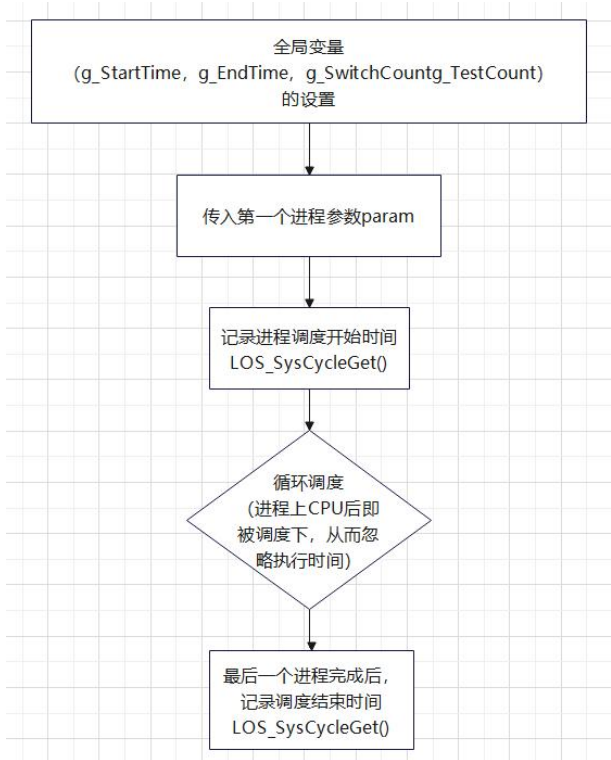


图 3.3 时间片轮转法平均抢占时间求解代码流程图

3.3 算法代码实现

由于时间片轮转法为 LiteOS 自带的调度算法，本文通过 API 接口使用该算法进行调度。带 LOS 的函数为该求算程序调用 LiteOS 接口，每个 LOS 函数里面都有借用该调度算法的接口。因此本文在这里的算法实现工作主要围绕抢占式任务调度的代码实现展开。

抢占式任务调度的实现依赖于信号量机制实现。下面简析其过程及代码实现细节。

PreemptionTask 为低优先级任务进程。首先其调用 GetTime_state 获取当前时间状态；之后其调用 LOS_TaskDelay 函数延迟 3000 个时钟周期。当系统检测到有高优先级进程即将需要上 CPU 运行时，自动跳转至该抢占式任务调度算法代码开始运行。低优先级进程首先要停止运行，并调用 LOS_SysCycleGet()函数记录低优先级任务停止运行的时间（即开始进行任务调度的时间）。之后低优先级任务进程释放信号量资源（g_Sem01）。

HighPriorityTask 为高优先级任务，其依赖一个信号量 g_Sem01 执行。该高优先级进程需要等待信号量才能执行，等待操作调用了 LOS_SemPend()函数。

待得到信号量资源后，操作系统记录高优先级任务开始执行的时间（即结束任务进程调度的时间）。之后操作系统开始运行高优先级任务的代码。

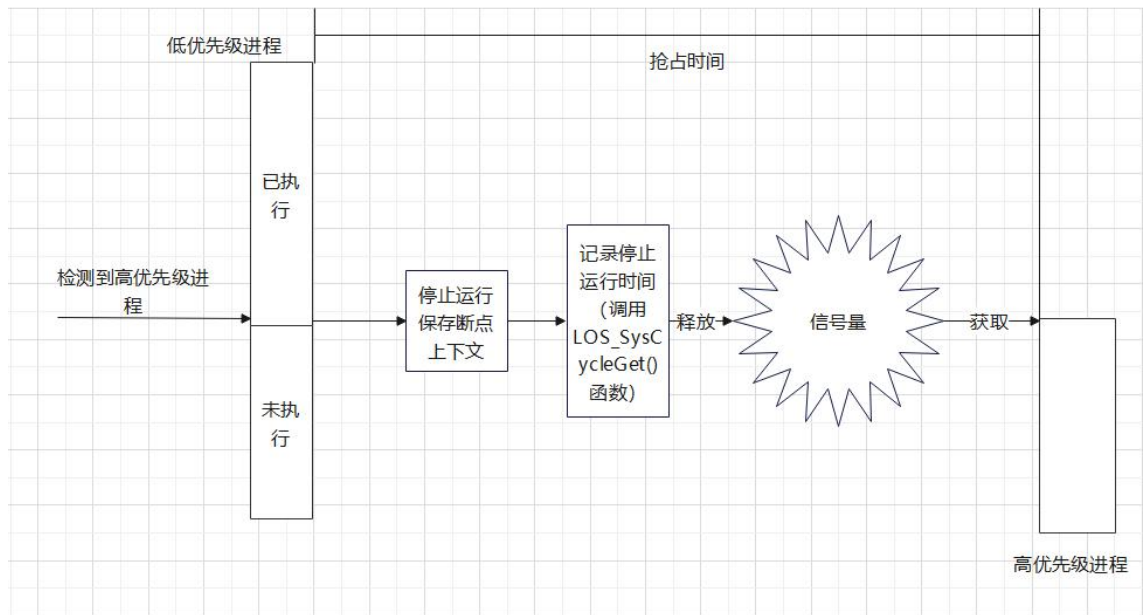


图 3.4 抢占式优先级调度的实现

第四章 算法的硬件移植实验

4.1 GD32F303 开发板简述

星空派（GD）开发板是一款由旗点科技推出的开发板，其带有 GD32F303ZET6 芯片，可实现 STM32F103 和 GD32F103 等型号开发板的全部功能。该开发板实现代码指令和数据分开存储，支持 WiFi，4G，LoRa 等物联网通信接口，支持多种通信协议，自带 flash 等存储器，所有 IO 接口均引出可用于外设开发，功能齐全。该开发板使用基于 Arm Cortex-M4 处理器的 32 位通用微处理器，非常适合 LiteOS 内核的移植，故在本课题中，本文选用 GD32F303 开发板完成实验。

对于 GD32F303 开发板，编译出来的固件一般分为四部分：code，即程序代码部分，主要由程序指令组成，它会被烧录到开发板的片内 flash 上；RO-data，即只读数据段，其保存全局常量和字符串，因为其只读特性，这段数据直接放入 flash 中；RW-data，即已经初始化好的可供读写的数据，主要由全局变量和静态变量组成，它会事先被储存在 flash 中，上电后会将这些变量初始值存入开发板内存当中；最后一部分为 ZI-data，为未经初始化或初始化为 0 的全局变量，其默认均为 0，CPU 上电后，直接清除这一部分内存即可。

在 CPU 上电之前，前三部分将被烧录入开发板的 flash 当中，其中 code 部分位于最前面。CPU 上电之后，ARM 核会将 PC 寄存器中值设为 0x8000000，即 flash 的起始地址，之后开始按顺序执行指令。

为了便于代码的烧录操作，GD32F303 开发板自带一个 Jlink 烧录器，其一端通过 JTAG 接口连接于开发板上，另一端通过 USB 接线与主机相连。

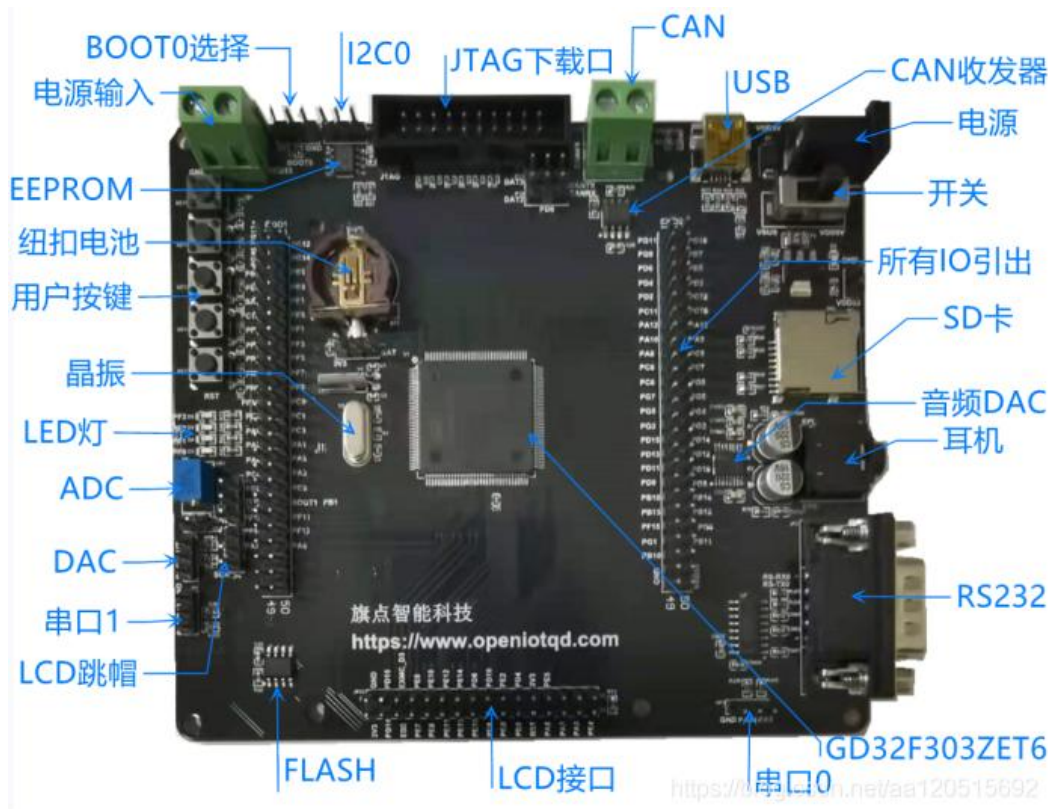


图 4.1 GD32F303 开发板

4.2 开发环境的搭建及代码的烧录

本文在这里选用 Ubuntu 20.04 的 VirtualBox 虚拟机作为代码的开发环境。开发所使用的开源鸿蒙内核使用 liteos-m 内核，源码版本使用 OpenHarmony 3.0 版本源码，源代码拷贝在虚拟机中。

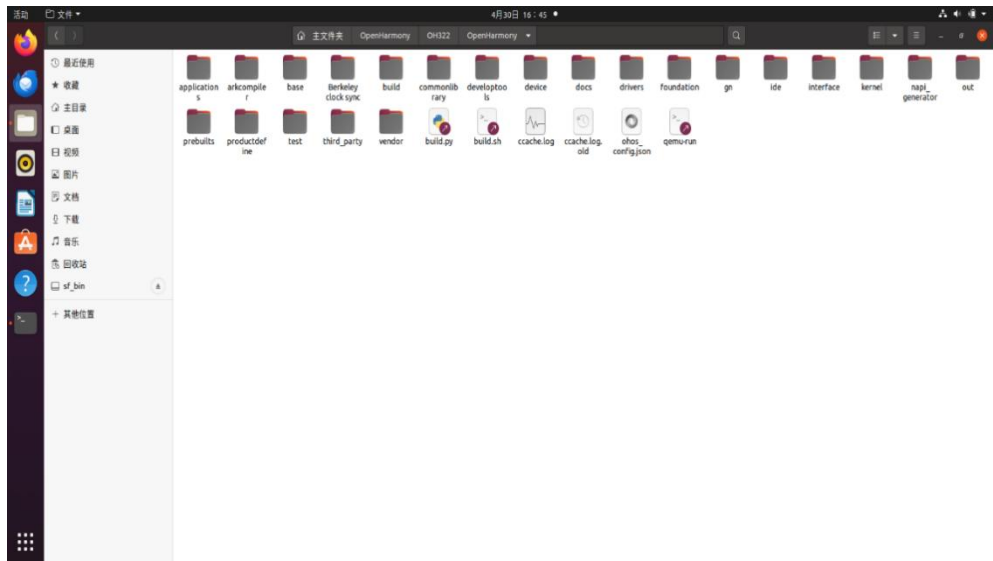


图 4.2 开源鸿蒙源代码目录结构

代码编辑是在 `device-board-lianzhian-gd32f303_lianzhian-sdk` 源码目录下，编辑的代码位于 `kernal_test.c`，`kernal_test.h`，`main.c` 三个代码文件下，这里其中 `main.c` 为代码运行的主程序，该主程序声明了可以调用 `kernal_test.c`，`kernal_test.h` 两个文件函数库，因此在这两个文件中编写好的函数可直接在主函数中被调用。

因在虚拟机下进行代码的编辑不太方便，本文使用主机端的 Visual Studio Code 进行代码的编辑，这里需要将主机端与虚拟机端进行 ssh 连接。首先需要通过 `ifconfig` 命令获取待连接虚拟机的 IP 地址，之后进入 Remote Explorer 页面，点击 SSH 旁的加号（添加新的虚拟机），以 “ssh name@IP 地址” 格式输入需要连接的主机名称和主机的 IP 地址。在本次实验中，`han` 为待连接的虚拟机名称，`192.168.56.106` 为待连接的虚拟机地址，按照上述格式输入，回车开始建立连接。建立好连接后，点击向右箭头（即 Connect in Current Window），输入虚拟机密码，待进入连接后，即可实现在 VSCode 下进行代码的编辑。

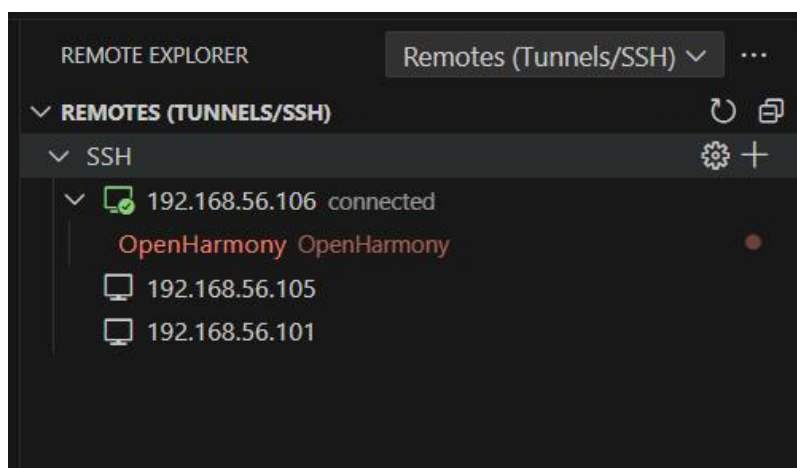


图 4.3 建立 ssh 连接

别机器级代码（二进制文件），因此需要进行编译操作。打开待编译文件所在的目录，在终端输入 `hb build -f` 命令，回车开始编译，当显示 `build success out` 目录时，编译完成，并会在目录下面的 GD32 文件夹生成一个 `.bin` 文件，该文件为可供机器硬件直接运行的机器级代码二进制文件，后续烧录操作即为烧录该 `.bin` 文件。

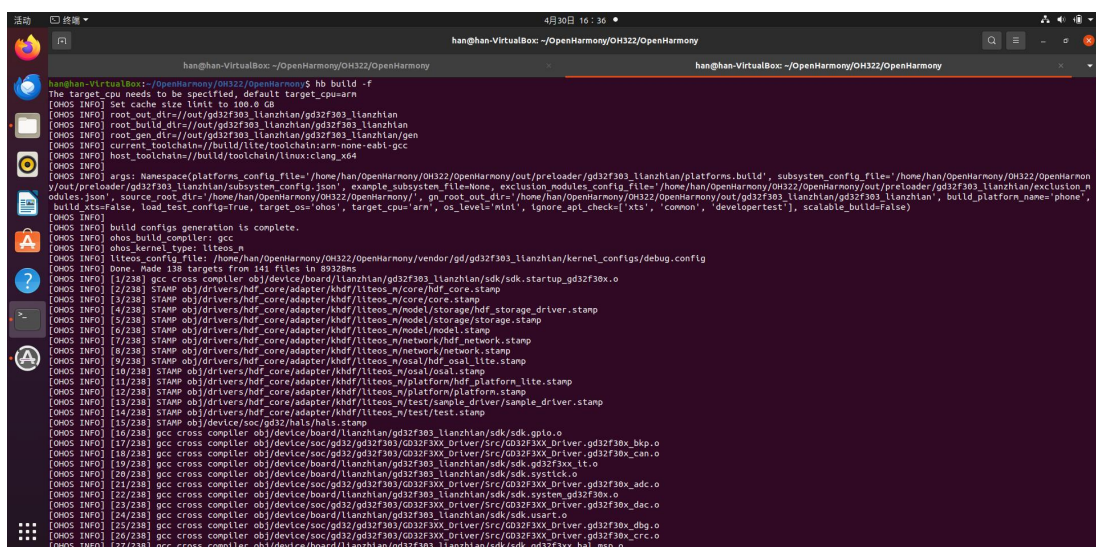


图 4.6 开始进行代码烧录

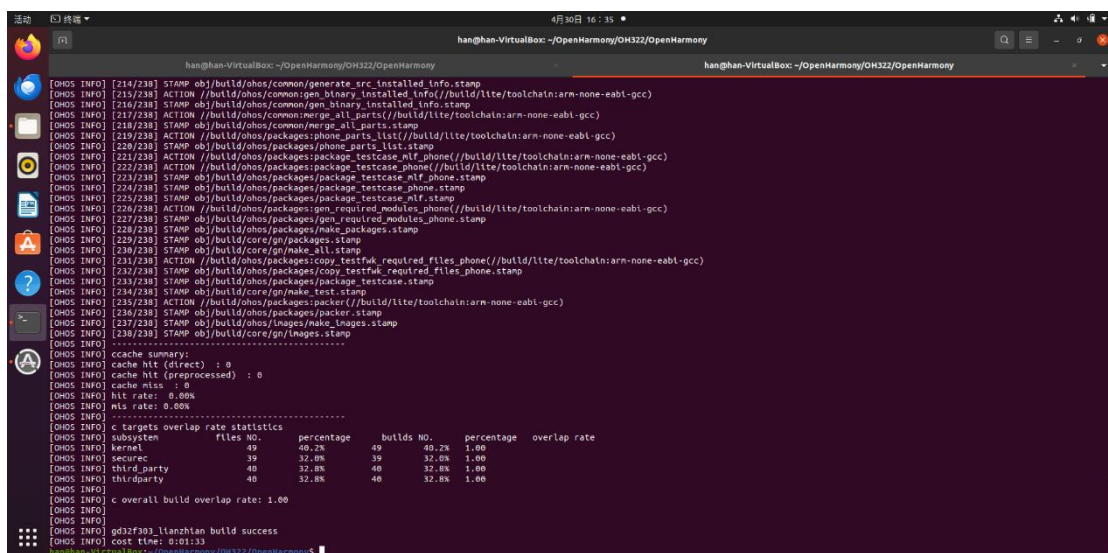


图 4.7 代码烧录完成



图 4.8 烧录好的代码文件

接下来进行第三步烧录操作。本文首先通过共享文件夹将该文件传输至主机。由于本文使用的开发板 GD32F303 为基于 ARM 架构的开发板，因此需使用针对 ARM 架构处理器的烧录软件 Jlink 进行烧录。本文使用主机上安装好的 Jlink 软件打开上述生成的 .bin 二进制文件，进行代码的烧录工作。

当烧录完成后，打开 XCOM V2.6 串口调试软件，观察代码运行的效果。

4.3 实验方案设计

在第三章中，本文对无调度算法参与进行进程任务调度和基于本文提出的调度算法进行进程任务调度两种情形进行了代码实现，这里本文编写代码进行进程任务的创建并进行调度实验，计算两种情形下进程平均抢占时间。

4.3.1 无调度算法参与进行进程任务调度情形

本调度实验设计为创建 5 个优先级相同进程参与调度，因为增加了抢占式调度算法，这里必须要控制优先级变量相同。进程创建流程为先设置所需创建任务进程的个数，之后进行任务进程的配置，最后进行进程的创建。

下面对系统创建五个相同优先级任务进程程序的执行流程和代码实现细节进行分析。首先定义变量 `uwRet` 为存储函数调用返回值的变量，之后分别定义存储任务 ID 和任务定义结构体的数组。

接下来本文使用一个 `for` 循环设定参与任务调度的任务数量。本文定义一个字符数组 `tasknum`，大小设为 5，之后使用 `int_to_char` 函数将任务索引 `i` 转为字符数组 `tasknum`，因为 `tasknum` 数组大小为 5，从而设定了创建的任务数量为 5 个。与此同时，本文需要进行任务结构体的初始化，方法为使用 `memset` 函数将每个任务结构体中的结构体成员变量设为 0。

在这之后，对它们进行任务名称的生成和任务属性的设置。任务的名称生成使用 `strcpy` 和 `strcat` 函数，将 “TaskSample_” 和任务索引字符 “tasknum” 相连接，并将生成好的任务名称字符型变量赋值给 `taskname` 变量。接下来进行任务属性的设置步骤，这里需要设置以下几个属性：

表 4.1 任务属性配置

pcName	设置为任务名称
pfnTaskEntry	设置任务的入口函数，这里使用类型转换指定 <code>TasktestSWFunction</code> 函数作为任务的入口函数
uwStackSize	设置任务堆栈大小
usTaskPrio	设置任务的优先级（调度为基于时间片轮转的相同优先级调度，这里将任务优先级设置为 2）
uwArg	设置传递入入口函数的参数

以上代码实现了进程的配置操作，接下来进行任务的创建操作。这里本文调用 `LOS_TaskCreate()` 函数进行任务的创建。函数将返回值存储在定义好的 `uwRet` 变量中，若 `uwRet` 的值为 `LOS_OK`，则说明任务进程创建成功，打印创建成功消息，否则说明任务创建失败，打印失败消息。

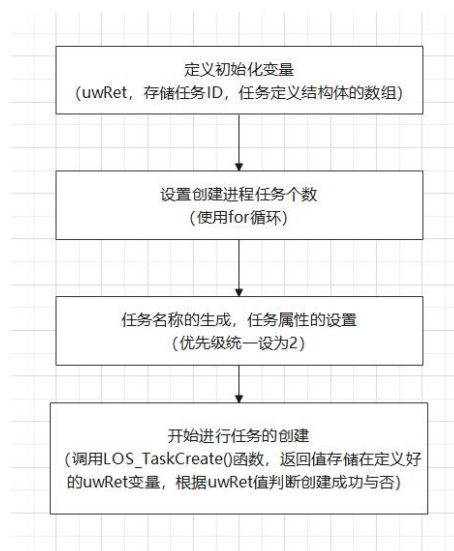


图 4.9 5 个相同优先级进程创建代码实现流程图

4.3.2 基于本文提出的调度算法进行进程任务调度情形

相同优先级进程基于时间片轮转算法进行调度。在不同优先级任务基于优先级抢占式调度实验中，本文创建两个进程参与调度，其中一个为高优先级进程，另外一个为低优先级进程。

在 LiteOS 原有内核下，进程使用时间片轮转法进行进程任务调度。在第三章所述抢占式进程任务调度算法代码中，当系统检测到高优先级进程任务后，自动跳转至抢占式进程调度算法代码执行，因此在这里，两个不同优先级任务会自动使用抢占式进程任务调度算法进行调度，对于优先级相同的进程系统仍使用原有时间片轮转法进行任务调度。

以下为对创建两个不同优先级任务进程的代码实现流程和代码实现细节的分析，其在一个函数 `StartPreemptionTest()` 中实现，该函数没有返回值。这段代码分为两部分，一部分为创建高优先级任务进程但不立即启动，另一部分为创建低优先级进程。

在开始创建进程之前，本文需要首先调用 `LOS_SemCreate()` 函数进行信号量 `g_Sem01` 的创建，之后本文需要调用 `LOS_TaskLock()` 函数进行任务调度的锁定，这样就可以避免在任务创建过程中发生任务切换。

接下来本文对于这两部分代码的实现过程进行具体的分析：

（1）创建高优先级任务但不立即启动函数段

同相同优先级任务抢占式调度的任务创建函数相同，本文首先定义变量 `uwRet` 为存储函数调用返回值的变量，之后定义待创建的任务 ID 变量 `taskID`，之后声明用于创建任务的结构体 `stTask`。

接下来进行任务属性的设置。首先进行任务结构体的初始化，方法为使用 `memset` 函数将每个任务结构体中的结构体成员变量设为 0，接下来设置任务属性的设置，需要设置以下任务属性：

表 4.2 任务属性配置

<code>pcName</code>	设置创建的任务进程的名称，由于这里创建的为高优先级任务进程，因此 <code>pcName</code> 设置为 “HighPriorityTask”
<code>pfnTaskEntry</code>	设置任务入口函数 <code>HighPriorityTask</code> ，该入口函数在其他地方定义
<code>uwStackSize</code>	设置任务堆栈大小
<code>usTaskPrio</code>	设置任务优先级大小，高优先级任务的优先级设置为 <code>TASK_PRIORITY-1</code> （优先级数值越小，进程优先级越高）

高优先级进程的配置操作目前已经得到实现，接下来进行高优先级任务的创建操作。这里本文调用 `LOS_TaskCreate()` 函数进行任务的创建。若创建成功，`LOS_TaskCreate()` 函数会将返回值 `LOS_OK` 赋给 `uwRet`。之后检查 `uwRet`，若其

值为 `LOS_OK`，则确定高优先级进程创建成功，并打印创建成功信息；否则确定创建失败，并打印创建失败信息。

(2) 创建低优先级进程

低优先级进程的创建类似于高优先级进程的任务创建，创建的基本流程也基本相同。不同之处在于低优先级进程的任务优先级设为 `TASK_PRIORITY`，同时也为其设置了不同的任务名称等地方。

待任务创建完成后，本文使用 `LOS_TaskUnlock()`函数进行任务调度的解锁，此后开始进行任务调度。

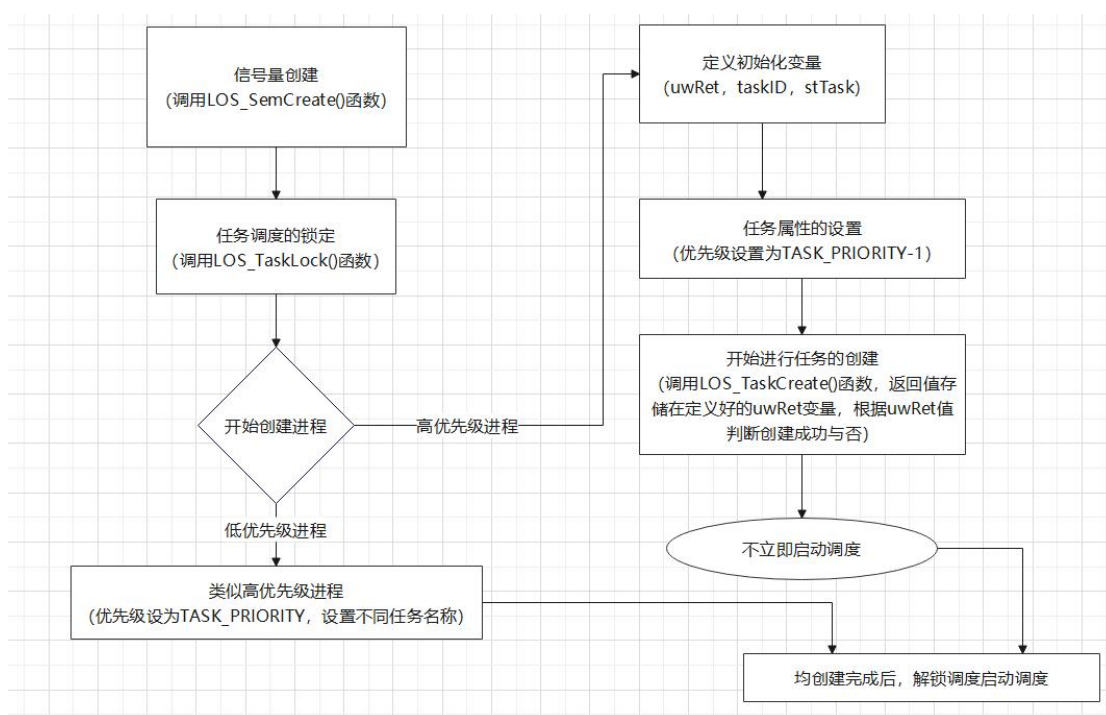


图 4.10 创建高低优先级任务流程图

4.4 实验结果

4.4.1 无调度算法参与进行进程任务调度情形实验结果

下图为相同优先级任务进程基于时间片轮转法调度的实验结果。串口首先进行了通信总线的初始化，当显示 `uart init ok` 时说明通信总线初始化完成。之后将开发板中的 `liteos-m` 内核初始化并启动。

启动完成后，系统首先执行创建进程代码进行创建进程，创建成功后显示 `create success`。之后系统开始进行五个优先级相同的任务进程的基于时间片轮转

法的任务进程调度。

在本次实验中，任务进程切换的次数为 5000 次，这 5000 次切换的平均任务切换时间为 1148 微秒。



图 4.11 无调度算法参与进行进程任务调度情形实验结果

4.4.2 基于本文提出的调度算法进行进程任务调度情形实验结果

下图为不同优先级任务进行抢占式调度的实验结果。串口依然是首先进行了通信总线的初始化，当显示 `uart init ok` 时说明通信总线初始化完成。之后将开发板中的 `liteos-m` 内核初始化并启动。这一步同前面相同优先级任务调度。

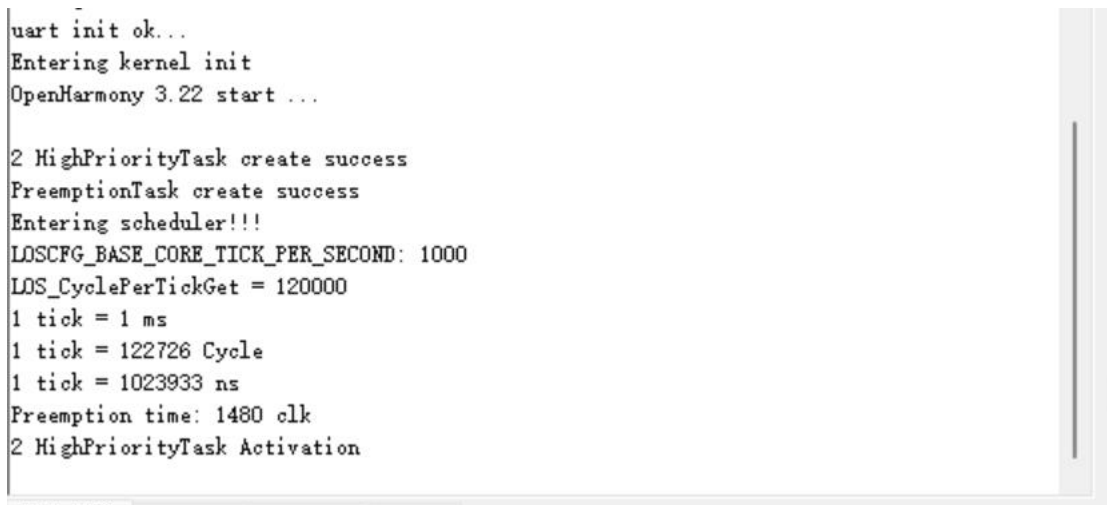
开源鸿蒙代码启动后，首先系统先创建两个高优先级任务，但是当高优先级任务创建成功后，系统不会立即启动它们，而是会先创立低优先级进程。当低优先级进程创建成功时，进入进程的调度环节。在进入调度环节时，系统内核会自动检测到两个进程优先级不同，从而触发抢占式优先级调度算法代码的运行。

操作系统的基本计时单位为 `tick`。将 1s 分为 `n` 份，那么一份的时间即为一个 `tick`。在本次实验中本文将 `n` 设为 1000，这样一个 `tick` 的时间就为 1ms，其设置方法为将变量 `LOSCFG_BASE_CORE_TICK_PER_SECOND` 的值设置为 1000。一个 `cycle` 指计算机处理器的一个基本操作周期，它是系统的最小计时单位。

系统的主时钟频率即为系统在 1s 内的操作数。这里本文设置一个 tick 包含了 120000 个 cycle，通过设置变量 LOS_CyclePerTickGet 的值为 120000 确定。

理想状态下，1 个 tick 等于 1 毫秒，包含了 120000 个 cycle。但是由于系统的运行会涉及到系统运行延迟和接收中断滞后的影响，因此在实验结果中，1 个 tick 包括了 122726 个 cycle，因此最终实验结果中一个 tick 的时间长度为 1023933 ns，大于 1ms 的时间。

clk 为 clock 的缩写，它表示了一个 cycle 的时间长度。实验结果表明最终的系统调度时间长度为 1480 clk。



```
uart init ok...
Entering kernel init
OpenHarmony 3.22 start ...

2 HighPriorityTask create success
PreemptionTask create success
Entering scheduler!!!
LOSCFG_BASE_CORE_TICK_PER_SECOND: 1000
LOS_CyclePerTickGet = 120000
1 tick = 1 ms
1 tick = 122726 Cycle
1 tick = 1023933 ns
Preemption time: 1480 clk
2 HighPriorityTask Activation
```

图 4.12 基于本文提出的调度算法进行进程任务调度情形实验结果

由此可见，在不适用任何调度算法的情况下，系统进程调度的平均抢占时间为微秒级别，在使用本文提出的调度算法后，系统进程调度的平均抢占时间为纳秒级别，大大减少了调度的平均抢占时间，极大的提高了系统的实时性能，也反映了本文提出的调度算法的高效性。

第五章 未来研究展望

5.1 对测试实验的优化设想

在测试实验中，对于测试进程的设计还可以做进一步优化。在测试基于本文提出的调度算法进行任务调度时，仅设置了两个进程，但是在这里，本文可以设置多个进程，其中包括优先级相同和优先级不同的进程。这样优化测试实验可以将实验的偶然性进一步降低，进一步确认该调度算法的高效性。

另外，在测试实验中，测试进程的优先级是固定的，在程序中指定好了。但是在实际情况中，进程的优先级不可能是一成不变的，例如，在无人机飞控应用场景中，在正常情况下，坠机保护进程的优先级会比较低；但是在无人机遭遇危险时，坠机保护进程的优先级必须要提高。因此，针对外界环境及实际需要，对进程的优先级进行动态调整，也是后续需要对本实验进行完善的地方。在这里，本文可以引入深度学习的算法，以外界不同条件下进程所需的不同优先级为数据集进行模型的训练，并使用训练好的模型进行进程优先级的预测，以此实现动态优先级进程调度。

5.2 对算法设计的优化设想

针对在本文提出的时间片轮转和抢占式优先级调度相结合的调度算法，本文可基于 Min-Min 和 Max-Min 任务调度算法对其进行优化，这样优化好的算法可以更适合应用于类似于无人机集群的多任务分布式应用场景中。

与单纯的以优先级作为调度任务进程上 CPU 运行的指标不同，Min-Min 和 Max-Min 算法将进程的优先级与进程的执行时间长短综合起来作为调度进程上 CPU 运行的指标。

Min-Min 算法致力于最小化所有任务的总完成时间。这个算法的核心思想是优先调度预计完成时间最短任务进程上 CPU 运行，同时兼顾任务的优先级。这个算法适合应用于任务进程所需执行时间相差不大的场景，以及需要快速完成全部任务的场景。

Max-Min 算法致力于实现更平衡的资源分配。在 Max-Min 任务调度算法中，操作系统会优先调度任务优先级较高的任务上 CPU 运行，因为这样的任务往往更

加紧急和重要。其次，**Max-Min** 算法会优先处理预计完成时间最长的任务，这样可以提升系统资源分配的均衡程度，提高系统资源的利用率。这个算法适合应用于任务进程所需执行时间相差较大的场景，以及对于任务进程资源分配均衡性要求较高的场景中。

致谢

四年的时光转瞬即逝，我的本科学习生活也即将告一段落。回首四年大学生活，有过成绩不尽如人意，保研考研均惨败的失落与痛苦，有过对未来发展方向的迷茫与焦虑，但是我仍然坚持走下来，没有自暴自弃，顺利的通过了所有的课程，也找到了自己比较满意的工作。这一路走下来，有老师的关心和帮助，朋友们的鼓励，这些关心和鼓励在我迷茫无助的时候给予了我很大的慰藉，也给予了我勇敢走出困境，继续走下去的力量。值此论文完成之际，我想对于曾给我关心和鼓励，给予我帮助的人致以衷心的感谢。

首先我要感谢我的毕设指导老师韩高老师。出于对于计算机科学的兴趣，我选择了韩高老师的这个有关开源鸿蒙的毕设题目。在实验方案设计，代码编写，论文撰写的过程中，韩高老师都耐心对我进行了指导，同时韩高老师也经常对我的进度进行检查，这样使得我在论文写作的过程中游刃有余，避免了卡最终期限完成论文影响论文的质量。韩老师的严格要求和严谨的治学风格深深的影响了我，感谢韩老师对我的帮助和教导。

其次我要感谢我的两位师兄阎涵旭学长和杨浩鑫学长。当我的代码编写和实验过程中，遇到难以解决的问题时，两位学长都会积极的帮助我解决问题，并教会了我解决技术问题方法。在论文写作的过程中，两位学长也提出了论文写作中的很多问题，根据他们的意见，我不断地完善我的论文。与此同时，在我考研和求职的过程中，两位学长也给了我很多建议，在他们的帮助下，最终我也收获了一份满意的工作。祝愿两位学长在未来的科研中一切顺利，顺利完成研究生学业，前程似锦。

感谢和我同组的两位同学，相志龙同学和卢寺强同学。由于我们三个人的课题都是开源鸿蒙相关，在完成整个课题的过程中，我们相互帮助，互相学习，一起讨论实验方案，互相帮忙解决遇到的问题。在四年的学习中，我们也会共享学习资料，一起准备期末考试，一起参加招聘会求职。祝你们在未来的工作中，一切顺利。

感谢衡中校友群，反传小群的所有群友们。大一刚入学校，衡中校友群热心的学长学姐们帮助了我很多，让我迅速适应了在西电的新生活，在我后面的学习生活中也是有问必答，同时我也在群里认识了很多陪伴我走过大学四年的好朋友。

也感谢反传小群的全体群友们，我们一起在 C129 自习，一起吃饭打卡，一起出去玩的场景会是我四年最美好的回忆。这里也是我的树洞，当我迷茫失落的时候，特别是在我考研压力很大，以及考研失败的时候，群友们会为我加油，会安慰我鼓励我，让我迅速找回状态投入到下一阶段中；在群里和大家聊聊天会帮助我缓解一天积累下来的焦虑。祝愿准备保研的群友能选到心仪的课题组，考研的群友能够顺利上岸理想的学校，选择就业的群友能拿到理想的 offer，已经上研的科研顺利，我们顶峰相见。

我要感谢我的父母。其实我并不是一个特别让人省心的孩子，从小到大，我都是那种问题学生，不好好学习，被老师请家长，但是我的家长始终包容我。在 19 年第一次高考结束时，我没有选择像和我同分数段的同学一样选择天津本地的普通一本学校平平淡淡的过四年，而是选择不顾老师同学们，家里长辈亲戚们的不解，以及第一届新高考改革带来的不确定性，毅然前往衡中复读一年，我的父母也选择了支持我的选择。在大三选择考研时，我也选择了从空间科学与技术专业跨考计算机专业 408 统考，选择了北京大学作为我的报考院校，尽管风险很大，但是我的家长仍然支持了我的选择。我的与众不同让父母为我操了更多的心，但是仍然选择支持我，感谢父母的理解与支持。

最后，我也想感谢坚持我走下来这一路的我自己。若是当年动摇了没有选择去复读而是选择了家门口的普通大学，抑或是考研的时候知难而退而放弃，那我的人生会变得平淡乏味，也得不到好的发展。尽管考研以失败告终，但是凭借西电的名气和考研所学知识，我也获得了一份薪资高，发展好的工作 offer，所以我相信过去受的一切苦和累，不解与彷徨，都会以一种形式回馈给自己，付出的努力不会白费。毕业后，我也会带着这样一种不服输，不甘平庸的精神，带着父母老师的教导，朋友们的祝福，坚定的走下去。

参考文献

- [1] 余兵,黎忠文.Linux 操作系统实时性分析[J].计算机技术与发展,2007,17(9):41-44.
- [2] 陈万桥,杜承烈,江南.操作系统实时性指标研究与测试结果分析[J].航空计算技术,2014,44(01):109-112.
- [3] 郝英华,王瑞,李晓娟.NuttX 操作系统进程切换调度实时性的形式化验证[J].计算机应用与软件,2018,35(07):15-20+51.
- [4] 李孝成.SylixOS 大型实时操作系统实时性分析[J].单片机与嵌入式系统应用,2021,21(04):8-11.
- [5] 王远鹏.提高实时操作系统的实时性能和可靠性策略[J].计算机光盘软件与应用,2012,15(16):134+136.
- [6] 任慰.以实时操作系统为中心的嵌入式系统平台化设计研究[D].华中科技大学,2013.
- [7] 周洁,李正凡.嵌入式实时操作系统的调度策略[J].华东交通大学学报,2005,(04):110-112+133.
- [8] 丁仕林, 吴强.嵌入式实时系统 μ C/OS-II 任务调度算法分析及改进[J].电子技术应用,2009,35(01):20-23+27.2009-4-29
- [9] 薄紫彤,乔颖,冷昶等.基于深度强化学习的实时调度策略[C].中国科学院软件研究所学术年会, 2021.
- [10] 张文越.基于 Min-Min 和 Max-Min 算法改进的网格调度算法的研究[D].北京工商大学,2010.
- [11] Augonnet C, Thibault S, Namyst R. StarPU: a runtime system for scheduling tasks over accelerator-based multicore machines[D]. INRIA, 2010.
- [12] Meng Q, Humphrey A, Schmidt J, et al. Investigating applications portability with the uintah dag-based runtime system on petascale supercomputers[C]//Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. 2013: 1-12.
- [13] Etminani K, Naghibzadeh M. A min-min max-min selective algorithm for grid task scheduling[C]//2007 3rd IEEE/IFIP International Conference in Central Asia on Internet. IEEE, 2007: 1-7.
- [14] Sharma N, Tyagi S, Atri S. A Comparative Analysis of Min-Min and Max-Min

Algorithms based on the Makespan Parameter[J]. International Journal of Advanced Research in Computer Science, 2017, 8(3).

- [15] Parsa S, RASA R E M. A New Grid Task Scheduling Algorithm[J]. International Journal of Digital Content Technology and its Applications, 2009, 3(4).

附录 A

下附本文中涉及的关键代码：

(1) 时间片轮转法平均抢占时间求解代码：

```
UINT64 g_StartTime=0; UINT64 g_EndTime = 0;
UINT32 g_SwitchCount =0;
UINT32 g_TestCount = 10000; //测试切换的次数

void TasktestSWFunction(UINT32 param) {
    UINT32 taskIndex=(UINT32)param;
    UINT64 localtime;
    // 最后一个任务记录开始时间
    if(taskIndex == 0){
        g_StartTime =LOS_SysCycleGet();
    }
    for(UINT32 i = 0; i < g_TestCount; i++) {
        // 执行任务切换
        LOS_Taskyield(); // 放弃 CPU 使用权，导致任务切换
    }
    g_SwitchCount +=g_TestCount;// 第一个任务记录结束时间
    if(taskIndex ==TASK_COUNT -1){
        g_EndTime = LOS_SyscycleGet();
        UINT64 switchTime=(g_EndTime-g_StartTime);
        printf("g_SwitchCount:%d\r\n",g_Switchcount);
        printf("Average Context Switch Time: %llu us\r\n", switchTime / g_SwitchCount);
    }
}
```

(2) 抢占式优先级调度的实现代码:

```
UINT32 g semaphoreID;
//高优先级任务
void HighPriorityTask(void) {
    //等待信号量
    LOS SemPend(g Sem01, LOS WAIT FOREVER);
    //记录高优先级任务开始执行的时间
    g_ EndTime = LOS_ SysCycleGet();
    // ...高优先级任务的代码...
    Taskworkload();
    printf("Preemption time: %llu clk \r\n", g EndTime-g. StartTime);
}

void Preempt ionTask(UINT32 param) {
    GetTime_ state();
    LOS TaskDelay( 3000);
    //触发高优先级任务的事件
    g. startTime = LOS SysCycleGet();
    UINT32 ret = LOS SemPost(g Sem01);
    if(ret == LOS OK) printf(' %d HighPriorityTask Activation \r\n" , (UINT32)param);

    //使高优先级任务就绪并触发抢占
}
```

(3) 5 个相同优先级进程创建代码实现:

```

UINT32 TaskSample_create_sw(VOID)
{
    UINT32 uwRet ;
    UINT32 taskID[TASK_ COUNT];
    TSK_ INIT_ PARAM s stTask[TASK_ COUNT];
    for (int i=0; i < TASK_ COUNT; i++)
    {
        memset(&stTask[i], 0, sizeof(TSK_ INIT_ PARAM_ S));
        char tasknum[5];
        int len;
        int_ to_ char(i, tasknum, &len);
        char taskname[4 + len];
        strcpy(taskname, "TaskSample ." );
        strcat(taskname, tasknum);
        stTask[i].pcName = taskname;
        stTask[i].pfnTaskEntry=(TSK_ ENTRY_ FUNC)TasktestSWFunction;
        stTask[i].uwStackSize=TASK_ STACK_ SIZE;
        stTask[i].usTaskPrio = 2;
        stTask[i].uwArg = (UINT32)i;
        //stTask[i].uwResved = LOS_ TASK_ STATUS_ DETACHED;
        UwRet = LOS TaskCreate(&taskID[i], &stTask[i]);
        if (uwRet != LOS_ OK){
            PRINT_ ERR("TestTask create failed IrIn" );
            return LOS_ NOK ;
        }
        else
            printf("Xs create success IrIn" ,st Task[i]. pcName) ;
    }
    return LOS_ OK;
}

```

（4）两个不同优先级进程创建代码实现：

第一部分：创建高优先级任务但不立即启动代码实现

//创建高优先级任务但不立即启动

```

UINT32 uwRet;
UINT32 taskID;
TSK_INIT_PARAM_S stTask;
memset(&stTask, 0, sizeof(TSK_INIT_PARAM_S));
stTask.pcName = "HighPriorityTask" ;
stTask.pfnTaskEntry = (TSK_ENTRY_FUNC)HighPriorityTask;
stTask.uwStackSize = TASK_STACK_SIZE;
stTask.usTaskPrio = TASK_PRIORITY-1;
//stTask[i].uwResved = LOS_TASK_STATUS_DETACHED;
uwRet = LOS_TaskCreate(&taskID, &stTask);
if (uwRet == LOS_OK)
{
    printf( "%d HighPriorityTask create success Ir'n" , (UINT32)taskID);
}
else
    printf("TestTask state: %d rIn" , uwRet);

```

第二部分：创建低优先级进程代码实现

```

UINT32 uwRet_Preemption;
UINT32 taskID_Preemption;
TSK_INIT_PARAM_S stTask_Preemption;
memset (&stTask_Preemption, 0, sizeof(TSK_INIT_PARAM_S));
stTask_Preemption.pcName ="PreemptionTask";
stTask_Preemption.pfnTaskEntry=(TSK_ENTRY_FUNC)PreemptionTask;
stTask_Preemption.uwStacksize=TASK_STACK_SIZE;
stTask_Preemption.usTaskPrio.TASK_PRIORITY;
stTask_Preemption.uwArg= (UINT32)taskID;
uwRet=LOS_TaskCreate(&taskID_Preemption, &stTask_Preemption);
if (uwRet = LOS_OK)
{
    printf("PreemptionTask create success \r\n");
}
else
    printf("PreemptionTask state: %d \r\n" ,uwRet);

```

班级 2016021

学号 20009100883

本科毕业设计（论文）

外文资料翻译

毕业设计题目 开源鸿蒙操作系统的高效任务调度策略

外文资料题目 The real-time task scheduling algorithm of RTOS+

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 魏子捷

指导教师姓名 韩高

摘要

任务调度器是实时操作系统（RTOS）内核中的一个重要组件。本文研发了一种新的便携式 RTOS 架构 RTOS+，该架构采用新颖的基于多队列的任务调度方法。该任务调度程序有效地协调任务与各种运行时资源、事件和中断，以防止优先级反转问题的发生。实时过程代数（RTPA），一种形式化规范符号，被用来严格描述任务调度程序的体系结构、静态和动态行为。基于 RTOS+ 架构和算法，任务调度和事件处理的框架是在 RTOS+ 平台上实现的。该算法和 RTOS+ 已经被应用于软件工程中基于 RTPA 的代码生成器支撑环境的开发。

关键词：软件工程，实时操作系统，任务调度算法，RTOS+，RTPA

第一章 简介

实时操作系统（RTOS）是一种支持并保证及时响应外部和实时应用程序的内部事件的操作系统。RTOS 内核的主要组件是任务调度器，负责根据给定的调度策略将系统资源分配给任务。由于外部事件的不可预测性，在实现任务调度程序时必须小心，以便它能够以严格定时的方式对事件做出反应。为了实现 RTOS，已经开发了各种任务调度算法和方案，可分为时间驱动、优先级驱动和共享驱动方案。大多数商业 RTOS 使用优先级驱动的任务调度方案。

优先级驱动技术根据任务的要求和系统资源可用性为任务分配优先级。调度程序选择并执行具有最高优先级的任务。优先级驱动的调度技术可以分为静态优先级策略和动态优先级策略。在前者中，任务的优先级是固定的，并且在创建时保持不变。在后者中，任务的优先级可能在运行时根据系统环境和资源可用性而改变。

使用优先级驱动方案在 RTOS 中实现任务调度时经常遇到的一个常见问题是优先级反转。当具有不同优先级的任务共享公共资源时，就会出现此问题。为了解决这个问题已经做了很多工作，并且已经提出并实施了各种解决方案，例如基本优先级继承和优先级上限协议。然而，这些解决方案通常会导致副作用，例如任务的多次阻塞和死锁。

本文扩展了 RTOS 中提出的任务调度和事件处理算法，RTOS 是 2017 年开发的便携式实时操作系统。它提供了任务调度和事件处理的正式描述技术及其在 RTOS+ 中的实现。任务调度和事件处理的算法在第二节中使用实时过程代数（RTPA）正式指定。RTOS+ 的任务调度程序和事件处理程序的实现在第三节中进行了描述。第四节讨论了这项工作的总结和未来的发展。

第二章 RTOS+ 的任务调度

任务调度是将任务分配给操作系统中的处理器和相关资源。在实时系统中，所有

任务均由其计时规范定义，例如截止日期、类型（定期或零星）和所需资源。零星任务是事件驱动处理相关的任务，例如响应用户输入。周期性任务是定期安排的任务。实时系统存在各种调度范例，但大多数商业 RTOS 使用优先级驱动的调度方案，其中根据给定算法为每个任务分配静态或动态优先级。现有的优先级驱动调度方案流行的算法包括速率单调调度(RMS)算法、最早截止时间优先(EDF)调度算法和最小松弛优先(LLF)调度算法。

此类算法采用调度方案，其中任务的优先级基于定时约束，例如任务的周期、频率或截止时间。基于这些属性的优先级驱动调度方案有两个主要问题。第一个问题是优先级属性是由时序约束定义的，但没有考虑任务的重要性。因此，很难安排一个周期较长或截止日期较晚且需要快速响应时间的任务。第二个问题源于这样一个事实：为了使优先级驱动的调度方案有效，任务调度程序应该是抢占式的。这意味着任务调度程序应该允许较高优先级的任务通过接管处理器来“介入”较低优先级任务的执行。传统 RTOS 中抢占式任务调度的问题是，如果应用程序中的各个任务必须共享资源，则会导致“优先级倒置”。RTOS 中有限的计算资源要求这些资源必须由同时运行的多个任务共享。因此，为了使 RTOS 实现其所期望的可预测性、及时性和可靠性，需要采用能够消除优先级倒置、避免任务死锁和多次阻塞的调度算法。在[7]（其他论文）中，我们开发了便携式实时系统的形式化描述操作系统 RTOS+。我们在本文中通过提出一种形式化算法来扩展任务调度和事件处理方面的工作，该算法可以最大限度地减少 RTOS+ 中优先级反转和死锁的可能性。

A. RTOS+架构

RTOS+的架构如图 1 所示，该图说明了系统资源、组件和内部控制模型之间的交互。RTOS+ 中有四个子系统：a) 处理器和任务调度程序，b) 资源控制器，c) 事件处理程序，以及 d) 系统资源。任务调度器是最里面的操作系统内核，直接控制 CPU 在任务和所有其他系统资源之间的切换。RTOS+作为 CPU 的补充，其资源主要是存储器、系统时钟、I/O 端口、中断设施、文件以及队列和任务/资源控制块等内部控制模型，如图 1 所示。不同的队列以相同的优先级保存任务。这使得任务调度程序可以轻松地将处理器分配给每个就绪队列中的第一个任务。

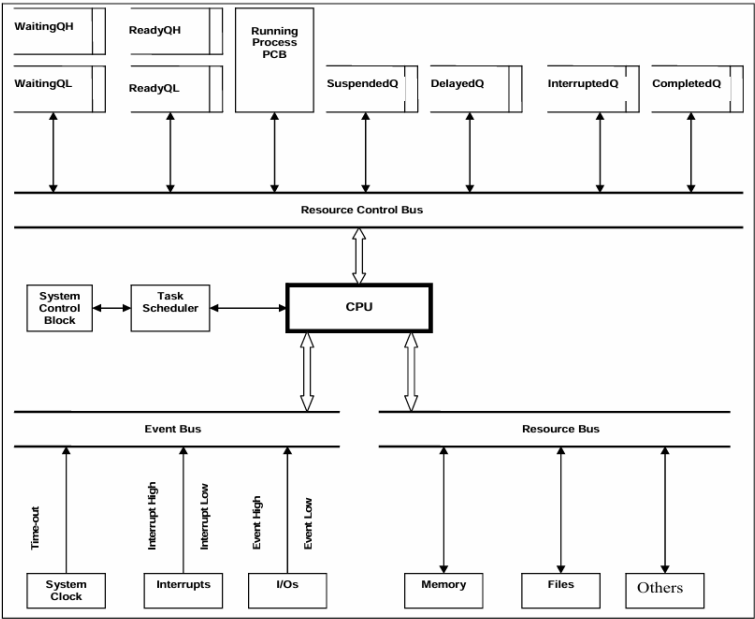


图 1 RTOS+架构

在多队列架构和资源配置的基础上，RTOS+的状态转移图如图 2 所示。任务（进程）请求通过创建状态引入到系统中。当一个新进程生成时，它首先进入等待状态，并具有如图 7 所定义的任务控制块 (TCB) 和唯一任务 ID (Proc#)。系统使用资源控制块 (RCB) 来管理系统资源，如图 7 中所定义。每个处于等待状态的任务都必须检查是否有足够的资源来执行。如果资源可用，则转入就绪状态，否则必须重新排队到等待队列的尾部，直到资源可用。与状态转换图中的状态相对应的是队列，它们在任务从一种状态转换到另一种状态时实际保存任务。基于这个框架，可以解释 RTOS+的任务调度和事件处理算法。

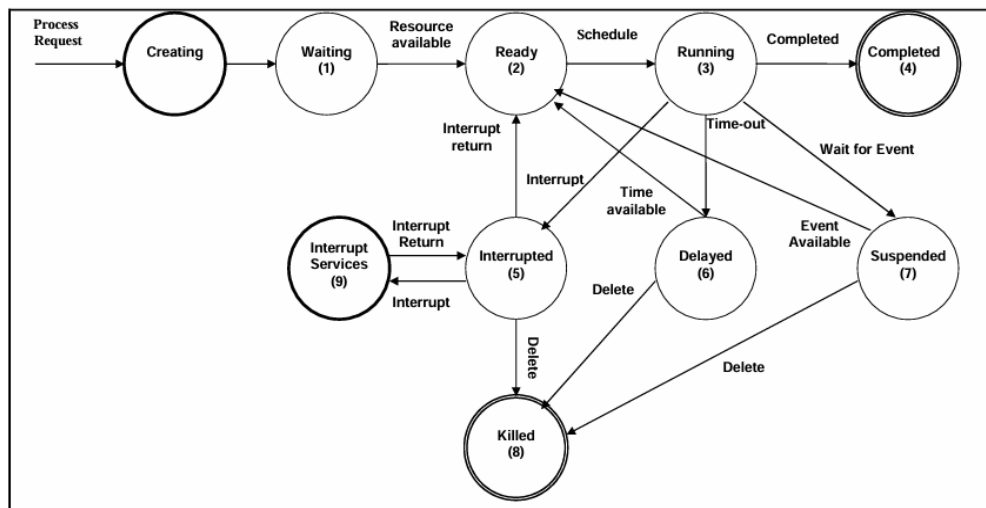


图 2 RTOS+的状态转换图

RTOS+的正式规范是通过使用 RTPA 来进行的。RTPA 中的基本概念是组件逻辑模型 (CLM) 和流程。CLM 描述了系统架构组件的抽象模型，该组件表示系统的硬件接口、内部逻辑模型和/或公共控制结构。进程是对系统行为的最小单元进行建模的软件组件。

RTPA 有一个预定义抽象数据类型 (ADT) 库。我们使用这些 ADT 之一——队列来执行 RTOS+ 中的任务调度和事件处理规范。整个规范中都调用了队列上的相应操作，例如测试空队列（空测试）、从队列中提供元素（出队）以及将元素添加到队列（入队）。基于 RTOS+的体系结构和进程调度策略，开发了 RTOS+的任务调度算法，如图 3 所示。

B. RTOS+的任务调度技术

RTOS+的任务调度是基于优先级和事件驱动的。根据任务的重要性或关键性为任务分配固定的优先级。任务的重要性是当任务满足其时间要求（截止日期）时赋予系统的价值。任务分为四种优先级类型：高级别中断和低级别中断以及高级别任务和基础级别任务。RTOS 中调度中断和例程任务的机制是不同的。例行任务通过时间驱动机制定期调度，对于基本和高级例行任务都使用 FIFO 策略。换句话说，给定优先级的例行任务被分配给 CPU 和相关资源相同的执行优先级。然而，当中断任务发生时，事件驱动机制会使用每个低优先级和高优先级的预定义任务优先级来立即调度中断任务。

任务创建后，将由任务调度程序放入适当的队列中，对应于四个类别之一的预定义优先级类型。根据任务的关键性对任务进行分类是 RTOS+ 中的一个重要考虑因素，通过该分类，任务不仅可以根据其截止日期、优先级和最不松弛时间来保证运行，还可以根据其重要性和关键性来保证任务的运行。

RTOS+ 的任务调度程序负责确定在给定时间应在处理器上执行哪个任务。它与时间管理器和事件管理器协作，在任务之间连续切换处理器。RTOS+转换图处于就绪状态的任务存储在两个队列（ReadyQH 和 ReadyQL）中，分别保存高级别中断任务和低级别事件任务。任务调度程序首先调度 ReadyQH 中的所有任务，然后再调度 ReadyQL 中的任务。任务调度程序不断检查就绪队列以查看是否有任何任务准备好运行。如果有，则比较当前正在执行的任务和队列中第一个任务的优先级。如果队列中的第一个任务具有较高优先级，则它会抢占当前正在运行的任务的 CPU，将 CPU 分配给队列中较高优先级的任务，然后将较低优先级的任务置于挂起状态，如图 2 所示。当 CPU 可用时，挂起的任务最终被重新分派到适当的就绪队列。

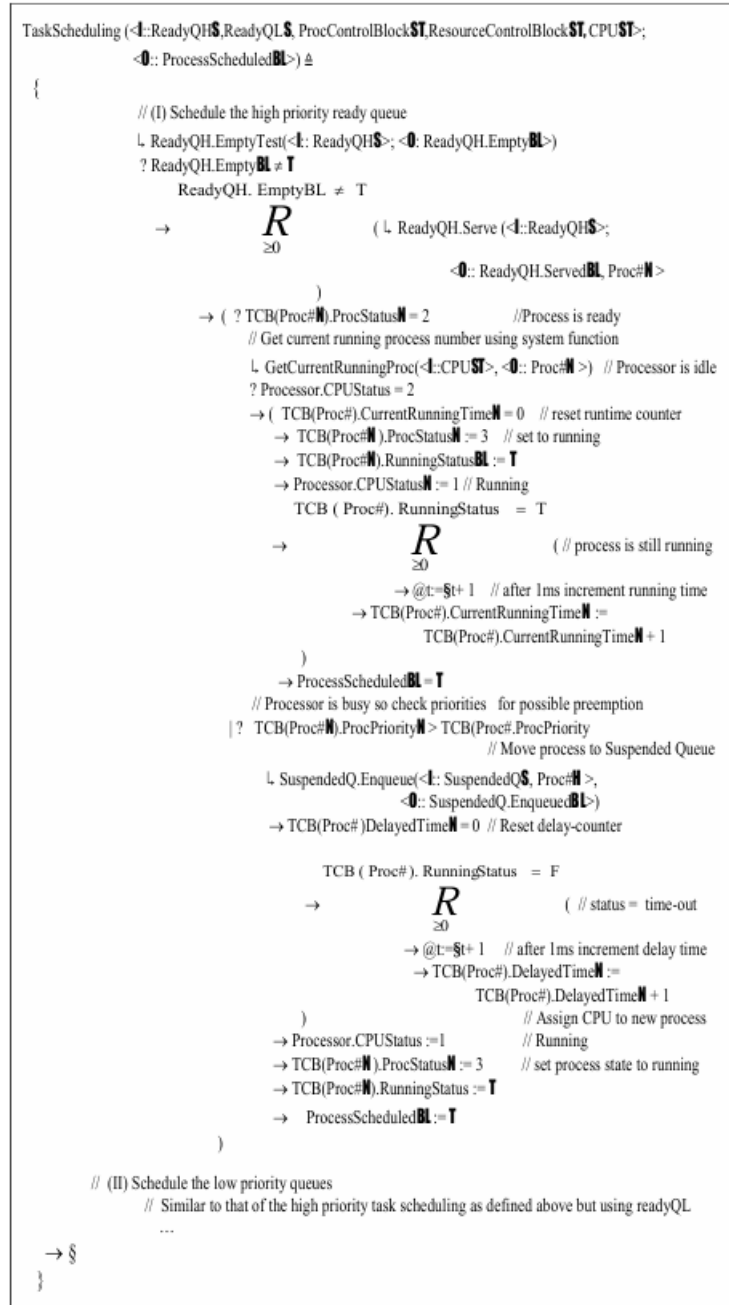


图 3 RTOS+中任务调度规范

RTOS+ 中的每个任务都有一个运行时计数器，用于跟踪处理器上使用的时间。一旦将任务分配给处理器，其运行时间计数器就会根据给定频率（例如 1ms）的系统时钟

递增。以这种方式，时间管理器可以验证使用处理器的任务是否已用尽其时间片，以便应将其发送到延迟队列。如果任务被抢占并延迟，系统会跟踪以查看是否有新的时间片可用于延迟的任务。如果是，该任务将被重新调度到就绪队列中。

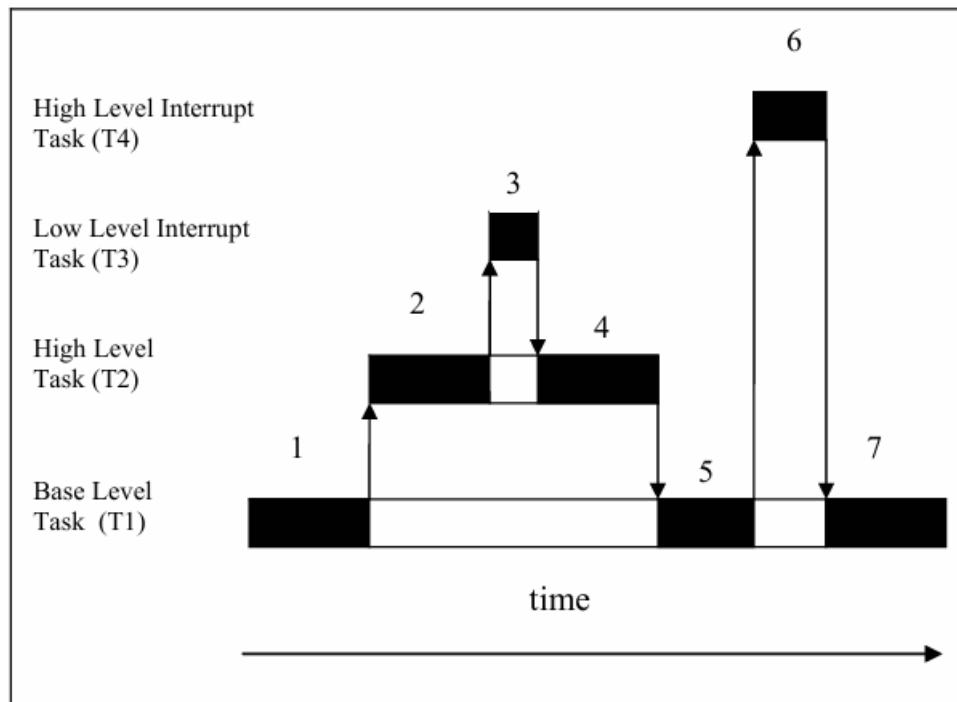


图 4 RTOS+中的抢占式调度方案

图 4 说明了 RTOS+中的抢占式调度方案。基本级别任务 (T1) 可以被高级任务 (T2) 中断。当 T2 在处理器上执行时，异步事件导致低级中断任务 (T3) 中断 T2。控制权移交给 T3。处理完中断后，控制权返回到 T2，然后返回到 T1。当 T1 正在执行时，高级别中断任务 (T4) 准备就绪并抢占 T1。T4 完成后，控制返回到基础级别的 T1。该方案说明了抢占式事件驱动调度方案，其中新事件触发高优先级任务，并且调度器立即响应该任务以满足其定时要求。数字 1、2、3、4、5、6 和 7 表示处理器上不同任务的执行顺序。被抢占的任务将进入挂起状态（图 2），当有可用时隙（Time_Available）时，该任务将返回到就绪队列。

在传统的 RTOS 中，优先级倒置问题是指高优先级任务由于占用公共资源而被较低优先级任务阻止在处理器上执行的情况。通过使用 RTOS+任务调度技术，任务在创建时被分为四个优先级。对于高低中断任务，事件驱动的调度机制保证了中断事件一发生就调度任务。对于基础级和低级例程任务，由于任务是通过 FIFO 机制调度的，因此在同一级内不存在优先级倒置。然而，如果周期性调度的高级别任务和低级别任务之间发生优先级反转，根据 RTOS+调度算法，低级别任务将被挂起，甚至被杀死。因此，采用 RTOS+的新架构和任务调度机制，可以有效消除实时操作系统中的优先级倒置、任务多重阻塞、系统死锁等问题。

C. RTOS+的事件和中断处理技术

事件处理程序确保 RTOS+与创建事件的内部/外部请求之间的交互。RTOS+的事件处理程序检测预定义的事件，并通知调度程序执行相应的任务。在 RTOS+ 中，事件处理程序是资源分配器的一个组件，并与资源管理器协作，以确保因不可用资源而挂起的任务最终获得该资源。第 II(D) 节提供了资源管理的详细描述。事件处理程序的实现将

在第 III(B) 节中讨论。

中断处理是导致正在执行的低优先级任务将 CPU 让给更高优先级任务的特殊情况之一。图 2 中的状态 5 和 9 说明了中断处理的机制。正在运行的任务可能会因较高的中断事件而被中断。如果发生中断，则被中断的任务进入中断状态（状态 5）。中断由中断检测过程（HighPriorityEventScan）识别。一旦识别出中断源，控制权就会传递给被中断的服务（状态 9），该服务响应中断，然后将控制权返回给被中断的任务。中断服务完成后，被中断的任务重新调度到就绪状态。如果资源不足以服务中断，则可以从系统中删除被中断的任务。由于任务是根据其优先级进行调度的，因此高级别中断也可以中断其他较低级别的中断。

D. RTOS+的资源管理技术

图 5 显示了 RTOS+中资源管理的 RPA 规范。内存、I/O 端口和文件等计算机资源是有限的，必须始终共享。为了能够识别任务资源需求，与任务关联的任务控制块（TCB）包含一个字段（ResourceID），该字段指示任务在执行期间所需的资源。每当任务在资源上挂起时，资源标识符就会写入此字段，否则该字段包含空值。对于任何任务，其资源请求可以根据其 TCB 的资源字段来确定。资源管理器确保任务之间系统资源的正确管理。它执行其他职责，例如向系统添加资源、删除不需要的资源；并将资源分配给任务。

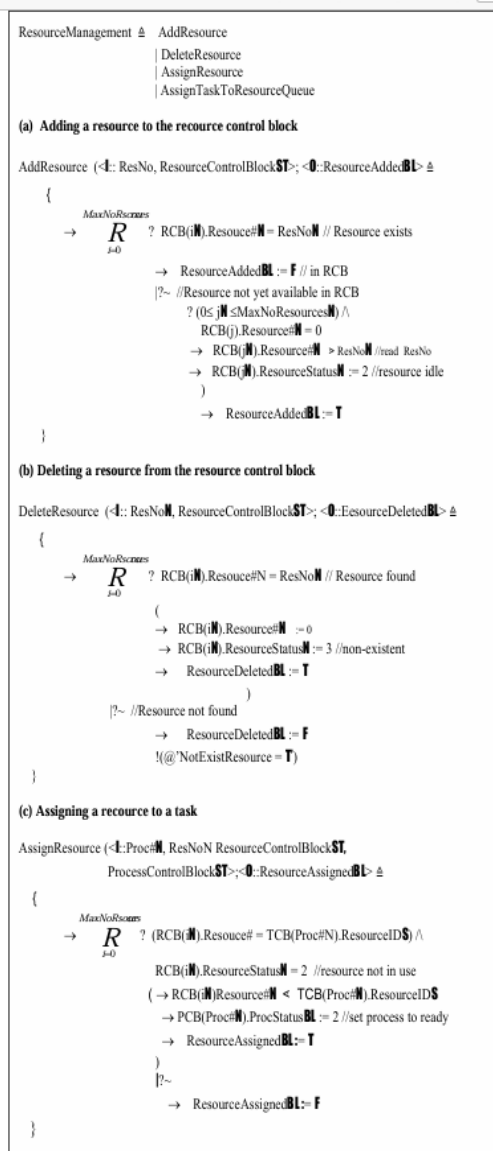


图 5 RTOS+ 中的资源管理

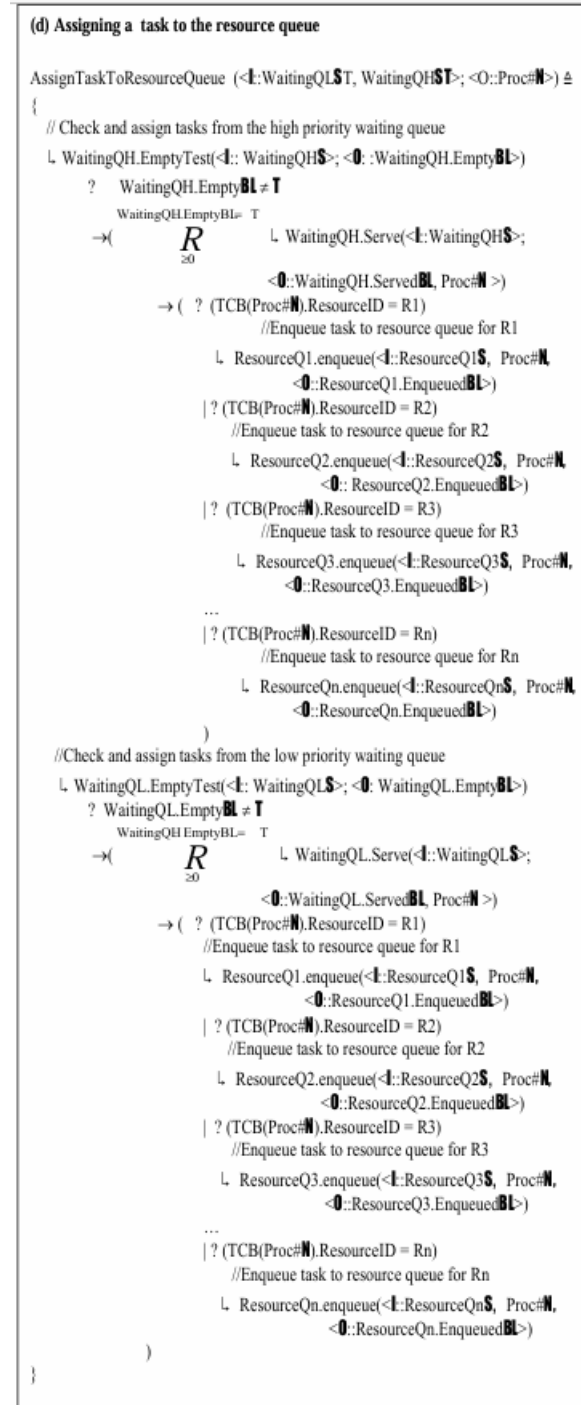


图 5 RTOS+ 中的资源管理（续）

第三章 RTOS+ 中任务调度程序和事件处理程序的实现

在 RTOS+实现的形式化描述中，采用了组件逻辑模型（CLM）。CLM 可以进一步细化为 CLM 模式，CLM 模式是 CLM 的抽象模型，采用记录的形式，通过一组字段

及其类型和约束来指定 CLM 的配置。

A. RTOS+架构组件建模

CLM 的模式包含必要的信息，以便于通过一组进程描述 RTOS+ 的静态和动态行为。在 RTOS+的任务调度实现中，我们使用指定的 CLM 来定义任务调度器操作的数据结构。图 6(a)是 RTOS+架构的顶层规范，图 6(b)是相应的任务调度器的详细顶层规范。

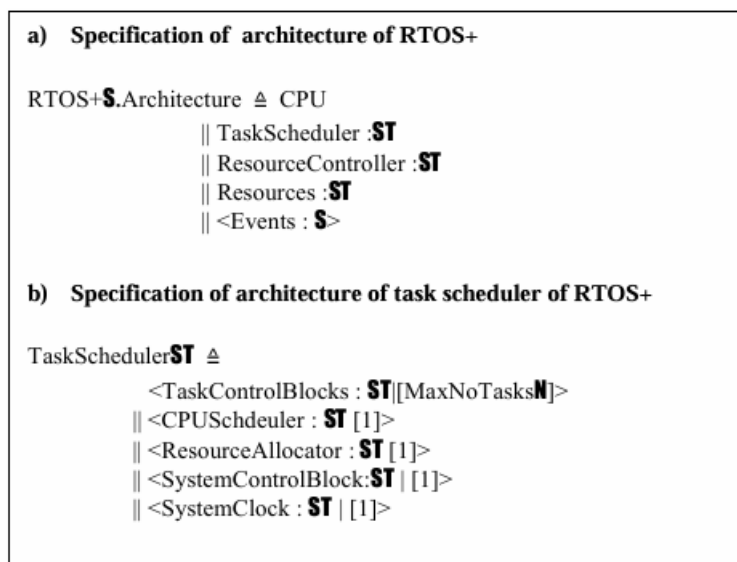


图 6 RTOS+顶层架构定义

图 7 定义了 RTOS+的三个系统 CLM，即任务控制块、资源控制块和系统控制块。它们是基本的控制模型，分别描述了 RTOS+的每个任务、资源和系统控制模型的信息。例如，任务控制块 CLM 保存有关每个任务的以下信息：任务 ID、任务优先级、任务状态、大小、资源标识符、可能的事件、使用的处理器时间、延迟时间以及指向任务易失性环境的指针。每个任务的易失性环境在任务执行期间保存有关寄存器、堆栈指针、状态寄存器和程序计数器的信息。这样的系统类型定义有利于任务调度器的实现。例如，资源管理器使用资源标识符来识别任务的资源请求，并在资源可用时将资源分配给任务。

使用正式指定的 CLM，RTOS+ 的任务调度器由两个主要组件实现：CPU 调度器和资源分配器。图 8 显示了任务调度程序组件之间的交互。主要调度器组件称为 CPU 调度器，负责使用第 II (B) 节中讨论的任务调度算法和图 9 中给出的上下文切换算法在任务之间切换处理器。另一个称为资源分配器的调度程序组件负责将资源分配给系统中运行的不同任务以及管理系统事件。它还将新任务引入系统并释放附加到已完成或终止的任务的资源。

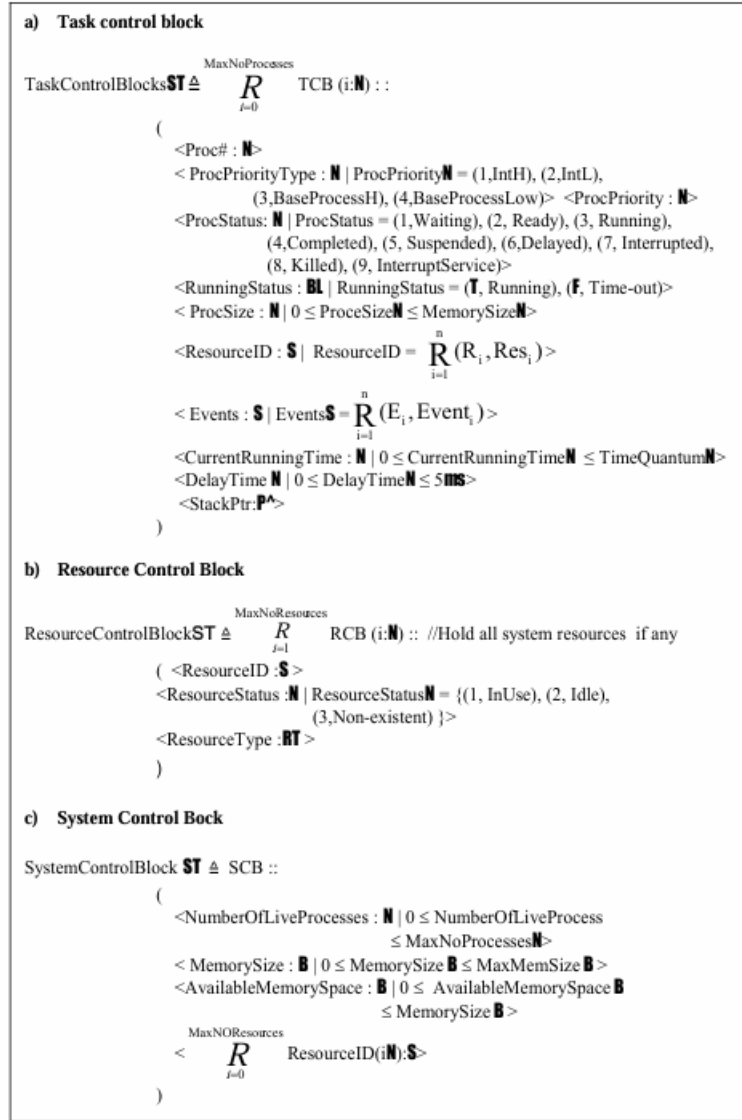


图7 RTOS+中的 CLM 规范

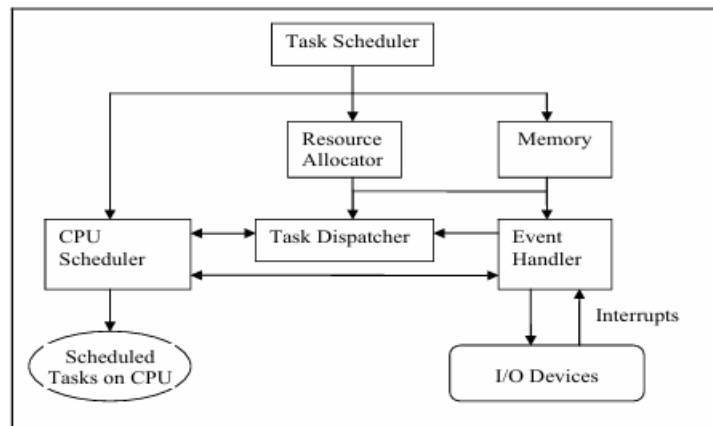


图8 RTOS+ 中任务调度程序和事件处理程序之间的交互

以下小节将解释用于实现任务调度程序各个组件的算法。

B. RTOS+的 CPU 调度器

CPU 调度器可以使用第 II(B)节中指定的算法来实现。这里采用 ReadyQH 和 ReadyQL 两个队列来调度不同优先级的任务。任务调度器首先检查 CPU 的状态以及当前运行任务的优先级。如果 CPU 空闲或者当前正在执行较低优先级的任务，则在当前任务挂起后，将分配队列中具有较高优先级的新任务。任务（进程）由任务调度程序放入就绪队列，以便由 CPU 调度程序执行。当发生触发新任务并导致 RTOS+ 中的调度状态发生变化的事件时，采用事件处理程序通知 CPU 调度程序。CPU 调度程序使用图 9 中的上下文切换算法来在事件处理程序发出警报时确定上下文切换。

```

ContextSwitchAlgorithm(<I:: TaskControlBlockST, CPUST>; <O::>) Δ
{
  ↳ getTaskID(<I::CurrentTaskIDS>;<O::>)
  ? CurrentRunningTaskIDS.ProcPriority < ReadyTaskIDS.ProcPriorityN
    → ( ? ReadyTaskIDS.ResourceS = null    // task does not need any resource
        → L1: (
            ↳ SaveEnvironment (<I::CurrentRunningTaskIDS>;<O::>)
            ↳ SetUpEnvironment (<I::ReadyTaskIDS>;<O::>)
            ↳ SwithProcessor (<I::ReadyTaskIDS>;<O::>)
          )
        | ? ~
        → Ø // continue executing current task until resource becomes ready
        ↳ ResourceManager (<I::ReadyTaskIDN>; <O::resource_availableBL>
          ? ReadyTaskIDN.resource = null // resource field contains null
                                          // value if task does not need resource
          → L1 // Jump to L1 above
          | ? ~
          → Ø // continue executing current task if it has higher priority.
        )
    )
}

```

图 9 RTOS+的上下文切换算法

C. 资源分配器

资源分配器使用两个组件实现：任务调度程序和事件处理程序，如图 10 和 11 所示。

任务调度程序负责根据任务的当前状态和需求将各种任务调度到适当的队列。图 10 是 RTOS+的任务调度算法的 RTPA 规范。它通常负责管理系统中的所有任务队列，并确保一旦触发事件，每个任务就被发送到适当的队列。任务调度器使用第二(B)章中描述的任务调度算法和图 10 中指定的任务调度算法来实现。任务调度程序不断检查每个任务的状态，并将它们调度到适当的队列。只有调度到就绪队列的任务才会由 CPU 调度程序执行。


```

TaskDispatcher (<I::TaskControlBlockST>;<O::TaskdispatchedBL>)△
{
    SysShutDow nBL ≠ T
    
$$R_{\geq 0}$$
 //Run continuously
    {
        L GetTaskId(I:: TaskIDS; <O:: TaskDispatchedBL>)
        ? TaskIDS.ProcStatusN = 1 //Waiting
        L WaitingQL.Enqueue(I:: WaitingQLS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 2 ∧
        ( ( taskIDS. ProcPriorityTypeN = 1) ∨ //interrupt
          taskIDS. ProcPriorityTypeN = 2))
        L ReadyQH.Enqueue(I:: ReadyQHS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 1 ∧
        ( (TaskIDS. ProcPriorityTypeN = 3) ∨ //base task
          TaskIDS. ProcPriorityTypeN = 4))
        L ReadyQL.Enqueue(I:: ReadyQLS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 4 //Completed task
        L CompletedQ.Enqueue(I:: CompletedQS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 5 //Suspended task
        L SuspendedQ.Enqueue(I:: SuspendedQS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 6 // Delayed task
        L DelayedQ.Enqueue(I:: DelayedQS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 7 // Interrupted task
        L InterruptedQ.Enqueue(I:: InterruptedQS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 8 //Killed task
        L KilledQS.Enqueue(I:: KilledQS,TaskIDS>
        | ? TaskIDS.ProcStatusN = 9 //Interrupted services
        L InterruptedServicesQS.Enqueue (I::
          InterruptedServicesQS,TaskIDS>
    }
}

```

图 10 RTOS+的任务调度算法

RTOS+ 资源分配器的第二个组件是事件处理程序。图 11 给出了 RTOS+ 事件处理程序的架构。它监视系统可接受的事件，并确保将某些事件待处理的任务放入挂起队列中。事件处理程序还管理中断、I/O 事件和其他与系统资源（特别是内存）相关的事件。当事件发生时，它会通知 CPU 调度程序和任务调度程序，从而采取适当的操作。RTOS+ 中的事件处理算法已在第二(C)章中进行了描述。它与任务调度程序协作，为优先级倒置问题提供解决方案。事件处理程序由四个进程实现，即 EventListener、EventRegister、EventNotifier 和 InterruptHandler。

```

EventHandler (<I::SystemControlBlockST, CPUST>; <O::>)
{
    SysShutDow nBL ≠ T
    
$$R_{\geq 0}$$
 // run continuously
    {
        L EventListener (<I:: SystemControlBlockST>;<O::eventCapturedBL>)
        L EventRegister (I:: SystemControlBlockST>;<O::eventRegisteredBL>)
        L EventNotifier (I::CPUST>;<O::NotifiedBL>);
        L InterruptHandler (<I::CPUST,SystemControlBlockST>;
          <O::interruptEventBL>);
    }
}

```

图 11 RTOS+ 事件处理程序的架构

EventListener 使用两个函数 HighPriorityEventScan() 和 BaseEventScan() 扫描外部和内部事件，这两个函数分别扫描中断事件和基本事件。EventRegister 将传入事件注册到适当的事件请求队列中，EventNotifier 通知 CPU 调度程序任何未决事件，以便实现适当的上下文切换或中断处理程序。一旦收到通知，中断服务例程 (ISR) 就会执行适当的操作来服务中断。中断处理程序使用图 12 中给出的算法来处理中断。

```

InterruptHandler(<I::CPUST, SystemControlBlockST>; <O::>) Δ
{
    ↳ HighPriorityEventScan (<I::CPUST, SystemControlBlockST>;
                           <O::EventIDS, interruptEventBL>)
    ? InterruptEventBL = T
      → (
          ↳ SaveCPURegisters(<I::CPUST>; <O::RegistersSavedBL>
          ↳ ISR (<I::EventIDS>; <O::interruptServiceBL>)
          ↳ RestoreCPURegisters(<I::CPUST>; <O::RegisteredStoredBL>)
          ↳ ExitISR(<I::>; <O::RoutineExitedBL>)
          ↳ InterruptReturn(<I::>; <O::interruptReturnedBL>)
        )
    |?~ //No interrupt
      → ∅ // Do nothing
}

```

图 12 中断处理程序的算法

第四章 结论

实时操作系统在控制面向过程和安全关键的应用程序中发挥着重要作用。因此，应高度重视其设计和实施，以便提供所需的可预测性、及时性和可靠性。RTOS 的严格规范，特别是任务调度程序及其与实时操作系统内核中其他组件的关系，是 RTOS 架构建模和实现的基本需求。

本文报道了 RTOS+实时操作系统的任务调度器和事件处理程序的开发。RTOS+任务调度和事件处理技术及其算法已在 RTPA 中得到正式描述。使用形式化规范，介绍并讨论了 RTOS+ 中任务调度和事件处理的实现。RTOS+的核心组件是使用基于多队列的调度系统来实现的。该技术有助于消除优先级反转问题，并防止任务多重阻塞和死锁的发生。

该项目未来的工作是实现一个功能齐全的 RTOS+，将本文开发的任务调度程序和事件处理程序集成到一个连贯的框架中。RTOS+ 还将作为基于 RTPA 的代码生成器的支持环境的一部分应用。

参考文献

- [1] Y.C. Wang 和 K. Lin, “在 RED-Linux 实时内核中实现通用实时调度框架”, Proc. 第 20 届 IEEE 实时系统研讨会, 第 246 页, 1999 年。
- [2] D. Kalinsky 和 M. Baril, “优先级反转简介”, 嵌入式系统编程, CMP, LLC, <http://www.embedded.com/story/>, 2002 年。
- [3] J.H. Lee 和 H. N. Kim, “在 uC/OS 实时内核中实现优先级信号量”, IEEE 未来嵌入式系统软件技术研讨会, 日本, 2003 年 5 月。
- [4] L. Sha、R. Rajkumar 和 J.P. Lehoczky, “优先级继承协议: 一种实时同步方法”, IEEE Transactions Computers, 1990 年 9 月, 第 14 页。 1175.
- [5] J.B. Goodenough 和 L. Sha, “优先级上限协议: 一种最小化高优先级 Ada 任务阻塞的方法”, ACM SIGAda Ada Letters, 第 VIII 卷第 7 卷, 1998 年 6 月, 第 20-31 页。
- [6] J. Kreuzinger、U. Brinkschulte、M. Pfeffer、S. Uhrig 和 T. Ungerer。 “多线程 Java 微控制器上的实时事件处理和调度”, Microprocessors Microsystems, 27(1), 2003 年 2 月, 第 19-31 页。
- [7] Y. Wang 和 C. F. Ngolah, “使用 RTPA 的实时操作系统的形式化描述”, Proc. 2003 年 IEEE 加拿大电气与计算机工程会议 (CCECE'03), 加拿大蒙特利尔, 2003 年 5 月, 第 1247-1250 页。
- [8] C. F. Ngolah、Y. Wang 和 X. Tan, “在 RTOS+ 中实现任务调度和事件处理”, Proc. 2004 年加拿大电气与计算机工程会议 (CCECE'04) 的报告, IEEE CS Press, 加拿大安大略省尼亚加拉瀑布, 2004 年 5 月, 第 1523-1526 页。
- [9] Y. Wang, “操作系统”, 第一章。 144, 载于 R. Dorf 编辑, 《工程手册》, 第 2 版, CRC Press, 2004 年。
- [10] Y. Wang, “实时过程代数 (RTPA)”, 软件工程年鉴, 卷 14, Kluwer 学术出版社, 2002 年, 第 235-274 页。
- [11] Y. Wang, “实时系统规范的新方法”, Proc. 2002 年 IEEE 加拿大电气与计算机工程会议 (CCECE'02), 加拿大温尼伯, MB, 2002 年 5 月, 第 1 卷。 2, 第 663-668 页。
- [12] T. Xinming 和 Y. Wang, “使用实时过程代数的抽象数据类型规范”, Proc. 2003 年加拿大电气与计算机工程会议 (CCECE'03), IEEE CS 出版社, 加拿大蒙特利尔, 2003 年 5 月, 第 35.2.1-4 页。

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/255962355>

The real-time task scheduling algorithm of RTOS+

Article in Canadian Journal of Electrical and Computer Engineering · October 2004

DOI: 10.1109/CJECE.2004.1426051

CITATIONS

17

READS

939

3 authors, including:



Yingxu Wang

The University of Calgary

860 PUBLICATIONS 15,295 CITATIONS

SEE PROFILE



Xinming Tan

The University of Calgary

23 PUBLICATIONS 141 CITATIONS

SEE PROFILE

The Real-Time Task Scheduling Algorithm of RTOS+

Cyprian F. Ngolah, Yingxu Wang, and Xinming Tan

Theoretical and Empirical Software Engineering Research Center
Department of Electrical and Computer Engineering
University of Calgary
2500 University Drive NW, Calgary, AB, Canada T2N 1N4
Email: {ncfoinjo, yingxu, xintan}@ucalgary.ca

Abstract

An important component in the kernel of a real-time operating system (RTOS) is the task scheduler. This paper develops a new portable RTOS architecture, RTOS+, which adopts a novel multi-queue-based task scheduling methodology. The task scheduler coordinates tasks with various run-time resources, events, and interrupts efficiently to prevent the priority-inversion problem from happening. Real-time process algebra (RTPA), a formal specification notation, is adopted to rigorously describe the architecture, static and dynamic behaviours of the task scheduler. Based on the RTOS+ architecture and algorithms, a framework of task scheduling and event handling is implemented on the platform of RTOS+. The algorithm and RTOS+ have been applied in the development of the supporting environment of the RTPA-based code generator in software engineering.

Keywords: *Software engineering, real-time, operating system, task scheduling, algorithms, RTOS+, RTPA*

I Introduction

A real-time operating system (RTOS) is an operating system that supports and guarantees timely responses to external and internal events of real-time applications. A major component of the kernel of an RTOS is the task scheduler that is responsible for assigning system resources to a task according to a given scheduling policy. Due to the unpredictable nature of external events, care must be taken in implementing the task scheduler so that it is capable of reacting to events in a strictly timed manner. Various task scheduling algorithms and schemes have been developed for implementing RTOS's, which can be classified as the *time-driven*, *priority-driven* and *share-driven* schemes [1]. Most commercial RTOS's use the priority-driven task scheduling scheme.

Priority-driven techniques assign priorities to tasks based on their requirements and system resource availability. A task with the highest priority is selected and executed by the scheduler. Priority-driven scheduling techniques can be classified into *static* and *dynamic* priority policies. In the former, the priority of a task is fixed and remains constant when it is created. In the latter, the priority of a task may change at run-time depending on system environment and resource availability.

A common problem often encountered when implementing task scheduling in RTOS's using the priority-driven scheme is priority inversion [2]. This problem arises

when tasks with different priorities share common resources. Much work has been done to address this problem and various solutions such as the basic priority inheritance and priority ceiling protocols have been proposed and implemented [2-5]. However, these solutions often result in side-effects such as multiple blocking of tasks and deadlocks [3].

This paper expands on the task scheduling and event-handling algorithms proposed in RTOS+, which is a portable real-time operating system developed in [7-8]. It presents a formal description of task scheduling and event-handling techniques, and their implementation in RTOS+. The algorithms for task scheduling and event handling are formally specified using Real-Time Process Algebra (RTPA) [9-10] in Section II. The implementation of the task scheduler and event handler for RTOS+ is described in Section III. Summaries of this work and future development are discussed in Section IV.

II Task Scheduling in RTOS+

Task scheduling is the assignment of tasks to the processor and related resources in an operating system. In real-time systems, all tasks are defined by their timing specifications such as their deadline, type (periodic or sporadic), and required resources. Sporadic tasks are tasks associated with event-driven processing such as responding to user inputs. Periodic tasks are tasks scheduled at regular intervals. Various scheduling paradigms exist for real-time

systems [6] but most commercial RTOS's however use the priority-driven scheduling scheme in which a static or dynamic priority is assigned to each task based on a given algorithm. Popular existing algorithms for the priority-driven scheduling scheme include the rate monotonic scheduling (RMS) algorithm, the earliest deadline first (EDF) scheduling algorithm, and the least laxity first (LLF) scheduling algorithm [6].

Such algorithms employ a scheduling scheme in which the priority of a task is based on the timing constraints such as period, frequency or deadline of the task. A priority driven scheduling scheme based on these attributes has two major problems. The first problem is that the priority attribute is defined by the timing constraints but the importance of the task is not considered. It will therefore, be difficult to schedule a task with a long period or late deadline that needs a fast response time. The second problem stems from the fact that for the priority-driven scheduling scheme to be effective, the task scheduler should be pre-emptive. This means that the task scheduler should allow a higher priority task to 'break in' on the execution of a lower priority task by taking over the processor. The problem with pre-emptive task scheduling in conventional RTOS's is that it leads to 'priority-inversion' if resources must be shared between various tasks in the application [2-5]. Limited computing resources in an RTOS require that those resources must be shared by several tasks running concurrently. Therefore, for an RTOS to fulfill its desired features of predictability, timeliness and reliability, it is necessary to use a scheduling algorithm that can eliminate priority inversion and avoid deadlock and multiple blocking of tasks. In [7], we developed a formal description of a portable real-time operating system RTOS+. We expand this work on task scheduling and event handling in this paper by presenting a formalized algorithm that minimizes the chance of priority inversion and deadlock in RTOS+.

A. Architecture of RTOS+

The architecture of RTOS+ is shown in Fig.1, where interactions between system resources, components, and internal control models are illustrated. There are four subsystems in RTOS+: a) the processor and task scheduler, b) the resource controller, c) the event handler, and d) the system resources. The task scheduler is the innermost operating system kernel that directly controls the switching of the CPU among tasks and all other system resources. The resources of RTOS+, supplement to the CPU, are mainly the memory, system clock, I/O ports, interrupt facilities, files, and internal control models such as queues and task/resource control blocks as shown in Fig. 1. The different queues hold tasks at the same priority level. This makes it easy for the task scheduler to assign the processor to the first task from each ready queue.

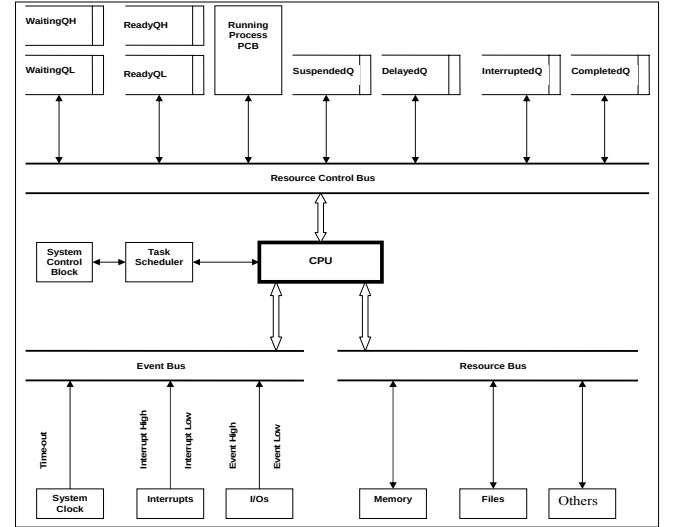


Fig. 1 Architecture of RTOS+

On the basis of the multi-queue architecture and resource configuration, a state transition diagram of RTOS+ is shown in Fig. 2. Task (process) requests are introduced into the system through the *creating state*. When a new process is generated, it is first put into the *waiting state* with a task control block (TCB) and a unique task ID (Proc#) as defined in Fig. 7. The system uses a resource control block (RCB) to manage system resources as defined in Fig. 7. Each task in the waiting state must be checked to see if there are enough resources for its execution. If resources are available, it is transferred into the *ready state*, otherwise it has to be re-queued at the tail of the waiting queue until resources are available. Corresponding to the states in the state transition diagram are queues that actually hold tasks as they transit from one state to another. Based on this framework, the task scheduling and event handling algorithms of RTOS+ can be explained.

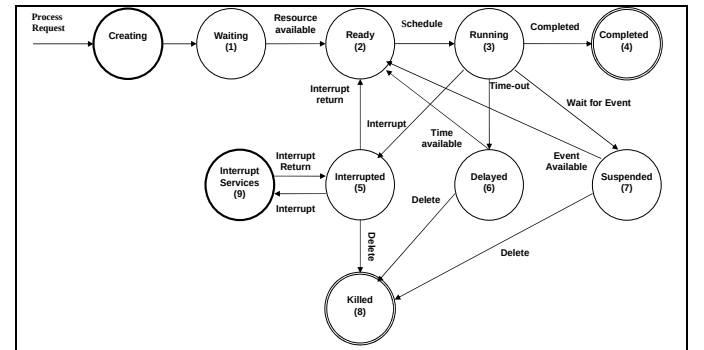


Fig. 2 State transition diagram of RTOS+

The formal specification of RTOS+ is carried out by using RTPA [10, 11]. Fundamental concepts in RTPA are the component logical models (CLMs) and the processes

[11]. A *CLM* describes the abstract model of a system architectural component that represents either a hardware interface, an internal logical model and/or a common control structure of a system. A *process* is a software component that models the minimum unit of system behaviors.

RTPA has a library of predefined abstract data types (ADTs) [12]. We use one of these ADTs, *queue*, in carrying out the specification of task scheduling and event handling in RTOS+. Corresponding operations on queues such as testing for an empty queue (empty test), serving an element from the queue (dequeue) and adding an element to the queue (enqueue) are called throughout the specification. Based on the architecture and process scheduling strategies of RTOS+, the task scheduling algorithm of RTOS+ is developed as shown in Fig. 3.

B. Task Scheduling Techniques of RTOS+

The task scheduling of RTOS+ is priority based and event driven. Fixed priorities are assigned to tasks based on their importance or criticality. The importance of a task is the value imparted to the system when the task satisfies its timing requirement (deadline). Tasks are categorized into four priority types: the *high* and *low level interrupts*, and *high* and *base level tasks*. The mechanisms for scheduling interrupts and routine tasks are different in RTOS+. The routine tasks are scheduled periodically by *time-driven mechanism* using the FIFO strategy for both the base and high level routine tasks. In other words, the routine tasks in a given priority level are assigned the same executing priority for CPU and related resources. However, the interrupt tasks are scheduled immediately when they occur by *event-driven mechanism* using predefined priorities of tasks at each of the low and high priority levels.

A task, when created, will be put in a proper queue by the task dispatcher, corresponding to its predefined priority types in one of the four categories. The categorization of tasks based on their criticality is an important consideration in RTOS+, by which tasks are not only guaranteed to run based on their deadlines, priorities, and least laxity time [6], but also on their importance and criticality.

The task scheduler of RTOS+ is responsible for determining which task should be executed on the processor at a given time. It works in collaboration with the time manager and event manager to continuously switch the processor between tasks. Tasks in the ready state of the RTOS+ transition diagram are stored in two queues (*ReadyQH* and *ReadyQL*) that hold high level interrupt tasks and low level event tasks respectively. The task scheduler first schedules all tasks in *ReadyQH* before scheduling those in *ReadyQL*. The task scheduler continuously checks the ready queues to see if there is any task ready to be run. If there is, it compares the priorities of the currently executing

task and that of the first task in the queue. If the first task in the queue has a higher priority, it pre-empts the CPU from the currently running task, assigns the CPU to the higher priority task from the queue, after sending the lower priority task to the suspended state as shown in Fig. 2. The suspended task is eventually re-dispatched to the appropriate ready queue when the CPU is available.

```

TaskScheduling (<L: ReadyQH, ReadyQL, ProcControlBlockST, ResourceControlBlockST, CPUST>;
    <O: ProcessSchedulBL>) Δ
{
    // (I) Schedule the high priority ready queue
    L ReadyQH.EmptyTest(<L: ReadyQHST>; <O: ReadyQH.EmptyBL>)
    ? ReadyQH.EmptyBL ≠ T
        ReadyQH.EmptyBL = T
        → 
$$\begin{matrix} R \\ \geq 0 \end{matrix}$$
 (L ReadyQH.Serve(<L: ReadyQHST>;
            <O: ReadyQH.ServedBL, Proc#N>
        ))
        → ( ? TCB(Proc#N).ProcStatusN = 2 // Process is ready
            // Get current running process number using system function
            L GetCurrentRunningProc(<L: CPUST>; <O: Proc#N>) // Processor is idle
            ? Processor.CPUStatus = 2
            → ( TCB(Proc#N).CurrentRunningTimeN = 0 // reset runtime counter
                → TCB(Proc#N).ProcStatusN = 3 // set to running
                → TCB(Proc#N).RunningStatusBL = T
                → Processor.CPUStatusN = 1 // Running
                TCB(Proc#N).RunningStatus = T
                → 
$$\begin{matrix} R \\ \geq 0 \end{matrix}$$
 ( // process is still running
                    → @t:=t+1 // after 1ms increment running time
                    → TCB(Proc#N).CurrentRunningTimeN :=
                        TCB(Proc#N).CurrentRunningTimeN + 1
                )
                → ProcessSchedulBL = T
            // Processor is busy so check priorities for possible preemption
            | ? TCB(Proc#N).ProcPriorityN > TCB(Proc#N).ProcPriority
                // Move process to Suspended Queue
                L SuspendedQ.Enqueue(<L: SuspendedQST, Proc#H>;
                    <O: SuspendedQ.EnqueueBL>)
                → TCB(Proc#N).DelayedTimeN = 0 // Reset delay-counter
                TCB(Proc#N).RunningStatus = F
                → 
$$\begin{matrix} R \\ \geq 0 \end{matrix}$$
 ( // status = time-out
                    → @t:=t+1 // after 1ms increment delay time
                    → TCB(Proc#N).DelayedTimeN :=
                        TCB(Proc#N).DelayedTimeN + 1
                )
                → Processor.CPUStatus := 1 // Running
                → TCB(Proc#N).ProcStatusN := 3 // set process state to running
                → TCB(Proc#N).RunningStatus := T
                → ProcessSchedulBL := T
            )
        )
    // (II) Schedule the low priority queues
    // Similar to that of the high priority task scheduling as defined above but using readyQL
    ...
}

```

Fig. 3 Specification of task scheduling in RTOS+

Each task in RTOS+ has a run-time counter which keeps track of time used on the processor. Once a task is assigned to the processor, its run-time counter is incremented based on the system clock at a given frequency, say 1ms. In this way, the time manager may verify if a task using the processor has exhausted its time-slice so that it should be

sent to the delayed queue. If a task is pre-empted and delayed, the system keeps track to see if a new time slice is available for the delayed task. If so, the task will be rescheduled into the ready queue.

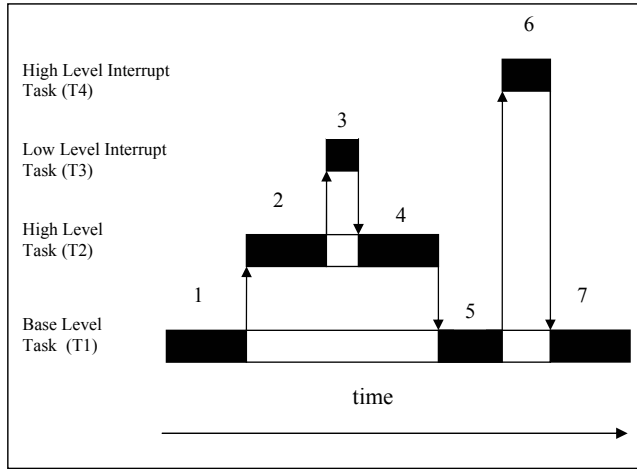


Fig. 4 Preemptive scheduling scheme in RTOS+

Fig. 4 illustrates the preemptive scheduling scheme in RTOS+. A base level task (T1) can be interrupted by a high level task (T2). While T2 is executing on the processor, an asynchronous event causes the low level interrupt task (T3) to interrupt T2. Control is handed over to T3. After servicing the interrupt, control is returned to T2 and then to T1. While T1 is executing, a high level interrupt task (T4) becomes ready and preempts T1. Upon completion of T4, control is returned to T1 at the base level. This scheme illustrates a preemptive event-driven scheduling scheme in which a new event triggers a high priority task and the scheduler responds to this task immediately in order for it to satisfy its timing requirement. The numbers 1, 2, 3, 4, 5, 6 and 7 indicate the execution sequence of the different tasks on the processor. A task preempted is sent to the suspended state (Fig. 2), which will be returned back to the ready queue when a time slot is available (*Time_Available*).

In conventional RTOS's, the priority inversion problem is a situation where a high priority task is blocked from executing on the processor by a lower priority task due a common resource that it occupies. By using the RTOS+ task scheduling technology, tasks are placed in four priority levels when they are created. For the high and low interrupt tasks, the event-driven scheduling mechanism ensures that a task is scheduled as soon as the interrupt event occurs. For the base and low level routine tasks, there is no priority inversion within the same level since tasks are scheduled by the FIFO mechanism. However, if a priority inversion would happen between the periodically scheduled high and low level tasks, the low level task will be suspended, even killed, according to the RTOS+ scheduling algorithm. Therefore,

adopting the new architecture and task scheduling mechanisms of RTOS+, the problems of priority inversion, multiple blocking of tasks, and system deadlock can be efficiently eliminated in real-time operating systems.

C. Event and Interrupt Handling Techniques of RTOS+

An event handler ensures the interaction between the RTOS+ and internal/external requests that create an event. The event handler of RTOS+ detects predefined events, and informs the scheduler to execute corresponding task(s). In RTOS+, the event handler is a component of the resource allocator and works in collaboration with the resource manager to ensure that tasks suspended for an unavailable resource eventually get it. A detailed description of resource management is provided in Section II(D). The implementation of the event handler is discussed in Section III(B).

Interrupt handling is one of the special cases that cause an executing low priority task to give up the CPU to a higher priority task. States 5 and 9 in Fig. 2 illustrate the mechanism of interrupt handling. A running task can be interrupted due to a higher interrupt event. If an interrupt occurs, the interrupted task goes to the *interrupted* state (State 5). The interrupt is identified by the interrupt detection process (*HighPriorityEventScan*). Once the interrupt source is identified, control is passed to the *interrupted service* (State 9), which responds to the interrupt and then returns control to the interrupted task. The interrupted task is re-scheduled to the *ready* state after completion of the interrupt service. An interrupted task may be deleted from the system if resources are not enough for servicing the interrupt. Since tasks are scheduled based on their priorities, high level interrupts can also interrupt other lower level interrupts.

D. Resource Management Techniques of RTOS+

Fig. 5 shows the RTPA specification of resource management in RTOS+. Computer resources such as memory, I/O ports, and files are limited and must always be shared. To be able to identify task resource needs, the task control block (TCB) associated with the task contains a field (*ResourceID*) that indicates the required resource of the task during execution. Whenever a task is pending on a resource, the resource identifier is written on this field, otherwise the field contains a null value. For any task, its resource requests can be determined from the resource field of its TCB. The resource manager ensures the proper management of system resources between tasks. It carries out other duties such as adding resources to the system, deleting resources that are not needed; and assigning resources to tasks.


```

ResourceManagement  $\triangleq$  AddResource
                    | DeleteResource
                    | AssignResource
                    | AssignTaskToResourceQueue

```

(a) Adding a resource to the resource control block

```

AddResource (<I: ResNo, ResourceControlBlockST>; <O: ResourceAddedBL>  $\triangleq$ 
{
   $\rightarrow \prod_{i=0}^{MaxNoResources} ? RCB(iN).Resource\#N = ResNoN$  // Resource exists
   $\rightarrow ResourceAddedBL := F$  // in RCB
  |?~ //Resource not yet available in RCB
  ? (0  $\leq jN \leq MaxNoResourcesN$ )  $\wedge$ 
    RCB(jN).Resource#N = 0
   $\rightarrow RCB(jN).Resource\#N > ResNoN$  //read ResNo
   $\rightarrow RCB(jN).ResourceStatusN := 2$  //resource idle
  )
   $\rightarrow ResourceAddedBL := T$ 
}

```

(b) Deleting a resource from the resource control block

```

DeleteResource (<I: ResNoN, ResourceControlBlockST>; <O: ResourceDeletedBL>  $\triangleq$ 
{
   $\rightarrow \prod_{i=0}^{MaxNoResources} ? RCB(iN).Resource\#N = ResNoN$  // Resource found
  (
     $\rightarrow RCB(iN).Resource\#N := 0$ 
     $\rightarrow RCB(iN).ResourceStatusN := 3$  //non-existent
     $\rightarrow ResourceDeletedBL := T$ 
  )
  |?~ //Resource not found
   $\rightarrow ResourceDeletedBL := F$ 
  !(@'NotExistResource = T')
}

```

(c) Assigning a resource to a task

```

AssignResource (<I: Proc#N, ResNoN ResourceControlBlockST,
                ProcessControlBlockST>; <O: ResourceAssignedBL>  $\triangleq$ 
{
   $\rightarrow \prod_{i=0}^{MaxNoResources} ? (RCB(iN).Resource\# = TCB(Proc\#N).ResourceIDS) \wedge$ 
    RCB(iN).ResourceStatusN = 2 //resource not in use
  (  $\rightarrow RCB(iN).Resource\#N < TCB(Proc\#N).ResourceIDS$ 
     $\rightarrow PCB(Proc\#N).ProcStatusBL := 2$  //set process to ready
     $\rightarrow ResourceAssignedBL := T$ 
  )
  |?~
   $\rightarrow ResourceAssignedBL := F$ 
}

```

(d) Assigning a task to the resource queue

```

AssignTaskToResourceQueue (<I: WaitingQLST, WaitingQHST>; <O: Proc#N>  $\triangleq$ 
{
  // Check and assign tasks from the high priority waiting queue
   $\hookrightarrow WaitingQH.EmptyTest(<I: WaitingQHS>; <O: WaitingQH.EmptyBL>)$ 
  ? WaitingQH.EmptyBL  $\neq T$ 
     $\rightarrow$  (  $\prod_{i=0}^{WaitingQH.EmptyBL = T} \hookrightarrow WaitingQH.Serve(<I: WaitingQHS>;$ 
       $<O: WaitingQH.ServedBL, Proc\#N>)$ 
       $\rightarrow$  ( ? (TCB(Proc\#N).ResourceID = R1)
        //Enqueue task to resource queue for R1
         $\hookrightarrow ResourceQ1.enqueue(<I: ResourceQ1S, Proc\#N,$ 
           $<O: ResourceQ1.EnqueueedBL>)$ 
        | ? (TCB(Proc\#N).ResourceID = R2)
          //Enqueue task to resource queue for R2
           $\hookrightarrow ResourceQ2.enqueue(<I: ResourceQ2S, Proc\#N,$ 
             $<O: ResourceQ2.EnqueueedBL>)$ 
        | ? (TCB(Proc\#N).ResourceID = R3)
          //Enqueue task to resource queue for R3
           $\hookrightarrow ResourceQ3.enqueue(<I: ResourceQ3S, Proc\#N,$ 
             $<O: ResourceQ3.EnqueueedBL>)$ 
        ...
        | ? (TCB(Proc\#N).ResourceID = Rn)
          //Enqueue task to resource queue for Rn
           $\hookrightarrow ResourceQn.enqueue(<I: ResourceQnS, Proc\#N,$ 
             $<O: ResourceQn.EnqueueedBL>)$ 
        )
      )
    //Check and assign tasks from the low priority waiting queue
     $\hookrightarrow WaitingQL.EmptyTest(<I: WaitingQLS>; <O: WaitingQL.EmptyBL>)$ 
    ? WaitingQL.EmptyBL  $\neq T$ 
       $\rightarrow$  (  $\prod_{i=0}^{WaitingQL.EmptyBL = T} \hookrightarrow WaitingQL.Serve(<I: WaitingQLS>;$ 
         $<O: WaitingQL.ServedBL, Proc\#N>)$ 
         $\rightarrow$  ( ? (TCB(Proc\#N).ResourceID = R1)
          //Enqueue task to resource queue for R1
           $\hookrightarrow ResourceQ1.enqueue(<I: ResourceQ1S, Proc\#N,$ 
             $<O: ResourceQ1.EnqueueedBL>)$ 
          | ? (TCB(Proc\#N).ResourceID = R2)
            //Enqueue task to resource queue for R2
             $\hookrightarrow ResourceQ2.enqueue(<I: ResourceQ2S, Proc\#N,$ 
               $<O: ResourceQ2.EnqueueedBL>)$ 
          | ? (TCB(Proc\#N).ResourceID = R3)
            //Enqueue task to resource queue for R3
             $\hookrightarrow ResourceQ3.enqueue(<I: ResourceQ3S, Proc\#N,$ 
               $<O: ResourceQ3.EnqueueedBL>)$ 
          ...
          | ? (TCB(Proc\#N).ResourceID = Rn)
            //Enqueue task to resource queue for Rn
             $\hookrightarrow ResourceQn.enqueue(<I: ResourceQnS, Proc\#N,$ 
               $<O: ResourceQn.EnqueueedBL>)$ 
          )
        )
      )
    )
}

```

Fig. 5 Resource management in RTOS+

Fig. 5 Resource management in RTOS+ (cont'd)

III Implementation of the Task Scheduler and Event Handler in RTOS+

In the formal description of the implementation of RTOS+, the component logical model (CLM) [10, 11] is adopted. A CLM can be further refined into CLM schemas which are abstract models of the CLMs in the form of records that specify the configuration of a CLM by a set of fields and their types and constraints.

A. Modeling of RTOS+ Architectural Components

The schemas of CLMs contain necessary information to facilitate the description of static and dynamic behaviors of RTOS+ by a set of processes. In the implementation of task scheduling of RTOS+, we use the specified CLMs to define data structures on which the task scheduler operates. Fig. 6(a) is the top level specification of the architecture of RTOS+ and Fig. 6(b) is the corresponding detailed top level specification of the task scheduler.

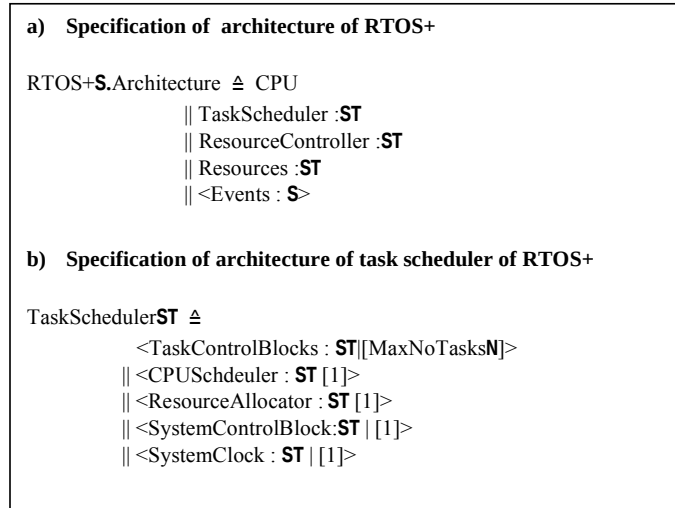


Fig. 6 Definition of the top-level architecture of RTOS+

Three system CLMs of RTOS+, the *task control block*, *resource control block* and *system control block* are defined in Fig. 7. These are fundamental control models that describe respectively information about each task, resource and the system control model of RTOS+. For example, the task control block CLM holds the following information about each task: the task ID, the task priority, task status, size, resource identifier, possible events, processor time used, delayed time, and a pointer to the task's volatile environment. The volatile environment of each task holds information about registers, stack pointer, status register, and program counter during the task's execution. Such a definition of system types facilitates the implementation of

the task scheduler. For example, the resource identifier is used by the resource manager to identify a task's resource request and to assign the resource to the task when the resource becomes available.

Using the formally specified CLMs, the task scheduler of RTOS+ is implemented by two major components: *CPU scheduler* and *resource allocator*. Fig. 8 shows the interaction between the task scheduler components. The main scheduler component called the *CPU scheduler* is responsible for switching the processor between tasks using the task scheduling algorithm discussed in Section II (B) and the context switching algorithm given in Fig. 9. Another scheduler component called the *resource allocator* is responsible for assigning resources to different tasks running in the system as well as managing system events. It also introduces new tasks into the system and releases resources attached to tasks that are completed or killed.

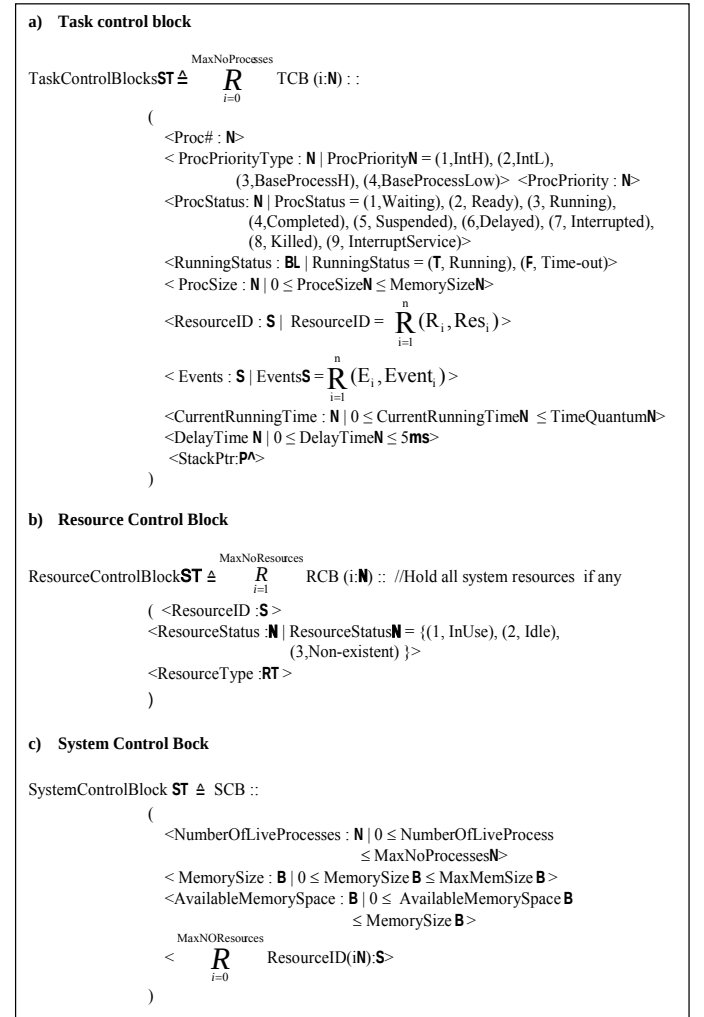


Fig. 7 Specification of CLMs in RTOS+

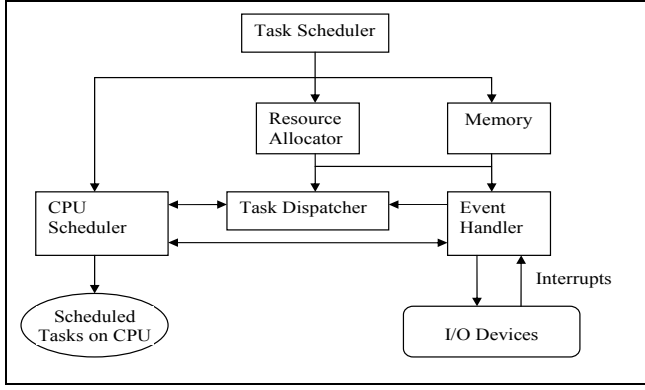


Fig. 8 Interaction between task scheduler and event handler in RTOS+

Algorithms for implementing the various components of the task scheduler are explained in the following subsections.

B. The CPU Scheduler of RTOS+

The CPU scheduler can be implemented by using the algorithm specified in Section II(B). Two queues, *ReadyQH* and *ReadyQL*, are adopted to schedule tasks with different priorities. The task scheduler first checks the status of the CPU and the priority of the current running task. If the CPU is idle or a lower priority task is currently executing on it, a new task with higher priority in the queue will be assigned after the current task is suspended. Tasks (processes) are put in the ready queue by the task dispatcher, in order to be executed by the CPU scheduler. An event handler is adopted to inform the CPU scheduler when an event has occurred that triggers new tasks and results in changes of scheduling statuses in RTOS+. The context switching algorithm in Fig. 9 is used by the CPU scheduler to determine a context switch whenever it is alerted by the event handler.

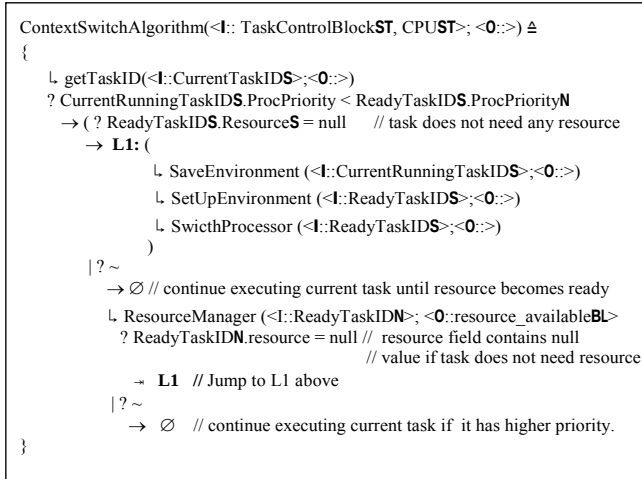


Fig. 9 The context switching algorithm of RTOS+

C. The Resource Allocator

The resource allocator is implemented using two components: the *task dispatcher* and the *event handler* as shown in Figs. 10 and 11 respectively.

The task dispatcher is responsible for dispatching various tasks to their appropriate queues based on their current statuses and requirements. Fig. 10 is an RTPA specification of the task dispatching algorithm of RTOS+. It is generally responsible for managing all task queues in the system and making sure that each task is sent to the appropriate queue as soon as an event is triggered. The task dispatcher is implemented using the task scheduling algorithm described in Section II(B) and the task dispatching algorithm specified in Fig. 10. The task dispatcher continuously checks the status of each task, and dispatches them to the appropriate queue. Only tasks dispatched to the ready queue will be served by the CPU scheduler for execution.

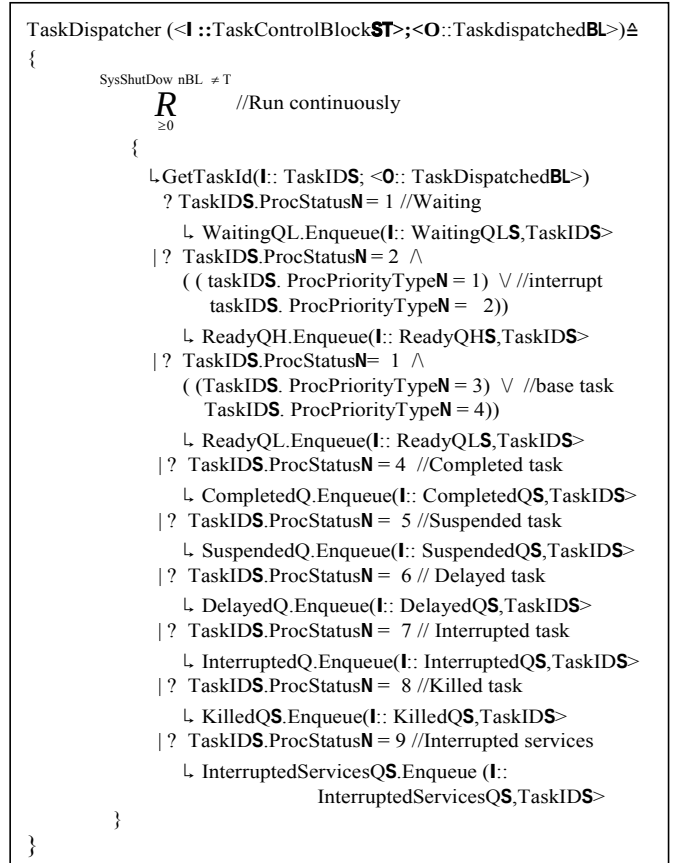


Fig. 10 The task dispatching algorithm of RTOS+

The second component of the resource allocator of RTOS+ is the event handler. A schema of event handler of RTOS+ is given in Fig. 11. It monitors events acceptable to

the system and ensures that tasks pending for certain events are put in the suspended queue. The event handler also manages interrupts, I/O events and other events related to system resources, particularly the memory. It causes appropriate actions to be taken by informing the CPU scheduler and the task dispatcher when an event has occurred. The algorithm for event handling in RTOS+ has been described in Section II(C). It works in collaboration with the task scheduler to provide solutions to the priority inversion problem. The event handler is implemented by four processes namely *EventListener*, *EventRegister*, *EventNotifier* and *InterruptHandler*.

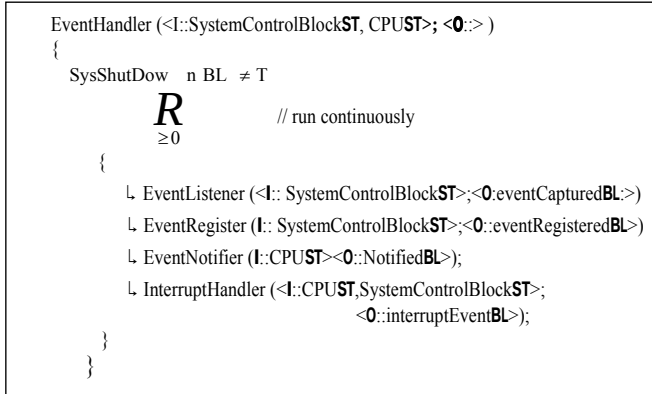


Fig. 11 The schema of the RTOS+ event handler

The *EventListener* scans for both external and internal events using two functions *HighPriorityEventScan()* and *BaseEventScan()* that scan respectively interrupt events and base events. The *EventRegister* registers incoming events into the appropriate event request queues and the *EventNotifier* notifies either the CPU Scheduler of any pending event in order to effect a suitable context switch or the interrupt handler. Once notified, the interrupt service routine (ISR) carries out an appropriate action in order to service the interrupt. The interrupt handler uses algorithm given in Fig. 12 to take care of the interrupt.

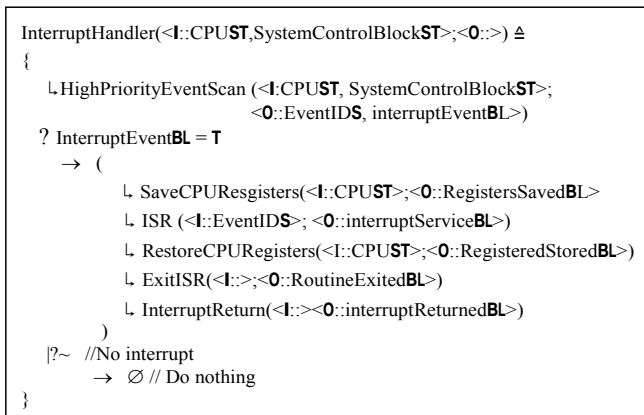


Fig.12 The algorithm of the interrupt handler

IV Conclusions

Real-time operating systems play an important role in controlling process-oriented and safety-critical applications. Their design and implementation should therefore be given great attention so that the desired features of predictability, timeliness and reliability can be delivered. A rigorous specification of an RTOS, particularly the task scheduler and its relations with other components in the kernel of a real-time operating system, is a fundamental need in RTOS architectural modeling and implementation.

This paper has reported the development of the task scheduler and the event handler of the RTOS+ real-time operating system. Techniques of RTOS+ task scheduling and event handling and their algorithms have been formally described in RTPA. Using the formal specification, the implementation of task scheduling and event handling in RTOS+ has been presented and discussed. The core components of RTOS+ have been implemented using a multi-queue based scheduling system. This technology has helped to eliminate the problem of priority inversion and to prevent multiple-blocking of tasks and deadlocks from happening.

Future work of this project is to implement a fully functional RTOS+ that integrates the task scheduler and the event handler developed in this paper into a coherent framework. RTOS+ will also be applied as part of the supporting environment for the RTPA-based code generator.

Acknowledgements

The authors would like to acknowledge the Natural Science and Engineering Council of Canada (NSERC) for its support to this work. We would like to thank the IEEE CCECE'04 organizers, Dr. Robert Hanna, Dr. Kostas Plataniotis and Dr. Alagan Anpalagan, and the anonymous referees for their comments that have improved the quality of this paper.

References

- [1] Y.C. Wang and K. Lin, "Implementing a General Real-Time Scheduling Framework in RED-Linux Real-Time Kernel," *Proc. of the 20th IEEE Real-Time Systems Symposium*, pp. 246, 1999.
- [2] D. Kalinsky, and M. Barrl, "Introduction to Priority Inversion", *Embedded Systems Programming*, CMP, LLC, <http://www.embedded.com/story/>, 2002.
- [3] J.H. Lee, and H. N. Kim, "Implementing Priority Semaphore in uC/OS Real-Time Kernel," *IEEE*

Workshop on Software Technologies for Future Embedded Systems, Japan, May 2003.

- [4] L. Sha, R. Rajkumar, and J.P. Lehoczky, "Priority Inheritance Protocols: An Approach to Real-Time Synchronization," *IEEE Transactions on Computers*, September, 1990, p. 1175.
- [5] J.B. Goodenough, and L. Sha, "*The priority ceiling protocol: A method for minimizing the blocking of high priority Ada tasks*", *ACM SIGAda Ada Letters*, Vol. VIII No.7, June 1998, p.20-31.
- [6] J. Kreuzinger, U. Brinkschulte, M. Pfeffer, S. Uhrig, and T. Ungerer. "Real-time Event-handling and Scheduling on a Multithreaded Java Microcontroller," *Microprocessors and Microsystems*, 27(1), Feb. 2003, pp. 19–31.
- [7] Y. Wang, and C. F. Ngolah, "Formal Description of a Real-Time Operating System using RTPA," *Proc. of the 2003 IEEE Canadian Conference on Electrical and Computer Engineering*, (CCECE'03), Montreal, Canada, May 2003, pp.1247-1250.
- [8] C. F. Ngolah, Y. Wang, and X. Tan, "Implementing Task Scheduling and Event Handling in RTOS+," *Proc. of the 2004 Canadian Conference on Electrical and Computer Engineering (CCECE'04)*, IEEE CS Press, Niagara Falls, ON, Canada, May 2004, pp. 1523-1526.
- [9] Y. Wang, "Operating Systems," Chap. 144, in R. Dorfed ed., *The Engineering Handbook*, 2nd ed., CRC Press, 2004.
- [10] Y. Wang, "The Real Time Process Algebra (RTPA)," *Annals of Software Engineering*, Vol. 14, Kluwer Academic Publishers, 2002, pp.235-274.
- [11] Y. Wang, "A New Approach to Real-Time Systems Specification," *Proc. of the 2002 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'02)*, Winnipeg, MB, Canada, May 2002, Vol. 2, pp. 663-668.
- [12] T. Xinming, and Y. Wang, "Specification of Abstract Data Types using Real-Time Process Algebra," *Proc. of the 2003 Canadian Conference on Electrical and Computer Engineering (CCECE'03)*, IEEE CS Press, Montreal, Canada, May 2003, pp.35.2.1-4.

班级 2016021

学号 20009100883

本科毕业设计（论文）

外文资料翻译

毕业设计题目 开源鸿蒙操作系统的高效任务调度策略

外文资料题目 RASA:A New Grid Task Scheduling Algorithm

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 魏子捷

指导教师姓名 韩高

摘要

本文考虑网格资源的分布性和可扩展性特点, 提出一种新的任务调度算法 RASA。该算法是通过对两种著名的任务调度算法 Min-min 和 Max-min 的综合研究和分析而构建的。RASA 充分利用了这两种算法的优点并避免了它们的缺点。为了实现这一点, RASA 首先估计每个可用网格资源上任务的完成时间, 然后交替应用 Max-min 和 Min-min 算法。在这方面, RASA 采用 Min-min 策略先执行小任务再执行大任务, 并采用 Max-min 策略避免大任务执行的延迟并支持大任务和小任务执行的并发。我们在网格环境中应用 RASA 来调度独立任务的实验结果证明了 RASA 在以相对较低的完工时间实现调度方面的适用性。

关键词: 网格计算、任务调度算法、完工时间、Max-Min、Min-Min

第一节 简介

网格是一个大规模的分布式系统, 涉及协调的资源共享和问题解决。网格基础设施提供了一种机制, 通过共享可能属于不同个人和机构的资源, 在自治和地理分布的节点上执行应用程序[1]。计算网格是一种网格环境, 旨在解决计算密集型问题。计算网格被定义为一种硬件和软件基础设施, 它提供对网络上现有计算资源的可靠、一致、普遍且廉价的访问[1]。

为了有效利用计算网格的巨大能力, 需要高效的任务调度算法。网格资源管理器通常应用任务调度算法来将任务最佳地分配给网格资源[2,3,4,5,6,7, 8, 9, 10]。通常, 网格用户将自己的任务提交给网格管理员, 以充分利用网格设施。计算网格中的网格管理器尝试以最小化总响应时间的方式在网格资源之间分配提交的任务。

有相对大量的任务调度算法来最小化分布式系统中任务的总完成时间[3,9,11,12,13,14]。这些算法试图通过寻找分配给任务的最合适的资源来最小化任务的总体完成时间。应该注意的是, 最小化任务的总体完成时间并不一定会导致每个单独任务的执行时间最小化。

这种算法的两个众所周知的例子是 Max-min 和 Min-min [3,9,11,12,14]。这两种算法估计每个网格资源上每个任务的执行和完成时间。Min-min 算法估计每个任务在不同资源上的执行时间, 选择完成时间最短的任务并将其分配给执行时间最短的资源。该算法对其余任务应用相同的过程。Min-min 算法的一个主要困难是将较小的任务分配给具有相对较高计算能力的资源。因此, 每当小任务的数量超过大任务时, 系统的完工时间就由大任务决定。为了解决这个困难, Max-min 算法优先考虑大任务。Max-min 算法首先将大的或者换句话说耗时的任务分配给资源, 然后分配小的任务。当小任务的数量远多于大任务时, Max-min 算法似乎比 Min-min 算法做得更好, 但在其他情况下, 大任务的早期执行可能会增加系统的总响应时间。另外, 在 Max-min 算法中, 小任务可能需要等待大任务完成。

为了避免 Min-min 和 Max-min 算法的缺点, 可以交替执行这两种算法, 将任务分配给适当的资源。这样, 在选择小任务后立即选择大任务, 反之亦然。从而实现了较

好的负载均衡，提高了电网系统的总响应时间。

本文提出了一种新的任务调度算法来解决上述 Min-min 和 Max-min 算法的问题。RASA1 算法交替应用最大-最小和最小-最小策略来将任务分配给资源。RASA 是在 GridSim 环境中实现的。GridSim 的实际实验证明了应用 RASA 在网格环境中调度独立任务的好处。

本文的其余部分组织如下：第 2 节介绍相关工作。在第 3 节中，介绍了网格环境中任务调度的概念和几种众所周知的调度算法，这些算法是许多其他工作的基准。在第 4 节中，提出了一种新的调度算法，并通过一个例子证明了该算法的突出性。第 5 节使用 GridSim 比较典型网格环境中的调度算法；并给出了比较结果。最后，第 6 节总结了本文并介绍了未来的工作。

第二节 相关工作

由于网格环境中的通信成本相对较高，大多数众所周知的调度算法不适用于大规模分布式系统，例如网格环境[3,7,8]。人们一直在尝试专门在网格环境中构建调度算法。

X. He 等人提出了一种基于传统 Min-min 算法的新算法 [3]。所提出的算法称为 QoS 引导的 Min-min，将需要高带宽的任务先于其他任务进行调度。因此，如果不同任务所需的带宽差异很大，QoS 引导的 Min-min 算法会比 Min-min 算法提供更好的结果。当所有任务的带宽要求几乎相同时，QoS 引导的 Min-min 算法的行为与 Min-min 算法类似。

E.Dong 等人提出了一种类似的算法，称为 QoS 优先级分组调度 [12]。该算法将任务的截止时间和接受率以及整个系统的完工时间作为任务调度的主要因素。与 Min-min 和 QoS 引导的 Min-min 相比，QoS 优先级分组调度算法对提交的任务取得了更好的接受率和完成时间。

E.乌拉·穆尼尔等人。提出了一种新的网格环境任务调度算法，称为 QoS Sufferage [13]。该算法考虑网络带宽，并根据任务的带宽需求来调度任务，就像 QoS 引导的 Min-min 算法一样。与 Max-min、Min-min、QoS 引导的 Min-min 和 QoS 优先级分组算法相比，QoS Sufferage 获得更小的完工时间。

K.Etminani 等人提出了一种使用 Max-min 和 Min-min 算法的新算法[14]。该算法根据每个资源上任务的预期完成时间的标准差来确定选择这两种算法中的一种。

L.穆罕默德·汗利等人在称为 Grid-JQA 的特定架构中提出了基于 QoS 的调度解决方案 [4, 5]。该调度解决方案应用聚合公式来评估 QoS，该聚合公式是参数与权重因子的组合。尽管优于 Max-min、Min-min 和 Sufferage，但 Khanli 的调度算法并不实用，而且似乎是一个不切实际的数学解决方案 [5]。

A.阿夫扎尔等人提出了一种新的网格调度算法，该算法可以最大限度地减少工作流执行成本，同时确保满足相关的 QoS 约束[15]。该算法将网格环境视为排队系统并在该系统内调度任务。该算法是面向系统的，考虑了执行成本。因此，它适合经济电网。由于该算法是非线性的，随着问题的规模变大，获得合适的调度所需的时间变得非常长且不可接受。

E.埃尔姆罗斯等人提出了一种面向用户的网格环境中任务调度算法，使用高级预留和资源选择[6]。该算法最大限度地减少各个任务的总执行时间，而不考虑所有已提交任务的总执行时间。因此，系统的整体完工时间不一定会变小。

BT 本杰明·邱等人提出了一种用于网格环境中工作负载分配的新调度算法[7]。该

算法被称为多资源调度（MRS）算法，同时考虑了站点能力和任务的资源需求。为了量化算法的性能，将算法的平均任务等待时间、队列完成时间和平均资源利用率与传统的回填和复制算法进行比较。对比结果表明该算法比其他两种算法更适用。

B.Yagoubi 等人。提供了一个模型来演示网格架构和一个算法来调度网格资源中的任务[8]。该算法试图在网格资源之间公平地分配网格环境的工作负载。尽管[8]中使用的机制和其他试图在网格资源内创建负载平衡的类似策略可以提高整个网格环境的吞吐量，但系统的总完工时间不一定会减少。

第三节 任务调度算法

假设 m 个资源 $R_j(j = 1, \dots, m)$ 必须处理 n 个任务 $T_i(i = 1, \dots, n)$ 。每项任务的调度是将一个或多个时间间隔分配给一个或多个资源[16]。资源 R_j 上任务 T_i 的预期执行时间 E_{ij} 定义为在分配 T_i 时 R_j 没有负载的情况下 R_j 执行 T_i 所花费的时间量。资源 R_j 上任务 T_i 的预期完成时间 C_{ij} 定义为 R_j 完成 T_i 的挂钟时间（在完成任何先前分配的任务之后）。令 b_i 表示任务 T_i 执行的开始。由上述定义可知， $C_{ij} = b_i + E_{ij}$ 。设 C_i 为任务 T_i 的完成时间，它等于 C_{ij} ，其中资源 R_j 被分配来执行任务 T_i 。完整进度表的完工时间定义为 $\max_{i \in T} K(i, C)$ 。Makespan 是异构计算系统（如计算网格）吞吐量的度量[9, 11]。

关于将任务分配给资源以及找到任务和资源之间的最佳匹配的决策是 NP 完全问题[3,10,11,16]。人们提出了许多调度算法，通过考虑一个或多个 QoS 参数来将任务分配给资源。这些算法根据使用环境的不同表现出不同的性能。传统的并行调度问题是将应用程序的子任务调度到并行机器上以减少周转时间。在网格环境中，调度问题是在一组计算资源上调度来自不同用户的一组任务，以最小化特定任务的完成时间或系统的完工时间。此外，还可以考虑其他参数，例如负载平衡、系统吞吐量、服务可靠性、服务成本、系统利用率等。

一般来说，调度算法分为两个基本类别：立即模式调度和批处理模式调度。在立即模式下，任务一到达调度程序就被映射到资源上。在批处理模式下，任务到达时不会映射到资源上；相反，它们被收集到一个集合中，在称为映射事件的预定时间检查映射。考虑在映射事件处进行映射的独立任务集称为元任务[14]。一些算法估计元任务中存在的任务在资源上的执行时间；然后将每个任务分配给具有该任务的最短预期执行时间的资源。采用这种机制的算法被称为最小执行时间（MET）算法[3,9,11,14]。最小完成时间（MCT）算法将每个任务分配给资源，从而得出该任务的最早完成时间。这会导致某些任务被分配给没有最小执行时间的资源 [3,9,11,14]。

机会负载平衡（OLB）算法是早期的调度算法之一，它不使用任务的执行或完成时间，并以任意顺序调度任务。OLB 背后的见解是让更多资源尽可能繁忙。OLB 的优点之一是其简单性，但由于 OLB 不考虑预期的任务执行时间，因此它找到的映射可能会导致非常差的完工时间 [11]。在 OLB 算法中，如果多个资源同时就绪，则任意选择一个资源。OLB 的复杂性取决于实现。作为一个例子，在[9]中考虑的实现中，算法的复杂度是 $O(m)$ ，其中 m 是所有资源的数量。

与 OLB 相比，Min-min 和 Max-min 算法通过考虑任务在资源上的执行时间来调度任务。Min-min 算法从所有未调度任务的集合 U 开始。然后，找到 U 中存在的每个任务的最小完成时间集合。接下来，选择未计划任务中总体完成时间最短的任务并将其分配给相应的资源（因此称为“Min-min”）。最后，新调度的任务从 U 中删除，重复

该过程直到所有任务都被调度[11]。 Min-min 算法如图 1 所示。

在图 1 中, r_j 表示资源 R_j 在完成分配给它的所有任务的执行后准备好执行任务的预期时间。首先,使用 E_{ij} (任务 T_i 在资源 R_j 上的估计执行时间)和 r_j 值计算 C_{ij} 条目。

对于每个任务 T_i , 通过扫描 C 矩阵(由 C_{ij} 值组成)的第 i 行来确定给出最早预期完成时间的资源。 确定具有最小最早预计完成时间的任务 T_k , 然后将其分配给相应的资源。

```
1.  for all tasks  $T_i$  in meta-task  $M_v$ 
2.      for all resources  $R_j$ 
3.           $C_{ij} = E_{ij} + r_j$ 
4.  do until all tasks in  $M_v$  are mapped
5.      for each task in  $M_v$  find the earliest
          completion time and the resource that
          obtains it
6.          find the task  $T_k$  with the minimum earliest
              completion time
7.          assigne task  $T_k$  to the resource  $R_l$  that gives the
              earliest completion time
8.          delete task  $T_k$  from  $M_v$ 
9.          update  $r_l$ 
10.         update  $C_{il}$  for all  $i$ 
11. end do
```

图 1 Min-Min 算法

更新矩阵 C 和向量 r , 并对尚未分配给资源的任务重复上述过程。

Max-min 算法与 Min-min 算法类似。 它与 Min-min 算法的不同之处在于, 一旦为每个任务找到提供最早完成时间的资源, 就确定最早完成时间最大的任务 T_k , 然后将其分配给相应的资源。 也就是说, 在图 1 的第 (6) 行中, “最小值” 将更改为 “最大值” [9]。

Min-min 和 Max-min 算法都考虑任务到资源的假设分配, 并根据假设的分配预测资源何时将变得空闲。 两种算法的时间复杂度都是 $O(mn^2)$, 其中 m 是系统中的资源数量, n 是应该调度执行的任务数量 [10]。

第四节 新调度算法

令 T_i 为 Min-min 映射到资源 R_j 上的第一个任务。 根据 Min-min 算法, 与其他资源相比, R_j 应该在尽可能短的时间内执行 T_i 。 只要分配给 R_j 的任务的总执行时间小于在其他资源上执行任务所花费的时间, 其余任务就会分配给最快的资源 R_j 。 如果任务的执行时间略有变化, 则此方法会导致相对较短的完工时间, 因为它尝试将任务分配给最快的资源。 但是, 如果同时存在大大小小的任务, 则较大的任务可能会被分配给较慢的资源, 并且系统的完工时间将显着增加。

在短任务数量远多于长任务的情况下, Max-min 算法似乎比 Min-min 算法做得更好。 例如, 如果只有一个长任务, 则 Max-min 算法会与长任务同时执行许多短任务。 在这种情况下, 系统的完工时间很可能由长任务的执行时间决定。 然而, 由于 Min-min 算法尝试在长任务之前分配短任务, 因此与 Max-min 相比, 完工时间会增加。 另一方面, 将最长的任务映射到最快的资源为在不同资源上并发执行小任务提供了更好的机会。 在这种特定情况下, Max-min 提供了更好的映射, 比 Min-min [11] 更能支持网格

资源之间的负载平衡。 尽管小规模分布式系统中的负载平衡是可取的，并且可以减少总完成时间，但是，在大规模分布式系统中，负载平衡并不一定会导致最短的完工时间。该算法在大规模系统中优于 Max-min，因为它专注于最小化任务的完成时间而不是负载平衡。

在第 4.1 小节中，提出了新算法，在第 4.2 小节中，提出了一个说明性示例，以将所提出的算法与现有算法进行比较。

4.1 RASA

我们提出的网格调度算法 RASA 如图 2 所示。该算法构建一个矩阵 C ，其中 C_{ij} 表示任务 T_i 在资源 R_j 上的完成时间。 如果可用资源数量为奇数，则应用 Min-min 策略来分配第一个任务，否则应用 Max-min 策略。 其余任务将通过两种策略之一分配给适当的资源。 例如，如果第一个任务按“最小-最小”策略分配给资源，则下一个任务将按“最大-最小”策略分配。 在下一轮中，任务分配以与上一轮不同的策略开始。 例如，如果第一轮以最大-最小策略开始，第二轮将以最小-最小策略开始。

实验结果表明，如果可用资源数量为奇数，则在第一轮中优选首先应用 Min-min 策略，否则最好首先应用 max-min 策略。 Min-min 和 Max-min 策略的交替交换，导致小任务和大任务在不同的资源上连续执行，从而得到 Max-min 算法中小任务的等待时间和大任务的等待时间 在 Min-min 算法中被忽略。 由于 RASA 由 Max-Min 和 Min-Min 算法组成，并且没有耗时的指令，因此 RASA 的时间复杂度为 $O(mn^2)$ ，其中 m 是资源数量， n 是任务数量（类似于 Max-Min 和 Min-min 算法）。

```

1. for all tasks  $T_i$  in meta-task  $M_v$ 
2.   for all resources  $R_j$ 
3.      $C_{ij} = E_i + r_j$ 
4. do until all tasks in  $M_v$  are mapped
5.   if the number of resources is even then
6.     for each task in  $M_v$  find the earliest
completion
           time and the resource that obtains it
7.     find the task  $T_k$  with the maximum earliest
           completion time
8.     assigne task  $T_k$  to the resource  $R_l$  that gives
the
           earliest completion time
9.     delete task  $T_k$  from  $M_v$ 
10.    update  $r_l$ 
11.    update  $C_{il}$  for all  $i$ 
12.   else
13.     for each task in  $M_v$  find the earliest
completion
           time and the resource that obtains it
14.     find the task  $T_k$  with the minimum earliest
           completion time
15.     assigne task  $T_k$  to the resource  $R_l$  that gives
the
           earliest completion time
16.    delete task  $T_k$  from  $M$ 

```

图 2 提出的算法（RASA）

4.2 一个说明性的例子

作为一个简单的示例，假设有一个具有两个资源的网格环境。 资源的处理速度以及将每个资源连接到网格管理器的通信链路的带宽如表 1 所示。四个任务 T1、T2、T3 和 T4 在元任务 M_v 中，并且网格 manager 应该将 M_v 内的所有任务调度到两个资源 R1 和 R2 上。 表 2 表示任务 T1 至 T4 中的指令和数据量。

Table 1- Specification of the resources

<i>Resources</i>	<i>Processing speed (MIPS)</i>	<i>Related Bandwidth (Mbps)</i>
R_1	50	100
R_2	100	5

Table 2- Specification of the tasks

<i>Tasks</i>	<i>Volume of instructions (MI)</i>	<i>Volume of data (Mb)</i>
T_1	128	44
T_2	69	62
T_3	218	64
T_4	21	59

应用表 1 和表 2 中提供的数据,可以计算每个资源上的任务的预期完成时间。计算出的任务完成时间如表 3 所示。

Table 3- Completion time of the tasks on each of the resources

<i>Tasks / Resources</i>	R_1	R_2
T_1	3	10
T_2	2	13
T_3	5	15
T_4	1	12

图 3 包括两个甘特图,表示在元任务 Mv 上应用 Max-min 和 Min-min 算法的结果,如表 1 和表 2 中所述。尽管 Max-min 和 Min-min 算法中安排的任务顺序不同的是,应用每种算法时系统的完工时间是相等的。

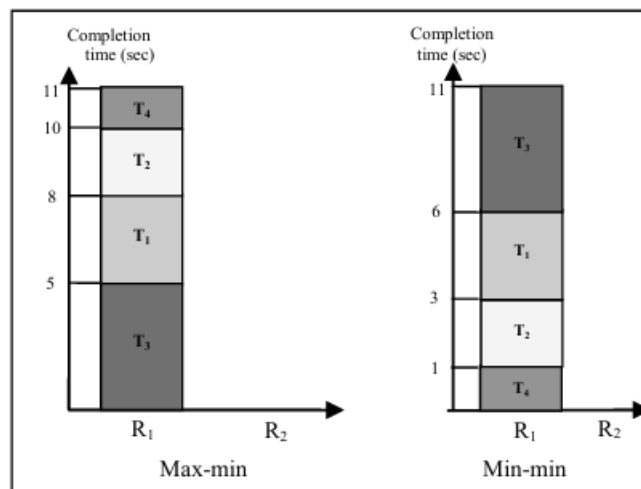


图 3 Max-Min 和 Min-Min 算法的甘特图

在元任务 Mv 上应用 RASA 的结果如图 4 所示。如图 4 所示,应用 RASA 时

系统的 makespan 为 10 秒，而 Max-min 和 Min-min 提供了 makespan 为 11 秒。

此外，考虑到图 3 和图 4 中的完成时间，可以观察到 RASA 通过提供相对较小的完工时间和更高的负载平衡，优于 Max-min 和 Min-min 算法。假设任务可以在任何资源上独立执行。因此，QoS 引导的 Min-min 和 QoS 优先级分组算法的作用与 Min-min 算法相同。上面的例子只是一个例子，它显示了 RASA 相对于其他算法的优势。然而，可以提供例子，其中 RASA 的结果与上述其他算法相同甚至更差。一般来说，当提交的任务大小几乎相同时，Min-min 或 Max-min 算法可能比 RASA 表现得更好。

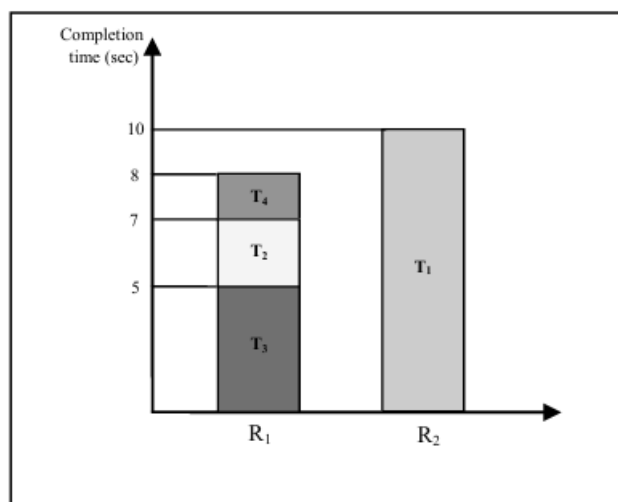


图 4 RASA 的甘特图

第五节 仿真和实验结果

为了评估和比较 RASA 与其他算法（例如 Max-min、Min-min、OLB、QoS 引导的 Min-min 和 QoS 优先级分组），使用了称为 GridSim 工具包 [17] 的模拟环境。为了证明 RASA 与其他算法相比的卓越性，做出了两个不同的假设：

1. 假设一：任务的计算时间胜过任务的通信时间。这种情况发生在多处理器和小规模分布式系统（例如集群环境）中。

2. 假设二：通信时间变得更多，甚至可以克服任务的计算时间。这种情况发生在大规模分布式系统中，例如网格环境，其中资源广泛分布并通过通信链路连接。

上述算法均在 GridSim 环境下进行了仿真。假设对不同资源上执行任务没有限制，并且每个任务可以在每个资源上执行。在这种情况下，QoS 引导的 Min-min 和 QoS 优先级分组算法的执行与 Min-min 算法相同。因此，在下图中呈现的所有图中，仅描绘了 Min-min 的结果，并且消除了与 QoS 引导的 Min-min 和 QoS 优先级分组算法相关的图。为了研究工作负载量对 RASA 效率的影响并与其他算法进行比较，在上述每个假设中，都考虑了轻负载、中负载和重负载三种不同的工作负载。在轻负载下，在 10 到 11 个网格资源内调度 200 个任务，并估计每个算法的完工时间。此外，在中等负载下，考虑了 1000 个任务，在重负载下，考虑了 5000 个任务。图 5 和图 6 显示了通过应用算法实现的完工时间，分别考虑假设 I 和假设 II。

如图 5-(a)所示，当任务的计算时间超过其通信时间并且资源的工作负载较轻时，Max-min 返回的完工时间比其他算法相对较小。随着资源工作负载的增加，Min-min 和

RASA 实现的完工时间会变小。如图 5-(b) 和图 5-(c) 所示, RASA 返回的完工时间小于其他算法返回的完工时间。考虑图 5-(b) 和 5-(c), 可以得出结论: 在小型分布式系统中, 与 Max-min、Min-min、OLB、QoS 引导的 Min-min 和 QoS 优先级分组算法相比, RASA 返回更小的完工时间。

图 6 显示了基于假设 II 的算法的完成时间。如图 6-(a)所示, 当工作负载较轻时, RASA 返回的完工时间小于其他算法。比较图 5-(a) 和 6-(a), 可以看出: Max-min 适合小型分布式系统, 而 Min-min 则适合大规模分布式系统。然而, 在小规模和大规模分布式系统中, RASA 返回的完工时间都比 Min-min 和 Max-min 算法小。当资源工作量较大时, 与其他算法相比, RASA 的完成时间更短。因此, RASA 在轻负载和重负载条件下都能实现更小的完工时间。

为了清楚起见, 在图 5 和图 6 中, 不同算法返回的完工时间被重新调整。此处, OLB 返回的最大完工时间为 100, 其他完工时间将根据此数量重新调整。图 5 和图 6 中观察到的六种不同情况被视为正六边形的顶点。六边形的顶点被命名为 Light 1、Light 2、Medium 1、Medium 2、Heavy 1 和 Heavy 2, 这意味着假设 I 为轻负载, 假设 II 为轻负载, 假设 I 为中负载, 假设 II 为中负载, 假设 I 中的重负载和假设 II 中的重负载。该六边形如图 7 所示。在该六边形中, 假设网格资源的数量为 10。图 8 中网格资源的数量为 11。如图 7 和图 8 所示, 非正六边形的面积与 RASA 相关, 比其他算法小。因此, 可以得出结论: 在几乎所有不同的条件下, RASA 返回的完工时间都小于其他算法。

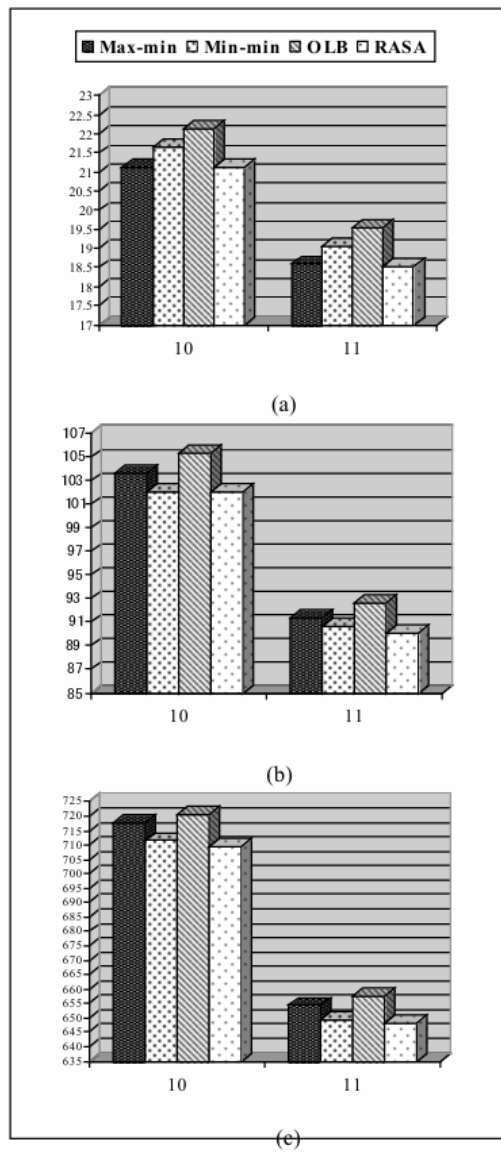


图 5 假设 1 中算法的完成时间

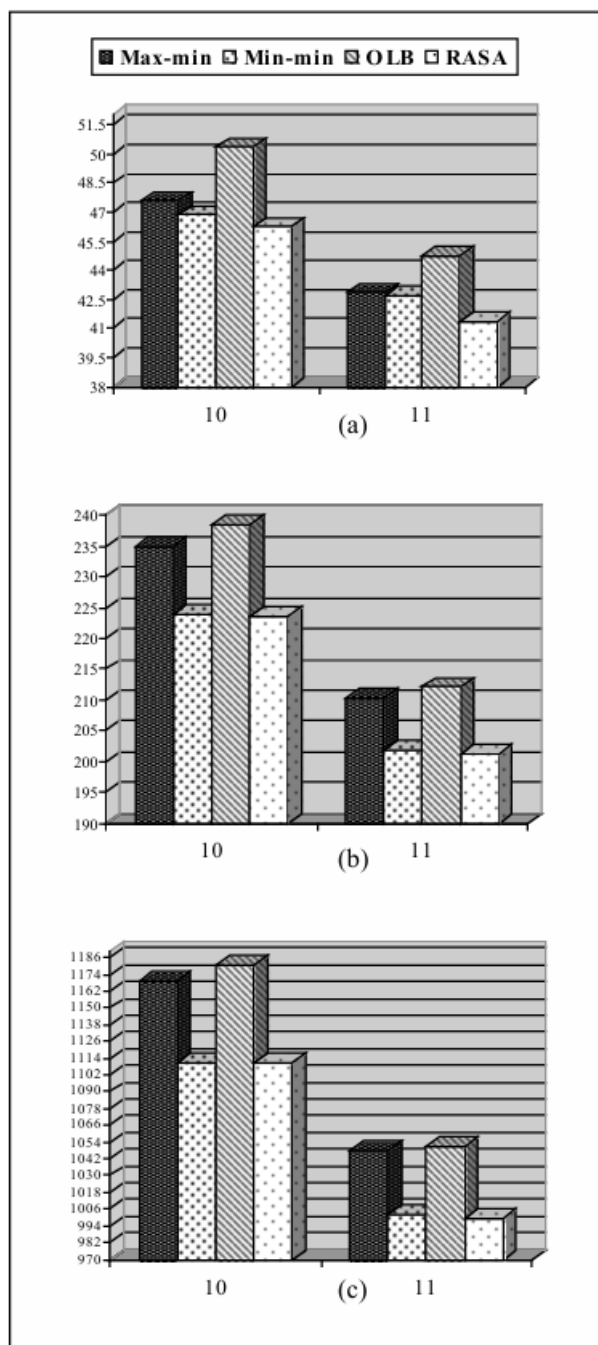


图 6 假设 2 中算法的完成时间

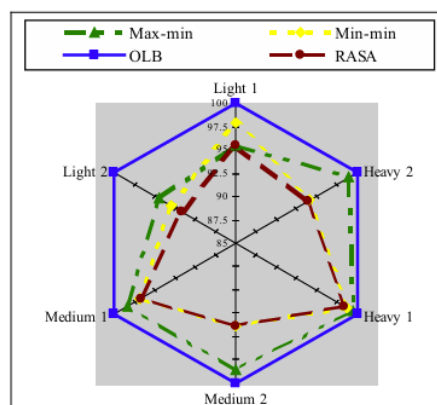


图 7 资源数为 10 时的算法完成时间

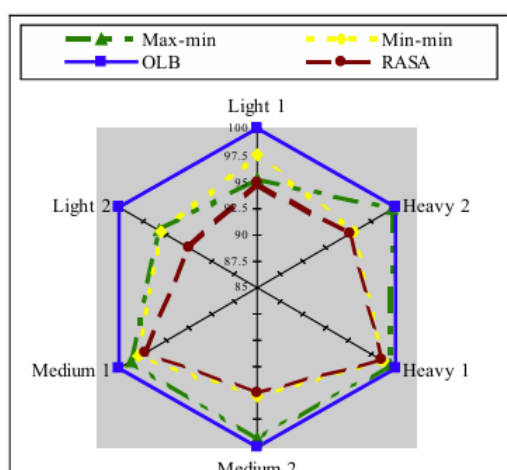


图 8 资源数为 11 时算法的完成时间

第六节 结论和未来工作

Min-min 和 Max-min 算法适用于小规模分布式系统。当元任务中小任务的数量多于大任务的数量时，Min-min 算法无法合理调度任务，系统的完工时间会变得相对较大。与 Min-min 算法不同，Max-min 算法尝试通过先调度大任务再调度小任务来实现资源内的负载平衡。然而，在计算网格环境中，高吞吐量比负载平衡更令人感兴趣。为了实现这一目标，本文提出了一种新的任务调度算法 RASA。

RASA 由两种传统的调度算法组成；Max-Min 和 Min-Min。RASA 利用了 Max-min 和 Min-min 算法的优点并弥补了它们的缺点。在 GridSim 模拟器中应用 RASA 获得的实验结果表明，RASA 在大规模分布式系统中优于现有的调度算法。

本研究仅关注资源的奇数或偶数个数，并分析了两种著名的传统算法 Max-min 和 Min-min 的优缺点。本文没有考虑每个任务的截止时间、任务的到达率、每个资源上任务执行的成本、通信的成本以及许多其他可以成为研究主题的情况。此外，将所提出的算法应用于实际网格环境进行实际评估可能是该领域的其他开放问题。

参考文献

- [1]I. Foster 和 C. Kesselman, 网络 2: 新计算基础设施的蓝图, 第二版, Elsevier 和 Morgan Kaufmann Press, 2004 年。
- [2]L. Chunlin 和 L. Layuan, “计算网格中计算经济的基于 QoS 的资源调度”, 信息处理快报杂志, 卷 98, 第 119-126 页, 2006 年。
- [3]X. He、X-He Sun 和 G. V. Laszewski, “QoS 引导的最小最小启发式网格任务调度”, 计算机科学与技术杂志, 卷 18, 第 442-451 页, 2003 年。
- [4]L. Mohammad Khanli 和 M. Anoloui, “通过 Grid-JQA 在桌面网格中进行资源调度”, 第三届网格和普适计算国际会议, IEEE, 2008 年。
- [5]L. Mohammad Khanli 和 M. Anoloui, “Grid_JQA: 网格计算的 QoS 引导调度算法”,

第六届并行和分布式计算国际研讨会 (ISPDC' 07), IEEE, 2007 年。

[6]E. Elmroth 和 J. Tordsson, “网格资源代理算法支持基于性能预测的提前预留和资源选择”, 《未来一代计算机系统杂志》, 卷 24, 第 585-593 页, 2008 年。

[7]BT Benjamin Khoo、B. Veeravalli、T. Hung 和 C.W. Simon See, “网格计算环境中的多维调度方案”, 并行与分布式计算杂志, 卷 67, 2007 年。

[8]B. Yagoubi 和 Y. Slimani, “网格计算的任务负载平衡策略”, 计算机科学杂志, 卷 3, 第 3 期, 第 186-194 页, 2007 年。

[9]M. Maheswaran, Sh.Ali、H. Jay Siegel、D. Hensgen 和 R. F. Freund, “一类独立任务到异构计算系统的动态映射”, 《并行与分布式计算杂志》, 第 59 卷, 第 107-131 页, 1999 年。

[10]R. F. Freund、M. Gherrity、S. Ambrosius、M. Campbell、M. Halderman、D. Hensgen、E. Keith、T. Kidd、M. Kussow、J. D. Lima、F. Mirabile、L. Moore、B. Rust 和 H. J. Siegel, “利用 SmartNet 在多用户异构计算环境中调度资源”, 第七届异构计算研讨会论文集, 1998 年。

[11]T. D. Braun、H. Jay Siegel、N. Beck、L. L. Boloni、M. Maheswaran、A. I. Reuther、J. P. Robertson、M. D. Theys 和 B. Yao, “将一类独立任务映射到异构任务的十一种静态启发式比较分布式计算系统, 并行和分布式计算杂志, 卷 61, 第 810-837 页, 2001 年。

[12]F. Dong, J. Luo, L. Gau, L. Ge, “一种基于 QoS 优先级分组的网格任务调度算法”, 载于第五届网格与协同计算国际会议论文集 (GCC'06)), IEEE, 2006。

[13]E. Ullah Munir、J. Li 和 Sh. Shi, 2007。网格中独立任务调度的 QoS 覆盖启发式。信息技术杂志, 6 (8): 1166-1170。

[14]K. Etminani 和 M. Naghibzadeh, “网格任务调度的最小-最小最大-最小选择性算法”, 第三届 IEEE/IFIP 国际互联网会议, 乌兹别克斯坦, 2007 年。

[15]A. Afzal、A. Stephen McGough 和 J. Darlington, “网格计算环境中的容量规划和调度”, 《未来一代计算机系统杂志》, 卷 24, 第 404-414 页, 2008 年。

[16]P. Brucker, 调度算法, 第五版, Springer Press, 2007 年。

[17]R. Buyya 和 M. Murshed, “GridSim: 网格计算分布式资源管理和调度的建模和模拟工具包”, 并发与计算实践与经验杂志, 第 1175-1220 页, 2002 年。

RASA: A New Grid Task Scheduling Algorithm

Saeed Parsa, Reza Entezari-Maleki**Corresponding author*

*Parallel Processing and Concurrent Systems Laboratory, Department of Computer Engineering,
Iran University of Science and Technology (IUST), Tehran, Iran*

parsa@iust.ac.ir, r_entezari@comp.iust.ac.ir

doi: 10.4156/jdcta.vol3.issue4.10

Abstract

In this paper, a new task scheduling algorithm called RASA, considering the distribution and scalability characteristics of grid resources, is proposed. The algorithm is built through a comprehensive study and analysis of two well known task scheduling algorithms, Min-min and Max-min. RASA takes advantages of the both algorithms and avoids their drawbacks. To achieve this, RASA firstly estimates the completion time of the tasks on each of the available grid resources, and then applies the Max-min and Min-min algorithms, alternatively. In this respect, RASA uses the Min-min strategy to execute small tasks before the large ones, and applies the Max-min strategy to avoid delays in the execution of the large tasks and to support concurrency in the execution of the large and small tasks. Our experimental results of applying RASA on scheduling independent tasks within grid environments demonstrate the applicability of RASA in achieving schedules with comparatively lower makespan.

Keywords

Grid computing, task scheduling algorithm, makespan, Max-min, Min-min

1. Introduction

Grid is a large scale distributed system, concerned with coordinated resource sharing and problem solving. The grid infrastructure provides a mechanism to execute applications over autonomous and geographically distributed nodes by sharing resources which may belong to different individuals and institutions [1]. Computational grid is a kind of grid environments, targeted at solving computationally intensive problems. Computational grid is defined as a hardware and software infrastructure which provides dependable, consistent, pervasive and inexpensive access to computational resources existing on the network [1].

To make effective use of the tremendous capabilities of the computational grids, efficient task scheduling algorithms are required. Task scheduling algorithms are commonly applied by grid resource managers to optimally dispatch tasks to grid resources [2, 3, 4, 5, 6, 7, 8, 9, 10]. Typically, grid users submit their own tasks to the grid manager to take full advantage of the grid facilities. The grid manager in a computational grid tries to distribute the submitted tasks amongst the grid resources in such a way that the total response time is minimized.

There are relatively a large number of task scheduling algorithms to minimize the total completion time of the tasks in distributed systems [3, 9, 11, 12, 13, 14]. These algorithms try to minimize the overall completion time of the tasks by finding the most suitable resources to be allocated to the tasks. It should be noticed that minimizing the overall completion time of the tasks does not necessarily result in the minimization of execution time of each individual task.

Two well known examples of such algorithms are Max-min and Min-min [3, 9, 11, 12, 14]. These two algorithms estimate the execution and completion times of each of the tasks on each of the grid resources. Estimating the execution time of each task on different resources, the Min-min algorithm selects the task with minimum completion time and assigns it to the resource on which the minimum execution time is achieved. The algorithm applies a same procedure to the remaining tasks. A major difficulty with the Min-min algorithm is to assign the smaller tasks to the resources with relatively higher computational power. As a result, the makespan of the system is determined by the large tasks whenever the number of the small tasks exceeds the large ones. To resolve the difficulty, the Max-min algorithm gives priority to the large tasks. The Max-min algorithm firstly assigns the large or in other words time consuming tasks to the resources and then assigns the small ones. The Max-min algorithm seems to do better than the Min-min whenever the number of small tasks is much more than the large ones, but in the other cases, early execution of the large tasks might increase the total response time of the system. Also, in the Max-min algorithm, the small

tasks may be required to wait for the completion of the large ones.

To avoid the drawbacks of the Min-min and Max-min algorithms, these two algorithms could be executed alternatively, to assign tasks to appropriate resources. In this way, a large task is selected immediately after a small one and vice versa. Thereby, a comparatively better load balancing is achieved and the total response time of the grid system is improved.

This paper offers a new task scheduling algorithm to resolve the above mentioned problems with the Min-min and Max-min algorithms. The algorithm, RASA¹, applies the Max-min and Min-min strategies alternatively to assign tasks to the resources. RASA is implemented in GridSim environment. Practical experiments with GridSim demonstrate the benefits of applying RASA to schedule independent tasks within grid environments.

The remaining parts of this paper are organized as follows: Section 2 presents the related works. In Section 3, the concept of task scheduling in grid environments and several well known scheduling algorithms which are benchmarks of many other works, are introduced. In section 4, a new scheduling algorithm is proposed and the prominence of the algorithm is demonstrated through an example. Section 5 compares the scheduling algorithms in a typical grid environment using GridSim; and presents the results of the comparison. Finally, section 6 concludes the paper and presents future works.

2. Related Works

Due to relatively high communication costs in grid environments most of the well known scheduling algorithms are not applicable in large scale distributed systems such as grid environments [3, 7, 8]. There has been an ongoing attempt to build scheduling algorithms specifically within grid environments.

X. He et al. have presented a new algorithm based on the conventional Min-min algorithm [3]. The proposed algorithm which is called QoS guided Min-min, schedules tasks requiring high bandwidth before the others. Therefore, if the bandwidth required by different tasks varies highly, the QoS guided Min-min algorithm provides better results than the Min-min algorithm. Whenever the bandwidth requirement of all of the tasks is almost the same, the QoS guided Min-min algorithm acts similar to the Min-min algorithm.

F. Dong et al. have proposed a similar algorithm called QoS priority grouping scheduling [12]. This algorithm, considers deadline and acceptance rate of

the tasks and the makespan of the whole system as major factors for task scheduling. In comparison with Min-min and QoS guided Min-min, the QoS priority grouping scheduling algorithm achieves better acceptance rate and completion time for the submitted tasks.

E. Ullah Munir et al. have presented a new task scheduling algorithm for grid environments called QoS Sufferage [13]. This algorithm considers network bandwidth and schedules tasks based on their bandwidth requirement as the QoS guided Min-min algorithm does. Compared with the Max-min, Min-min, QoS guided Min-min and QoS priority grouping algorithms, QoS Sufferage obtains smaller makespans.

K. Etminani et al. have proposed a new algorithm which uses Max-min and Min-min algorithms [14]. The algorithm determines to select one of these two algorithms, dependent on the standard deviation of the expected completion times of the tasks on each of the resources.

L. Mohammad Khanli et al. have presented QoS based scheduling solutions in a specific architecture called Grid-JQA [4, 5]. This scheduling solution applies an aggregation formula that is a combination of parameters together with weighting factors to evaluate QoS. Despite outperforming the Max-min, Min-min, and Sufferage, the Khanli's scheduling algorithm is not practical and seems to be an impractical mathematical solution [5].

A. Afzal et al. have proposed a new grid scheduling algorithm that minimizes the cost of the execution of workflows while ensuring that their associated QoS constraints are satisfied [15]. The algorithm views a grid environment as a queuing system and schedules tasks within this system. This algorithm is system oriented and considers the execution cost. Hence, it is suitable for economic grids. Since the algorithm is non-linear, as the size of the problem is gets large the time it takes to obtain a suitable scheduling becomes very long and unacceptable.

E. Elmroth et al. have proposed a user oriented algorithm for task scheduling in grid environments, using advanced reservation and resource selection [6]. The algorithm minimizes the total execution time of the individual tasks without considering the total execution time of all of the submitted tasks. Therefore, the overall makespan of the system does not necessarily get small.

B.T. Benjamin Khoo et al. have presented a new scheduling algorithm for workload distribution in grid environments [7]. This algorithm which is known as multiple resources scheduling (MRS) algorithm, takes into account both the site capabilities and the resource requirements of tasks. To quantify the performance of

¹ Resource Aware Scheduling Algorithm

the algorithm, the average task wait times, queue completion times, and average resource utilization factor, of the algorithm are compared to the conventional backfill and replication algorithms. The comparison results indicate that this algorithm is more applicable than the other two algorithms.

B. Yagoubi et al. have offered a model to demonstrate grid architecture and an algorithm to schedule tasks within grid resources [8]. The algorithm tries to distribute the workload of the grid environment amongst the grid resources, fairly. Although, the mechanism used in [8] and other similar strategies which try to create load balancing within grid resources can improve the throughput of the whole grid environment, the total makespan of the system does not decrease, necessarily.

3. Task Scheduling Algorithms

Suppose that m resources $R_j (j = 1, \dots, m)$ have to process n tasks $T_i (i = 1, \dots, n)$. A schedule for each task is an allocation of one or more time intervals to one or more resources [16]. The expected execution time E_{ij} of task T_i on resource R_j is defined as the amount of time taken by R_j to execute T_i given R_j has no load when T_i is assigned. The expected completion time C_{ij} of task T_i on resource R_j is defined as the wall-clock time at which R_j completes T_i (after having finished any previously assigned tasks). Let b_i denote to the beginning of the execution of task T_i . From the above definitions, $C_{ij} = b_i + E_{ij}$. Let C_i be the completion time for task T_i , and it is equal to C_{ij} where resource R_j is assigned to execute task T_i . The makespan for the complete schedule is then defined as $\max_{T_i \in K} (C_i)$. Makespan is a measure of the throughput of the heterogeneous computing system (like computational grid) [9, 11].

Decision about the assigning of tasks to the resources and Finding the best match between tasks and resources is NP-complete problem [3, 10, 11, 16]. Lots of scheduling algorithms are proposed to assign the tasks to the resources by considering one or several QoS parameters. These algorithms show different performances based on the environment in which they are used. The traditional parallel scheduling problem is to schedule the subtasks of an application on the parallel machines in order to reduce the turnaround time. In a grid environment, the scheduling problem is to schedule a set of tasks from different users on a set of computing resources to minimize the completion time of a specific task or the makespan of a system. Also, other parameters such as load balancing, system throughput, service reliability, service cost, system utilization, and so forth can be considered.

Generally, the scheduling algorithms are divided into two basic categories; immediate mode scheduling and batch mode scheduling. In the immediate mode, a task is mapped onto a resource as soon as it arrives at the scheduler. In the batch mode, tasks are not mapped onto the resources as they arrive; instead they are collected into a set that is examined for mapping at prescheduled times called mapping events. The independent set of tasks which is considered for mapping at the mapping events is called a meta-task [14]. Some algorithms estimate the execution time of the tasks existing in the meta-task on the resources; then assign each task to the resource with the minimum expected execution time for that task. The algorithms with this mechanism are named as minimum execution time (MET) algorithms [3, 9, 11, 14]. The minimum completion time (MCT) algorithms assign each task to the resource which results in that task's earliest completion time. This causes some tasks to be assigned to the resources that do not have the minimum execution time for them [3, 9, 11, 14].

One of the earlier scheduling algorithms which do not use the execution or completion time of the tasks, and schedules the tasks in the arbitrary order is opportunistic load balancing (OLB) algorithm. The insight behind OLB is to keep all resources as busy as possible. One advantage of OLB is its simplicity, but because OLB does not consider expected task execution times, the mappings it finds can result in very poor makespans [11]. In OLB algorithm, if multiple resources become ready at the same time, then one resource is arbitrarily chosen. The complexity of the OLB is dependent on the implementation. As an example, in the implementation considered in [9], the complexity of the algorithm is $O(m)$, where m is the number of all of the resources.

In contrast with OLB, Min-min and Max-min algorithms schedule tasks by considering the execution time of the tasks on the resources. The Min-min algorithm begins with the set U of all unscheduled tasks. Then, the set of minimum completion times for each of the tasks existing in U is found. Next, the task with the overall minimum completion time from unscheduled tasks is selected and assigned to the corresponding resource (hence the name Min-min). Last, the newly scheduled task is removed from U , and the process repeats until all tasks are scheduled [11]. The Min-min algorithm is shown in Figure 1.

In Figure 1, r_j denotes the expected time which resource R_j will become ready to execute a task after finishing the execution of all tasks assigned to it. First, the C_{ij} entries are computed using the E_{ij} (the estimated execution time of task T_i on resource R_j) and r_j values.

For each task T_i , the resource that gives the earliest expected completion time is determined by scanning

the i th row of the C matrix (composed of the C_{ij} values). The task T_k that has the minimum earliest expected completion time is determined and then assigned to the corresponding resource.

```

1. for all tasks  $T_i$  in meta-task  $M_v$ 
2.   for all resources  $R_j$ 
3.      $C_{ij} = E_{ij} + r_j$ 
4.   do until all tasks in  $M_v$  are mapped
5.     for each task in  $M_v$  find the earliest
       completion time and the resource that
       obtains it
6.     find the task  $T_k$  with the minimum earliest
       completion time
7.     assign task  $T_k$  to the resource  $R_l$  that gives the
       earliest completion time
8.     delete task  $T_k$  from  $M_v$ 
9.     update  $r_l$ 
10.    update  $C_{il}$  for all  $i$ 
11. end do

```

Figure 1- The Min-min algorithm

The matrix C and vector r are updated, and the above process is repeated for tasks that have not yet been assigned to a resource.

The Max-min algorithm is similar to the Min-min algorithm. It differs from the Min-min algorithm in that once the resource that provides the earliest completion time is found for every task, the task T_k that has the maximum earliest completion time is determined and then assigned to the corresponding resource. That is, in line (6) of Figure 1, “*minimum*” would be changed to “*maximum*” [9].

Both the Min-min and Max-min algorithms consider a hypothetical assignment of tasks to resources, projecting when a resource will become idle based on the hypothetical assignment. Both algorithms have time complexities of $O(mn^2)$, where m is the number of resources in the system and n is the number of tasks which should be scheduled to be executed [10].

4. The New Scheduling Algorithm

Let T_i be the first task mapped by Min-min onto the resource, R_j . According to the Min-min algorithm, R_j should execute T_i in the shortest possible time, compared with the other resources. The remaining tasks are assigned to the fastest resource, R_j , as long as the total execution times of the task assigned to the R_j is less than the time it takes to execute the tasks on the other resources. This approach results in a comparatively shorter makespan if the execution time of the tasks varies slightly because it attempts to assign the tasks to the fastest resources. However, if there are large and small tasks, the large ones may be assigned

to slower resources and the makespan of the system will be increased, dramatically.

The Max-min algorithm seems to do better than the Min-min algorithm in the cases when the number of short tasks is much more than the long ones. For example, if there is only one long task, the Max-min algorithm executes many short tasks concurrently with the long task. In this case, the makespan of the system is most likely determined by the execution time of the long task. However, since the Min-min algorithm attempts to assign the short tasks before the long one, the makespan increases compared with the Max-min. On the other hand, mapping the longest task to the fastest resource provides a better opportunity for concurrent execution of the small tasks on different resources. In this certain situation, the Max-min provides a better mapping which supports load balancing across the grid resources more than the Min-min [11]. Although load balancing in small scale distributed systems is desirable and leads to reduced total completion times however, in large scale distributed systems load balancing does not necessarily results in the shortest makespan. The proposed algorithm outperforms Max-min in large scale systems, because it focuses on minimizing the completion time of tasks rather than load balancing.

In subsection 4.1, the new algorithm is proposed and in subsection 4.2, an illustrative example is presented to compare the proposed algorithm with existing algorithms.

4.1 RASA

Our proposed grid scheduling algorithm, RASA, is presented in Figure 2. The algorithm builds a matrix C where C_{ij} represents the completion time of the task T_i on the resource R_j . If the number of available resources is odd, the Min-min strategy is applied to assign the first task, otherwise the Max-min strategy is applied. The remaining tasks are assigned to their appropriate resources by one of the two strategies, alternatively. For instance, if the first task is assigned to a resource by the Min-min strategy, the next task will be assigned by the Max-min strategy. In the next round the task assignment begins with a strategy different from the last round. For instance if the first round begins with the Max-min strategy, the second round will begin with the Min-min strategy.

Experimental results show that if the number of available resources is odd it is preferred to apply the Min-min strategy the first in the first round otherwise is better to apply the max-min strategy the first. Alternative exchange of the Min-min and Max-min strategies results in consecutive execution of a small and a large task on different resources, and hereby, the

waiting time of the small tasks in Max-min algorithm and the waiting time of the large tasks in Min-min algorithm are ignored. As RASA is consist of the Max-Min and Min-Min algorithms and have no time consuming instruction, the time complexity of RASA is $O(mn^2)$ where m is the number of resources and n is the number of tasks (similar to Max-min and Min-min algorithms).

```

1. for all tasks  $T_i$  in meta-task  $M_v$ 
2.   for all resources  $R_j$ 
3.      $C_{ij} = E_j + r_j$ 
4. do until all tasks in  $M_v$  are mapped
5.   if the number of resources is even then
6.     for each task in  $M_v$  find the earliest
completion
           time and the resource that obtains it
7.     find the task  $T_k$  with the maximum earliest
           completion time
8.     assigne task  $T_k$  to the resource  $R_l$  that gives
the
           earliest completion time
9.     delete task  $T_k$  from  $M_v$ 
10.    update  $r_l$ 
11.    update  $C_{il}$  for all  $i$ 
12.   else
13.     for each task in  $M_v$  find the earliest
completion
           time and the resource that obtains it
14.     find the task  $T_k$  with the minimum earliest
           completion time
15.     assigne task  $T_k$  to the resource  $R_l$  that gives
the
           earliest completion time
16.   delete task  $T_k$  from  $M$ 

```

Figure 2- The proposed algorithm (RASA)

4.2 An Illustrative Example

As a simple example, assume there is a grid environment with two resources. The processing speed of the resources and the bandwidth of the communication links which connect each of the resources to the grid manager are shown in Table 1. Four tasks T_1 , T_2 , T_3 , and T_4 are in the meta-task M_v , and the grid manager is supposed to schedule all the tasks within M_v on two resources R_1 and R_2 . Table 2 represents the volume of instructions and data in the tasks T_1 to T_4 .

Table 1- Specification of the resources

Resources	Processing speed (MIPS)	Related Bandwidth (Mbps)
R_1	50	100
R_2	100	5

Table 2- Specification of the tasks

Tasks	Volume of	Volume of
-------	-----------	-----------

	instructions (MI)	data (Mb)
T_1	128	44
T_2	69	62
T_3	218	64
T_4	21	59

Applying the data presented in Table 1 and Table 2, it is possible to calculate the expected completion time of the tasks on each of the resources. The calculated completion time of the tasks are demonstrated in Table 3.

Table 3- Completion time of the tasks on each of the resources

Tasks / Resources	R_1	R_2
T_1	3	10
T_2	2	13
T_3	5	15
T_4	1	12

Figure 3 includes two Gantt charts representing the results of applying Max-min and Min-min algorithms on the meta-task M_v , described in Table 1 and Table 2. Although the orders of the tasks scheduled in Max-min and Min-min algorithms are different, the makespan of the system is equal when applying each of the algorithms.

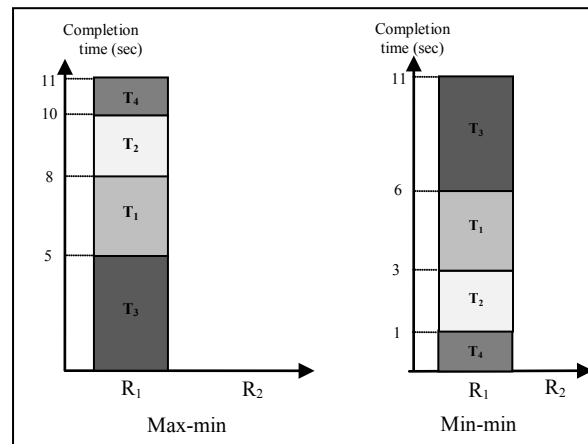


Figure 3- Gantt chart of the Max-min and Min-min algorithms

The result of applying RASA on meta-task M_v is exposed in Figure 4. As shown in Figure 4, the makespan of the system when applying RASA is 10 second, whilst Max-min and Min-min provide a scheduling with a makespan of 11 seconds.

Also, considering the completion times in Figures 3 and 4 it is observed that RASA outperforms Max-min and Min-min algorithms by providing relatively

smaller makespan and higher load balancing. It is assumed that tasks can be executed on any of the resources, independently. Therefore, the QoS guided Min-min and QoS priority grouping algorithms act the same as Min-min algorithm. The above example is only one case which shows the privilege of RASA compared to the other algorithms. However, one can provide an example in which the results of RASA are equal or even worse than the other above mentioned algorithms. In general when the submitted tasks have almost the same size, the Min-min or Max-min algorithms may act better than RASA.

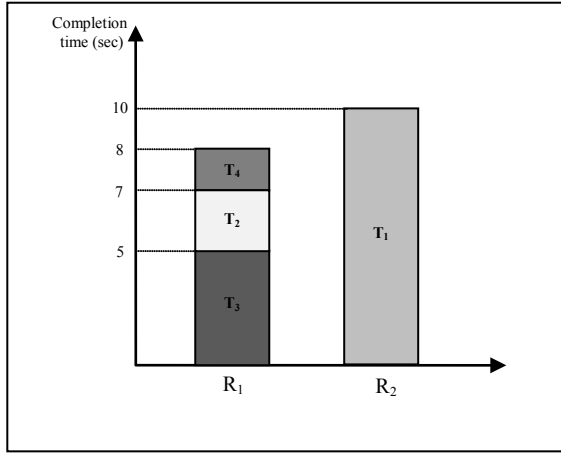


Figure 4- Gantt chart of RASA

5. Simulation and Experimental Results

To evaluate and compare RASA with other algorithms such as Max-min, Min-min, OLB, QoS guided Min-min, and QoS priority grouping, a simulation environment known as GridSim toolkit [17] has been used. In order to demonstrate the preeminence of RASA in comparison with other algorithms, two different assumptions are made:

1. **Assumption I:** the computation time of the tasks overcomes to the communication time of them. This situation occurs in multiprocessors and small scale distributed systems (such as cluster environments).
2. **Assumption II:** the communication time becomes more and even can be overcomes to the computation time of the tasks. This situation occurs in large scale distributed systems such as grid environments in which the resources are widely distributed and connected via the communication links.

The above mentioned algorithms have been simulated in GridSim environment. It has been assumed that there are no constraints for executing

tasks on different resources and each of the tasks could be executed on each of the resources. In this situation, the QoS guided Min-min and QoS priority grouping algorithms performed the same as the Min-min algorithm. Therefore, in all the plots presented in the following figures, only the results of Min-min are depicted and the plots related to QoS guided Min-min and QoS priority grouping algorithms are eliminated. In order to study the effect of the volume of workload on the efficacy of RASA and to compare it with the other algorithms, in each of the above mentioned assumptions, three different workloads of light, medium and heavy load are considered. In light load, 200 tasks are dispatched within 10 to 11 grid resources and makespan is estimated for each of the algorithms. Also, in medium load, 1000 tasks and in heavy load, 5000 tasks have been considered. Figure 5 and Figure 6 show the makespans achieved by applying the algorithms, considering Assumption I and Assumption II, respectively.

As shown in Figure 5-(a), when the computation time of the tasks exceeds their communication time and the workloads of the resources are light, Max-min returns relatively smaller makespan than the other algorithms. As the workload of the resources increases, the makespans achieved by the Min-min and RASA get smaller. As shown in Figure 5-(b) and Figure 5-(c), the makespan returned by RASA is less than the makespans returned by the other algorithms. Considering Figure 5-(b) and 5-(c), it can be concluded that; RASA returns smaller makespans compared with Max-min, Min-min, OLB, QoS guided Min-min, and QoS priority grouping algorithms, in small scale distributed systems.

Figure 6 shows the makespans of the algorithms based on Assumption II. As shown in Figure 6-(a), when the workload is light, the makespan returned by RASA is less than the other algorithms. Comparing Figures 5-(a) and 6-(a), it is observed that; Max-min is suitable for small scale distributed systems whilst Min-min well suits large scale distributed systems. However, RASA returns relatively smaller makespans than both the Min-min and Max-min algorithms in both small scale and large scale distributed systems. When the workload of the resources is heavy, RASA achieves smaller makespans in comparison with the other algorithms. Therefore, RASA achieves smaller makespans both in light and heavy load conditions.

For the sake of clarity, in Figures 5 and 6 the makespans returned by different algorithms are rescaled. Here, the largest makespan, returned by OLB, is 100 and the other makespans are rescaled with respect to this amount. The six different cases observed in Figures 5 and 6 are considered as vertices of a regular hexagon. The vertices of the hexagon are

named as Light 1, Light 2, Medium 1, Medium 2, Heavy 1, and Heavy 2 which imply light load in Assumption I, light load in Assumption II, medium load in Assumption I, medium load in Assumption II, heavy load in Assumption I, and heavy load in Assumption II, respectively. The hexagon is shown in Figure 7. In this hexagon the number of grid resources is assumed to be 10. The number of grid resources, in Figure 8, is 11. As shown in Figures 7 and 8 the area of the non-regular hexagon which is related to RASA, is

smaller than the other algorithms. Therefore, it can be concluded that; the makespans returned by RASA are smaller than the other algorithms in almost all different conditions.

6. Conclusion and Future Works

Min-min and Max-min algorithms are applicable in small scale distributed systems. When the number of the small tasks is more than the number of the large tasks in a meta-task, the Min-min algorithm can not

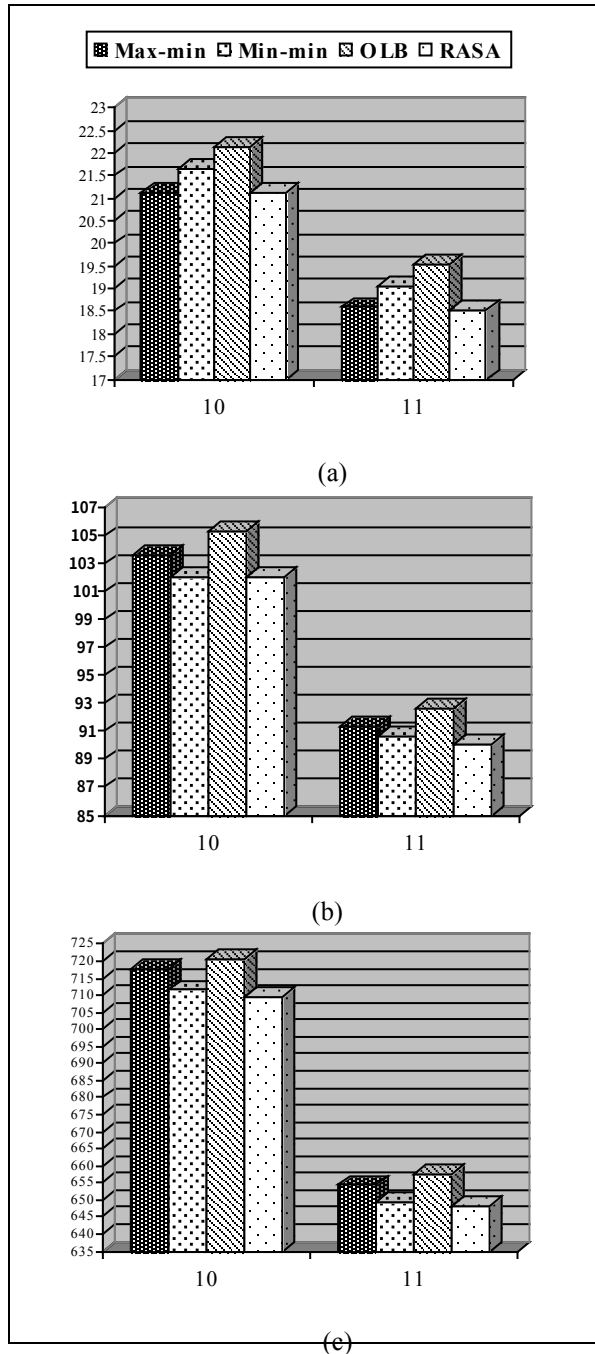


Figure 5- Makespans of the algorithms in Assumption I

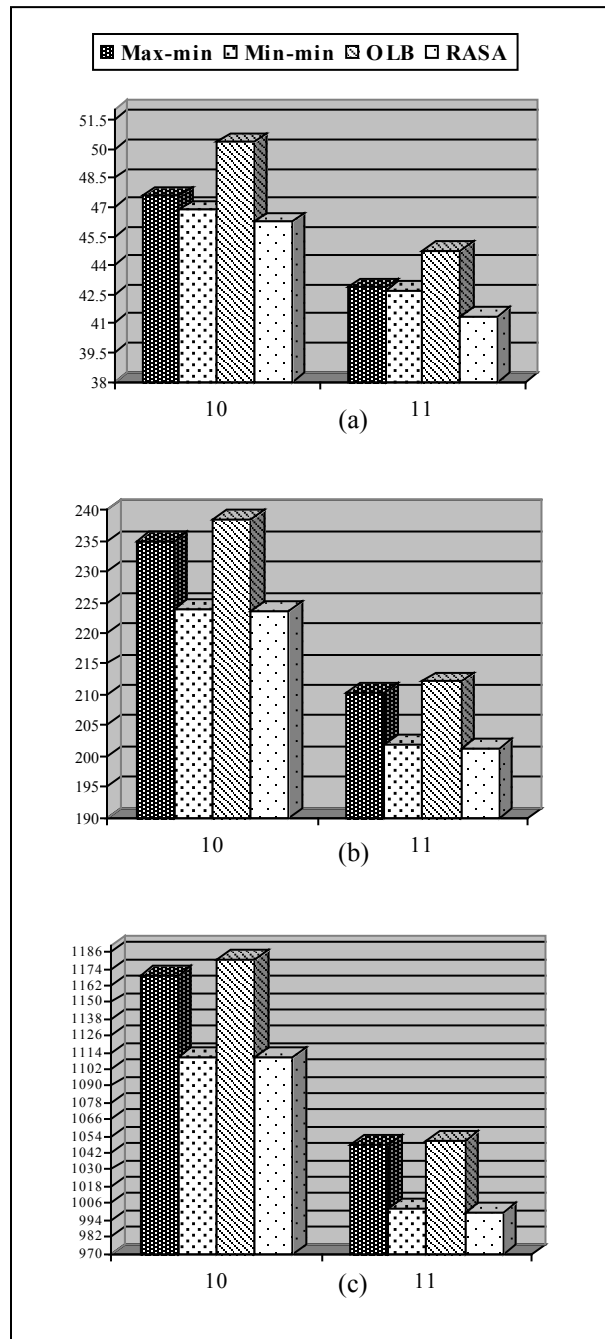


Figure 6- Makespans of the algorithms in Assumption II

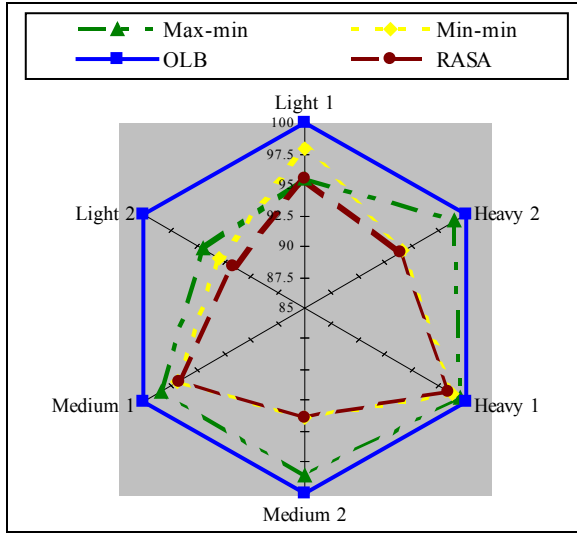


Figure 7 - Makespan of the algorithms when the number of resources is 10

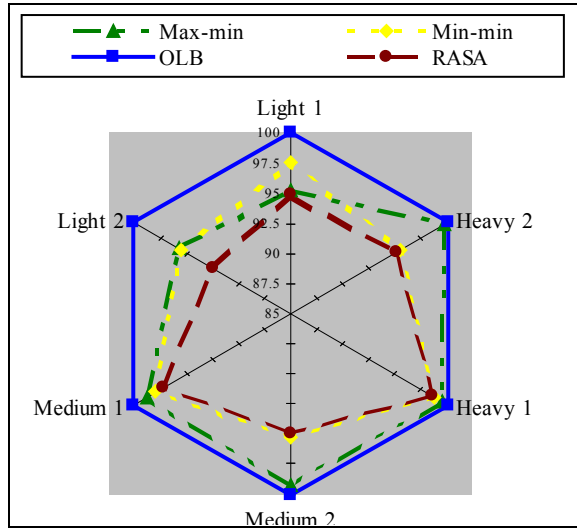


Figure 8 - Makespan of the algorithms when the number of resources is 11

schedule tasks, appropriately, and the makespan of the system gets relatively large. Unlike the Min-min algorithm, the Max-min algorithm attempts to achieve load balancing within resources by scheduling the large tasks prior to the small ones. However, within a computational grid environment high throughput is of great interest rather than the load balancing. To achieve this, in this article, a new task scheduling algorithm, RASA, is proposed.

RASA is composed of two traditional scheduling algorithms; Max-min and Min-min. RASA uses the advantages of Max-min and Min-min algorithms and covers their disadvantages. The experimental results obtained by applying RASA within the GridSim simulator, shows that RASA outperforms the

existing scheduling algorithms in large scale distributed systems.

This study is only concerned with the number of the resources to be odd or even and analyses the merits and drawbacks of two well known traditional algorithms, Max-min and Min-min. In this paper, the deadline of each task, arriving rate of the tasks, cost of the task execution on each of the resource, cost of the communication and many other cases that can be a topic of research are not considered. Also, applying the proposed algorithm on actual grid environment for practical evaluation can be other open problem in this area.

7. Acknowledgment

The authors want to express their gratitude to the Iranian National Elite Foundation and Iran

Telecommunication Research Center for their support of this paper.

Also, the authors would like to express their cordial thanks to Miss Pegah Moradi-Hamed and Miss Marziye Mehdi-Beyraghdar for their valuable assist.

8. References

- [1] I. Foster, and C. Kesselman, The Grid 2: Blueprint for a New Computing Infrastructure, Second Edition, Elsevier and Morgan Kaufmann Press, 2004.
- [2] L. Chunlin, and L. Layuan, "QoS based resource scheduling by computational economy in computational grid," Journal of Information Processing Letters, Vol. 98, pp. 119-126, 2006.
- [3] X. He, X-He Sun, and G. V. Laszewski, "QoS Guided Min-min Heuristic for Grid Task Scheduling," Journal of Computer Science and Technology, Vol. 18, pp. 442-451, 2003.
- [4] L. Mohammad Khanli, and M. Analoui, "Resource Scheduling in Desktop Grid by Grid-JQA," The 3rd International Conference on Grid and Pervasive Computing, IEEE, 2008.
- [5] L. Mohammad Khanli, and M. Analoui, "Grid_JQA: A QoS Guided Scheduling Algorithm for Grid Computing," The Sixth International Symposium on Parallel and Distributed Computing (ISPDC'07), IEEE, 2007.
- [6] E. Elmroth, and J. Tordsson, "Grid resource brokering algorithms enabling advance reservations and resource selection based on performance predictions," Journal of Future Generation Computer Systems, Vol. 24, pp. 585-593, 2008.
- [7] B.T. Benjamin Khoo, B. Veeravalli, T. Hung, and C.W. Simon See, "A multi-dimensional scheduling scheme in a Grid computing environment," Journal of

- Parallel and Distributed Computing, Vol. 67, pp. 659-673, 2007.
- [8] B. Yagoubi, and Y. Slimani, "Task Load Balancing Strategy for Grid Computing," *Journal of Computer Science*, Vol. 3, No. 3, pp. 186-194, 2007.
 - [9] M. Maheswaran, Sh. Ali, H. Jay Siegel, D. Hensgen, and R. F. Freund, "Dynamic Mapping of a Class of Independent Tasks onto Heterogeneous Computing Systems," *Journal of Parallel and Distributed Computing*, Vol. 59, pp. 107-131, 1999.
 - [10] R. F. Freund, M. Gherrity, S. Ambrosius, M. Campbell, M. Halderman, D. Hensgen, E. Keith, T. Kidd, M. Kussow, J. D. Lima, F. Mirabile, L. Moore, B. Rust and H. J. Siegel, "Scheduling Resource in Multi-User, Heterogeneous, Computing Environment with SmartNet," In the Proceeding of the Seventh Heterogeneous Computing Workshop, 1998.
 - [11] T. D. Braun, H. Jay Siegel, N. Beck, L. L. Boloni, M. Maheswaran, A. I. Reuther, J. P. Robertson, M. D. Theys, and B. Yao, "A Comparison of Eleven Static Heuristics for Mapping a Class of Independent Tasks onto Heterogeneous Distributed Computing Systems," *Journal of Parallel and Distributed Computing*, Vol. 61, pp. 810-837, 2001.
 - [12] F. Dong, J. Luo, L. Gao, and L. Ge, "A Grid Task Scheduling Algorithm Based on QoS Priority Grouping," In the Proceedings of the Fifth International Conference on Grid and Cooperative Computing (GCC'06), IEEE, 2006.
 - [13] E. Ullah Munir, J. Li, and Sh. Shi, 2007. QoS Sufferage Heuristic for Independent Task Scheduling in Grid. *Information Technology Journal*, 6 (8): 1166-1170.
 - [14] K. Etminani, and M. Naghibzadeh, "A Min-min Max-min Selective Algorithm for Grid Task Scheduling," The Third IEEE/IFIP International Conference on Internet, Uzbekistan, 2007.
 - [15] A. Afzal, A. Stephen McGough, and J. Darlington, "Capacity planning and scheduling in Grid computing environment," *Journal of Future Generation Computer Systems*, Vol. 24, pp. 404-414, 2008.
 - [16] P. Brucker, *Scheduling Algorithms*, Fifth Edition, Springer Press, 2007.
 - [17] R. Buyya, and M. Murshed, "GridSim: A toolkit for the modeling and simulation of distributed resource management and scheduling for grid computing," *Journal of Concurrency and Computation Practice and Experience*, pp 1175-1220, 2002.



Saeed Parsa received the BS in mathematics and computer Science from Sharif University of Technology, Iran, the MS degrees in computer science from the University of Salford at England, and the Ph.D. degree in computer science from the University of Salford at England. He is an associated professor of computer science at Iran University of Science and Technology. His research interests include software engineering, soft Computing and algorithms.



Reza Entezari-Maleki received the B.Sc. and M.Sc. degree in computer engineering (software) (2007 and 2009, respectively) from department of computer engineering, Iran University of Science and Technology (IUST). He is also a member of Iranian National Elite Foundation. His research interests include grid computing, task scheduling algorithms, and modelling and performance/dependability analysis within grid environments.

西安电子科技大学

毕业设计（论文）指导情况登记表

专业	空间科学与技术	题目	开源鸿蒙操作系统的高效任务调度策略		
学生姓名	魏子捷	学号	20009100883	教师姓名	韩高
指导次数	指导内容	指导效果	指导时间	指导教师签名	
1	指导查找国内外文献资料	教会了使用谷歌学术，知网等网站查找学术论文的方法	2023.12.25		
2	指导进行开发环境搭建	由于系统出现莫名其妙报错无法顺利安装，故拷贝现有环境	2024.1.3		
3	验收环境配置情况，汇报操作系统基础知识学习情况和论文阅读情况	验收环境配置完成，可以正常工作，操作系统学习情况和论文阅读不够扎实	2024.1.10		
4	验收操作系统基础知识学习情况	验收合格，后续结合源代码进行进一步理解	2024.2.26		
5	指导阅读源代码	掌握了配合鸿蒙官方文档，进行源代码阅读分析的方法	2024.3.2		
6	验收任务调度时间片轮转算法代码分析情况	通过讲解发现一些理解上问题，并进行纠正	2024.3.8		
7	进行调度算法方案设计	明确了调度算法初步设计，并进行代码的初步撰写；准备中期检查	2024.3.15		

指导次数	指导内容	指导效果	指导时间	指导教师签名
8	检查中期检查 PPT 和中期报告，检查初步编写的算法代码	检查发现报错，进行了改正	2024.3.22	韩子
9	进行移植实验方案设计	设计出初步方案，布置自学 GD32F303 开发板	2024.4.1	韩子
10	验收开发板学习情况	验收基本掌握移植所需开发板操作，完成代码移植	2024.4.8	韩子
11	检查论文框架设计情况	对论文框架进行修正使其合乎逻辑，后面开始进行论文写作	2024.4.15	韩子
12	检查论文三四章写作情况	对三四章写作初稿提出建议，回去修改	2024.4.22	韩子
13	检查三四章论文初稿修改情况以及其余章节写作情况	提出全文初稿建议，回去修改	2024.4.30	韩子
14	检查论文修改情况	内容无大问题，回去进行格式排版工作和答辩 PPT 制作	2024.5.8	韩子
15	验收终稿和答辩 PPT	论文定稿，答辩 PPT 润色	2024.5.15	韩子
16	进行答辩预演	顺利完成答辩，并通过答辩	2024.5.29	韩子

说明：

1. 本表由学生填写；
2. 学生提出的问题或导师指导的问题均填入指导内容栏内，经教师指导将解决结果记在效果栏内，并请指导教师签名作为记载。
3. 各学院可根据本学院专业认证要求及细则对表格进行修改，修改后的表格需向本科生院备案。